Optimal Control

Varad Vaidya

April 19, 2026

Abstract

These notes are intended to be a self-contained PhD-level reference on optimal control, beginning from the optimization and convex-analysis foundations on which modern control theory rests.

Contents

I Optimization Foundations 4

Lecture 1: Convexity and Unconstrained Optimality 4

Lecture 2: Numerical Methods for Unconstrained Optimization 16

Lecture 3: Constrained Optimization 35

II Dynamic Optimal Control 51

Lecture 4: Dynamic Programming and the Calculus of Optimal Control 51

Lecture 5: Numerical Methods for Trajectory Optimization 59

Lecture 6: Linear Quadratic Regulator 68

Lecture 7: Estimation – The Kalman Filter 82

Lecture 8: LQG Control, Trajectory Optimization, and iLQR 90

Lecture 9: Model Predictive Control 101

A Attitude Control and Quaternion Optimization 109

Part I
Optimization Foundations

Lecture 1: Convexity and Unconstrained Optimality

Notation. Throughout these notes, $\mathbb{R}^n$ is equipped with the standard inner product $x^\top y$ and the Euclidean norm $\|x\|_2$, usually written simply as $\|x\|$. Unless stated otherwise, all optimization problems are finite-dimensional and all domains are subsets of $\mathbb{R}^n$.

Intuition. Optimal control is built on optimization, and optimization becomes structurally transparent once convexity enters the picture. Convex sets describe feasible regions with no "holes" along line segments; convex functions describe landscapes with no spurious local valleys. The first lecture therefore has two jobs:

- build the convex-analysis toolkit that makes later control problems tractable,
- connect those ideas to local and global optimality conditions for unconstrained minimization.

The key theme is that curvature controls geometry, and geometry controls algorithms.

1.1 Convex Sets

Definition 1.1 (Convex set). A set $C \subseteq \mathbb{R}^n$ is *convex* if for every $x, y \in C$ and every $\theta \in [0, 1]$,

$$\theta x + (1 - \theta)y \in C.$$

In words: together with any two points, the set contains the entire line segment joining them.

Remark. The parameter θ should be read as an interpolation parameter. The points $\theta x + (1 - \theta)y$ are exactly the convex combinations of x and y. The distinction between affine combinations and convex combinations matters:

- affine combinations allow coefficients summing to 1 with arbitrary signs,
- convex combinations additionally require nonnegative coefficients.

Convexity is therefore a positivity-restricted affine property.

Proposition 1.1 (Intersection of convex sets). Let $\{C_\alpha\}_{\alpha \in A}$ be any family of convex subsets of $\mathbb{R}^n$, where the index set A may be finite or infinite. Then

$$C = \bigcap_{\alpha \in A} C_\alpha$$

is convex.

Proof. Take any $x, y \in C$. By definition of intersection, $x, y \in C_\alpha$ for every $\alpha \in A$. Since each C_α is convex, for every $\theta \in [0, 1]$ we have

$$\theta x + (1 - \theta)y \in C_\alpha \qquad \text{for every } \alpha \in A.$$

Hence the same point belongs to the intersection of all C_α, so

$$\theta x + (1 - \theta)y \in \bigcap_{\alpha \in A} C_\alpha = C.$$

Therefore C is convex. ■

Example (Basic examples and non-examples). (a) The closed unit ball

$$B = \{x \in \mathbb{R}^n : \|x\| \leq 1\}$$

is convex. Indeed, if $\|x\| \leq 1$ and $\|y\| \leq 1$, then by the triangle inequality

$$\|\theta x + (1-\theta)y\| \leq \theta\|x\| + (1-\theta)\|y\| \leq \theta + (1-\theta) = 1.$$

(b) The unit circle

$$S^1 = \{x \in \mathbb{R}^2 : x_1^2 + x_2^2 = 1\}$$

is not convex. The points $(1,0)$ and $(-1,0)$ belong to S^1, but their midpoint $(0,0)$ does not.

(c) The set

$$\{x \in \mathbb{R}^2 : x_1^2 - x_2^2 \leq 1\}$$

is not convex. Indeed, the points $(\sqrt{2},1)$ and $(\sqrt{2},-1)$ both satisfy $x_1^2 - x_2^2 = 1$, but their midpoint $(\sqrt{2},0)$ satisfies $x_1^2 - x_2^2 = 2 > 1$.

(d) Any affine set

$$\{x \in \mathbb{R}^n : Ax = b\}$$

is convex. If $Ax = b$ and $Ay = b$, then

$$A(\theta x + (1-\theta)y) = \theta Ax + (1-\theta)Ay = \theta b + (1-\theta)b = b.$$

(e) The positive semidefinite cone

$$\mathbb{S}_+^n = \{X \in \mathbb{R}^{n\times n} : X = X^\top,\ X \succeq 0\}$$

is convex. If $X, Y \in \mathbb{S}_+^n$, then $\theta X + (1-\theta)Y$ is symmetric and, for every $v \in \mathbb{R}^n$,

$$v^\top(\theta X + (1-\theta)Y)v = \theta v^\top Xv + (1-\theta)v^\top Yv \geq 0.$$

Remark (Unions are delicate). The statement "the union of two convex sets is convex" is false in general. A standard counterexample is

$$C_1 = \{(x,0) : x \in \mathbb{R}\}, \qquad C_2 = \{(0,y) : y \in \mathbb{R}\}$$

in $\mathbb{R}^2$: each is convex, but their union is a cross, not a line or a plane.

A very common sufficient condition is simple:

- if $C_1 \subseteq C_2$, then $C_1 \cup C_2 = C_2$, which is convex,
- if $C_2 \subseteq C_1$, then $C_1 \cup C_2 = C_1$, which is convex.

This is the condition usually meant in short exam-style answers. It is not necessary in full generality: for example, the union of the two half-spaces $\{x \in \mathbb{R} : x \leq 0\}$ and $\{x \in \mathbb{R} : x \geq 0\}$ is all of $\mathbb{R}$, which is convex even though neither set contains the other.

1.2 Convex Functions

Definition 1.2 (Convex and strictly convex functions). Let $C \subseteq \mathbb{R}^n$ be convex. A function $f : C \to \mathbb{R}$ is *convex* if for all $x, y \in C$ and all $\theta \in [0,1]$,

$$f(\theta x + (1-\theta)y) \leq \theta f(x) + (1-\theta)f(y).$$

It is *strictly convex* if for all distinct $x, y \in C$ and all $\theta \in (0,1)$,

$$f(\theta x + (1-\theta)y) < \theta f(x) + (1-\theta)f(y).$$

Intuition. The graph of a convex function lies below every secant line joining two points on the graph. The graph of a strictly convex function lies strictly below every nontrivial secant line. This is the analytic analogue of a bowl-shaped geometry.

Example (Convex but not strictly convex)**.** The function

$$f(x) = |x|$$

is convex on $\mathbb{R}$ but not strictly convex. Convexity follows from the triangle inequality:

$$|\theta x + (1-\theta)y| \leq \theta|x| + (1-\theta)|y|.$$

It is not strictly convex because equality holds on every interval contained entirely in $(0, \infty)$ or entirely in $(-\infty, 0)$; for example, for $x = 1$, $y = 2$, and $\theta = 1/2$,

$$f\left(\frac{x+y}{2}\right) = \frac{3}{2} = \frac{f(x)+f(y)}{2}.$$

Example (Affine functions)**.** Every affine function

$$f(x) = a^\top x + b$$

is both convex and concave, because

$$f(\theta x + (1-\theta)y) = \theta f(x) + (1-\theta)f(y).$$

Unless the domain is a single point, affine functions are never strictly convex.

Theorem 1.1 (Epigraph characterization)**.** Let $C \subseteq \mathbb{R}^n$ be convex and $f : C \to \mathbb{R}$. The function f is convex if and only if its epigraph

$$\operatorname{epi}(f) = \{(x,t) \in C \times \mathbb{R} : f(x) \leq t\}$$

is a convex subset of $\mathbb{R}^{n+1}$.

Proof. Suppose first that f is convex. Take any $(x,t), (y,s) \in \operatorname{epi}(f)$. Then $f(x) \leq t$ and $f(y) \leq s$. For $\theta \in [0,1]$,

$$f(\theta x + (1-\theta)y) \leq \theta f(x) + (1-\theta)f(y) \leq \theta t + (1-\theta)s.$$

Hence

$$(\theta x + (1-\theta)y,\ \theta t + (1-\theta)s) \in \operatorname{epi}(f),$$

so $\operatorname{epi}(f)$ is convex.

Conversely, suppose $\operatorname{epi}(f)$ is convex. For any $x, y \in C$, the points $(x, f(x))$ and $(y, f(y))$ belong to $\operatorname{epi}(f)$. Therefore, for every $\theta \in [0,1]$,

$$(\theta x + (1-\theta)y,\ \theta f(x) + (1-\theta)f(y)) \in \operatorname{epi}(f).$$

By definition of epigraph,

$$f(\theta x + (1-\theta)y) \leq \theta f(x) + (1-\theta)f(y).$$

So f is convex. ■

Theorem 1.2 (First-order characterization of convexity)**.** Let $C \subseteq \mathbb{R}^n$ be open and convex, and let $f : C \to \mathbb{R}$ be differentiable. Then the following are equivalent:

(i) f is convex on C.

(ii) For all $x, y \in C$,

$$f(y) \geq f(x) + \nabla f(x)^\top (y - x).$$

Proof. Assume first that f is convex. Fix $x, y \in C$, and define

$$\phi(t) = f(x + t(y - x)), \qquad t \in [0, 1].$$

Because C is convex, the segment lies in C. Convexity of f implies convexity of the one-variable function ϕ. For any $t \in (0, 1]$,

$$\phi(t) \leq (1 - t)\phi(0) + t\phi(1).$$

Rearranging gives

$$\frac{\phi(t) - \phi(0)}{t} \leq \phi(1) - \phi(0).$$

Letting $t \downarrow 0$ and using differentiability,

$$\phi'(0) = \nabla f(x)^\top (y - x) \leq f(y) - f(x),$$

which is exactly

$$f(y) \geq f(x) + \nabla f(x)^\top (y - x).$$

Conversely, assume the inequality holds for all $x, y \in C$. Fix $x, y \in C$ and $\theta \in [0, 1]$, and define

$$z = \theta x + (1 - \theta) y.$$

Applying the assumed inequality with base point z and test points x and y yields

$$f(x) \geq f(z) + \nabla f(z)^\top (x - z),$$

$$f(y) \geq f(z) + \nabla f(z)^\top (y - z).$$

Multiply the first relation by θ and the second by $(1 - \theta)$ and add:

$$\theta f(x) + (1 - \theta) f(y) \geq f(z) + \nabla f(z)^\top \big(\theta(x - z) + (1 - \theta)(y - z)\big).$$

But $\theta(x - z) + (1 - \theta)(y - z) = 0$, so

$$f(z) \leq \theta f(x) + (1 - \theta) f(y).$$

Since $z = \theta x + (1 - \theta) y$, this is convexity. ■

Remark (Geometric meaning). The first-order inequality says that every tangent hyperplane to the graph of a differentiable convex function is a global underestimator. In optimization this is decisive: the gradient is no longer merely a local linear approximation, but a certificate that the graph never drops below the tangent plane.

Theorem 1.3 (Second-order characterization). Let $C \subseteq \mathbb{R}^n$ be open and convex, and let $f : C \to \mathbb{R}$ be twice continuously differentiable. Then f is convex on C if and only if

$$\nabla^2 f(x) \succeq 0 \qquad \text{for all } x \in C.$$

Proof. Suppose first that f is convex. Fix $x \in C$ and $d \in \mathbb{R}^n$. Because C is open, for sufficiently small t the point $x + td$ lies in C. Define

$$\phi(t) = f(x + td).$$

The function ϕ is convex on a neighborhood of 0 because it is the restriction of a convex function to a line. Hence $\phi''(0) \geq 0$. By the chain rule,

$$\phi''(0) = d^\top \nabla^2 f(x)\, d.$$

Since this is nonnegative for every d, we conclude that $\nabla^2 f(x) \succeq 0$.

Conversely, suppose $\nabla^2 f(x) \succeq 0$ for every $x \in C$. Fix $x, y \in C$ and define again

$$\phi(t) = f(x + t(y - x)), \qquad t \in [0, 1].$$

Then

$$\phi''(t) = (y - x)^\top \nabla^2 f(x + t(y - x))(y - x) \geq 0$$

for all $t \in [0, 1]$. Hence ϕ is a convex function of one variable. Therefore,

$$\phi(\theta) \leq \theta\phi(1) + (1 - \theta)\phi(0)$$

for every $\theta \in [0, 1]$. Rewriting in terms of f gives

$$f(\theta y + (1 - \theta)x) \leq \theta f(y) + (1 - \theta)f(x),$$

so f is convex. ■

Example (Checking convexity by Hessians). For each function below, the Hessian provides a complete answer.

(a) For $f(x) = 3x^2 - 4x + 7$,

$$f''(x) = 6 > 0,$$

so f is strictly convex.

(b) For $f(x_1, x_2) = x_1^2 + x_1x_2 + x_2^2$,

$$\nabla^2 f = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}.$$

Its eigenvalues are 3 and 1, so the Hessian is positive definite. Hence f is strictly convex.

(c) For $f(x_1, x_2) = x_1^2 - x_1x_2 + x_2^2$,

$$\nabla^2 f = \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix},$$

whose eigenvalues are again 3 and 1. So f is strictly convex.

(d) For $f(x_1, x_2) = x_1^2 - 6x_1x_2 + x_2^2$,

$$\nabla^2 f = \begin{bmatrix} 2 & -6 \\ -6 & 2 \end{bmatrix}.$$

Its eigenvalues are 8 and -4, so the Hessian is indefinite. Therefore f is neither convex nor concave.

Remark (Non-differentiable convexity). Convexity does not imply differentiability. The function $f(x) = |x|$ is convex but has no derivative at $x = 0$. This matters algorithmically:

- classical gradient descent is not directly defined at nondifferentiable points,
- one must instead use subgradient, proximal, bundle, or smoothing methods,
- optimal control formulations with absolute values or norms routinely lead to such nonsmooth structures.

1.3 Operations Preserving Convexity

Proposition 1.2 (Nonnegative weighted sums). If $f_1, f_2 : C \to \mathbb{R}$ are convex on a convex set C, and $\alpha, \beta \geq 0$, then

$$g(x) = \alpha f_1(x) + \beta f_2(x)$$

is convex on C.

Proof. For $x, y \in C$ and $\theta \in [0,1]$,

$$\begin{aligned} g(\theta x + (1-\theta)y) &= \alpha f_1(\theta x + (1-\theta)y) + \beta f_2(\theta x + (1-\theta)y) \\ &\le \alpha\big(\theta f_1(x) + (1-\theta)f_1(y)\big) + \beta\big(\theta f_2(x) + (1-\theta)f_2(y)\big) \\ &= \theta g(x) + (1-\theta)g(y). \end{aligned}$$

■

Proposition 1.3 (Pointwise maxima)**.** If $f_1, f_2 : C \to \mathbb{R}$ are convex, then

$$g(x) = \max\{f_1(x), f_2(x)\}$$

is convex.

Proof. Fix $x, y \in C$ and $\theta \in [0,1]$. Then

$$\begin{aligned} g(\theta x + (1-\theta)y) &= \max\{f_1(\theta x + (1-\theta)y),\ f_2(\theta x + (1-\theta)y)\} \\ &\le \max\{\theta f_1(x) + (1-\theta)f_1(y),\ \theta f_2(x) + (1-\theta)f_2(y)\} \\ &\le \theta \max\{f_1(x), f_2(x)\} + (1-\theta)\max\{f_1(y), f_2(y)\} \\ &= \theta g(x) + (1-\theta)g(y). \end{aligned}$$

■

Proposition 1.4 (Composition with affine maps)**.** If $f : D \to \mathbb{R}$ is convex on a convex set $D \subseteq \mathbb{R}^m$, and if $A \in \mathbb{R}^{m\times n}$ and $b \in \mathbb{R}^m$ are such that $Ax + b \in D$ for every x in a convex set $C \subseteq \mathbb{R}^n$, then

$$h(x) = f(Ax + b)$$

is convex on C.

Proof. Let $x, y \in C$ and $\theta \in [0,1]$. Since affine maps preserve convex combinations,

$$A(\theta x + (1-\theta)y) + b = \theta(Ax + b) + (1-\theta)(Ay + b).$$

Therefore,

$$\begin{aligned} h(\theta x + (1-\theta)y) &= f\big(\theta(Ax + b) + (1-\theta)(Ay + b)\big) \\ &\le \theta f(Ax + b) + (1-\theta)f(Ay + b) \\ &= \theta h(x) + (1-\theta)h(y). \end{aligned}$$

■

Proposition 1.5 (Scalar composition rule)**.** Let $f : C \to I$ be convex, where $C \subseteq \mathbb{R}^n$ is convex and $I \subseteq \mathbb{R}$ is an interval. Let $g : I \to \mathbb{R}$ be convex and nondecreasing. Then

$$h = g \circ f$$

is convex on C.

Proof. For $x, y \in C$ and $\theta \in [0,1]$, convexity of f gives

$$f(\theta x + (1-\theta)y) \le \theta f(x) + (1-\theta)f(y).$$

Because g is nondecreasing,

$$g(f(\theta x + (1-\theta)y)) \le g(\theta f(x) + (1-\theta)f(y)).$$

Now apply convexity of g:

$$g(\theta f(x) + (1-\theta)f(y)) \leq \theta g(f(x)) + (1-\theta)g(f(y)).$$

Thus $h(\theta x + (1-\theta)y) \leq \theta h(x) + (1-\theta)h(y)$. ■

Remark (Why monotonicity is necessary). Convexity of both inner and outer functions is not enough by itself. Take

$$f(x) = x^2, \qquad g(t) = -t.$$

The function g is affine, hence convex, but decreasing. Then

$$(g \circ f)(x) = -x^2$$

is concave, not convex. The monotonicity assumption is therefore structural, not cosmetic.

1.4 Strong Convexity and Conjugates

Definition 1.3 (Strong convexity). A differentiable function $f : C \to \mathbb{R}$, with $C \subseteq \mathbb{R}^n$ convex, is *m-strongly convex* for some $m > 0$ if

$$x \mapsto f(x) - \frac{m}{2}\|x\|^2$$

is convex on C.

Theorem 1.4 (Strong convexity lower bound). If f is differentiable and m-strongly convex on an open convex set C, then for all $x, y \in C$,

$$f(y) \geq f(x) + \nabla f(x)^\top (y-x) + \frac{m}{2}\|y-x\|^2.$$

Proof. Define

$$g(x) = f(x) - \frac{m}{2}\|x\|^2.$$

By assumption, g is convex. Applying the first-order characterization of convexity to g gives

$$g(y) \geq g(x) + \nabla g(x)^\top (y-x).$$

Substituting g and $\nabla g(x) = \nabla f(x) - mx$ yields

$$f(y) - \frac{m}{2}\|y\|^2 \geq f(x) - \frac{m}{2}\|x\|^2 + \big(\nabla f(x) - mx\big)^\top (y-x).$$

Rearranging,

$$f(y) \geq f(x) + \nabla f(x)^\top (y-x) + \frac{m}{2}\big(\|y\|^2 - \|x\|^2 - 2x^\top (y-x)\big).$$

The bracket simplifies to

$$\|y-x\|^2,$$

so the result follows. ■

Corollary 1.1 (Uniqueness of minimizers). If f is strongly convex on a convex set C, then f has at most one global minimizer on C.

Proof. Suppose $x^\star$ and $y^\star$ are two distinct global minimizers. Strong convexity implies, with $\theta = 1/2$,

$$f\left(\frac{x^\star + y^\star}{2}\right) \leq \frac{f(x^\star) + f(y^\star)}{2} - \frac{m}{8}\|x^\star - y^\star\|^2.$$

Since $x^\star \neq y^\star$ and $m > 0$, the final term is strictly negative. Hence

$$f\left(\frac{x^\star + y^\star}{2}\right) < \frac{f(x^\star) + f(y^\star)}{2}.$$

But the right-hand side equals the minimal value, contradicting global minimality of $x^\star$ and $y^\star$. Therefore the minimizer is unique whenever it exists. ■

Corollary 1.2 (Hessian test for strong convexity). If $f \in C^2(C)$ on an open convex set C, then f is m-strongly convex if and only if

$$\nabla^2 f(x) \succeq mI \qquad \text{for all } x \in C.$$

Proof. Apply the second-order convexity criterion to $g(x) = f(x) - \frac{m}{2}\|x\|^2$, whose Hessian is

$$\nabla^2 g(x) = \nabla^2 f(x) - mI.$$

■

Definition 1.4 (Legendre–Fenchel conjugate). For a function $f : \mathbb{R}^n \to \mathbb{R} \cup \{+\infty\}$, its *conjugate* is defined by

$$f^*(y) = \sup_{x \in \mathbb{R}^n} \{y^\top x - f(x)\}.$$

Proposition 1.6 (Convexity of the conjugate). The conjugate f^* is always convex, regardless of whether f itself is convex.

Proof. For each fixed x, the function

$$\ell_x(y) = y^\top x - f(x)$$

is affine in y. The conjugate is the pointwise supremum of this family:

$$f^*(y) = \sup_{x \in \mathbb{R}^n} \ell_x(y).$$

A pointwise supremum of affine functions is convex. Explicitly, for $y_1, y_2 \in \mathbb{R}^n$ and $\theta \in [0, 1]$,

$$\begin{aligned} f^*(\theta y_1 + (1-\theta) y_2) &= \sup_x \{\theta y_1^\top x + (1-\theta) y_2^\top x - f(x)\} \\ &= \sup_x \{\theta \ell_x(y_1) + (1-\theta)\ell_x(y_2)\} \\ &\leq \theta \sup_x \ell_x(y_1) + (1-\theta) \sup_x \ell_x(y_2) \\ &= \theta f^*(y_1) + (1-\theta) f^*(y_2). \end{aligned}$$

■

Example (Conjugate of a quadratic form). Let

$$f(x) = \frac{1}{2} x^\top Q x, \qquad Q \succ 0.$$

Then

$$f^*(y) = \sup_x \left\{ y^\top x - \frac{1}{2} x^\top Q x \right\}.$$

The expression inside the braces is strictly concave in x, so its maximizer is found from the first-order condition

$$y - Qx = 0 \qquad \Longrightarrow \qquad x = Q^{-1} y.$$

Substituting this maximizing x gives

$$f^*(y) = y^\top Q^{-1} y - \frac{1}{2} y^\top Q^{-1} Q Q^{-1} y = \frac{1}{2} y^\top Q^{-1} y.$$

So the conjugate of a positive definite quadratic is again a positive definite quadratic.

1.5 Optimality Conditions for Unconstrained Problems

Definition 1.5 (Local and global minima). Let $f : C \to \mathbb{R}$ and let $x^\star \in C$.

- $x^\star$ is a *local minimizer* if there exists $r > 0$ such that

$$f(x^\star) \leq f(x) \qquad \text{for all } x \in C \cap B_r(x^\star).$$

- $x^\star$ is a *strict local minimizer* if the inequality is strict for all $x \neq x^\star$ sufficiently close to $x^\star$.
- $x^\star$ is a *global minimizer* if

$$f(x^\star) \leq f(x) \qquad \text{for all } x \in C.$$

Definition 1.6 (Stationary point). If f is differentiable, a point $x^\star$ with

$$\nabla f(x^\star) = 0$$

is called a *stationary* or *critical* point.

Theorem 1.5 (First-order necessary condition). Let $f : U \to \mathbb{R}$ be continuously differentiable on an open set $U \subseteq \mathbb{R}^n$. If $x^\star \in U$ is a local minimizer of f, then

$$\nabla f(x^\star) = 0.$$

Proof. Fix any direction $d \in \mathbb{R}^n$, and define

$$\phi(t) = f(x^\star + td).$$

Because U is open, ϕ is well defined for sufficiently small t. Since $x^\star$ is a local minimizer of f, the point $t = 0$ is a local minimizer of the scalar function ϕ. Therefore

$$\phi'(0) = 0.$$

By the chain rule,

$$\phi'(0) = \nabla f(x^\star)^\top d.$$

This holds for every direction d. The only vector orthogonal to every d is the zero vector, so $\nabla f(x^\star) = 0$. ■

Theorem 1.6 (Second-order necessary and sufficient conditions). Let $f : U \to \mathbb{R}$ be twice continuously differentiable on an open set $U \subseteq \mathbb{R}^n$.

(i) If $x^\star$ is a local minimizer, then

$$\nabla f(x^\star) = 0, \qquad \nabla^2 f(x^\star) \succeq 0.$$

(ii) If

$$\nabla f(x^\star) = 0, \qquad \nabla^2 f(x^\star) \succ 0,$$

then $x^\star$ is a strict local minimizer.

Proof. Part (i): the first assertion is the first-order necessary condition. To prove positive semidef-

initeness of the Hessian, fix any direction d and again consider

$$\phi(t) = f(x^\star + td).$$

Since $t = 0$ is a local minimizer of the scalar function ϕ, we have $\phi''(0) \geq 0$. The chain rule gives

$$\phi''(0) = d^\top \nabla^2 f(x^\star) d.$$

Because this is nonnegative for every d, the Hessian is positive semidefinite.

Part (ii): let $\lambda_{\min} > 0$ be the smallest eigenvalue of $\nabla^2 f(x^\star)$. Then

$$d^\top \nabla^2 f(x^\star) d \geq \lambda_{\min} \|d\|^2 \qquad \text{for all } d.$$

By continuity of the Hessian, there exists $r > 0$ such that for every x with $\|x - x^\star\| < r$,

$$\nabla^2 f(x) \succeq \frac{\lambda_{\min}}{2} I.$$

Now Taylor's theorem with remainder gives, for x near $x^\star$,

$$f(x) = f(x^\star) + \nabla f(x^\star)^\top (x - x^\star) + \frac{1}{2}(x - x^\star)^\top \nabla^2 f(\xi)(x - x^\star)$$

for some point ξ on the segment between $x^\star$ and x. Since $\nabla f(x^\star) = 0$,

$$f(x) - f(x^\star) \geq \frac{1}{2} \cdot \frac{\lambda_{\min}}{2} \|x - x^\star\|^2 = \frac{\lambda_{\min}}{4} \|x - x^\star\|^2$$

for all $x \neq x^\star$ sufficiently close to $x^\star$. Hence $x^\star$ is a strict local minimizer. ■

Remark (The gap between necessary and sufficient conditions)**.** The second-order necessary condition $\nabla^2 f(x^\star) \succeq 0$ is not sufficient for local minimality.

- For $f(x) = x^3$, we have $f'(0) = 0$ and $f''(0) = 0$, but $x = 0$ is neither a local minimum nor a local maximum.
- For $f(x) = x^4$, we again have $f'(0) = 0$ and $f''(0) = 0$, but now $x = 0$ *is* a strict local minimum.

So a semidefinite Hessian leaves unresolved higher-order behavior.

Theorem 1.7 (Local minima of convex functions are global)**.** Let $C \subseteq \mathbb{R}^n$ be convex and let $f : C \to \mathbb{R}$ be convex. Every local minimizer of f is a global minimizer.

Proof. Let $x^\star$ be a local minimizer, and suppose for contradiction that it is not global. Then there exists $y \in C$ with

$$f(y) < f(x^\star).$$

For $\theta \in (0, 1)$, define

$$x_\theta = (1 - \theta)x^\star + \theta y.$$

By convexity,

$$f(x_\theta) \leq (1 - \theta) f(x^\star) + \theta f(y) < (1 - \theta) f(x^\star) + \theta f(x^\star) = f(x^\star).$$

As $\theta \downarrow 0$, the points x_θ converge to $x^\star$. Thus arbitrarily close to $x^\star$ there are points with strictly smaller objective value, contradicting local minimality. Hence $x^\star$ must be global. ■

Proposition 1.7 (Indefinite Hessian implies saddle point)**.** Let $f \in C^2(U)$ and let $x^\star \in U$ satisfy

$$\nabla f(x^\star) = 0.$$

If the Hessian $\nabla^2 f(x^\star)$ has both a positive and a negative eigenvalue, then $x^\star$ is a saddle point.

Proof. Choose unit eigenvectors v_+ and v_- corresponding to eigenvalues $\lambda_+ > 0$ and $\lambda_- < 0$, respectively. Taylor expansion along those directions gives, for sufficiently small t,

$$f(x^\star + tv_+) = f(x^\star) + \frac{t^2}{2}\lambda_+ + o(t^2),$$

$$f(x^\star + tv_-) = f(x^\star) + \frac{t^2}{2}\lambda_- + o(t^2).$$

For sufficiently small nonzero t, the first expression is strictly larger than $f(x^\star)$ and the second strictly smaller. Therefore every neighborhood of $x^\star$ contains points with both larger and smaller objective values, so $x^\star$ is a saddle point. ■

Definition 1.7 (Coercive function)**.** A function $f : \mathbb{R}^n \to \mathbb{R}$ is *coercive* if

$$f(x) \to +\infty \qquad \text{as } \|x\| \to \infty.$$

Theorem 1.8 (Existence of global minimizers under coercivity)**.** If $f : \mathbb{R}^n \to \mathbb{R}$ is continuous and coercive, then f attains a global minimum on $\mathbb{R}^n$.

Proof. Choose any point $x_0 \in \mathbb{R}^n$. By coercivity, there exists $R > 0$ such that

$$\|x\| \geq R \Rightarrow f(x) \geq f(x_0).$$

Thus every point with objective value smaller than $f(x_0)$ lies in the closed ball $\overline{B_R(0)}$, which is compact. The continuous function f therefore attains its minimum on $\overline{B_R(0)}$ by the Weierstrass theorem; call the minimizer $\bar{x}$. For points outside the ball, the function value is at least $f(x_0)$ and hence at least as large as the minimum attained inside the ball. Therefore $\bar{x}$ is a global minimizer on all of $\mathbb{R}^n$. ■

Remark (Even-degree polynomials)**.** If a polynomial has even degree and positive leading behavior in the sense that $f(x) \to +\infty$ as $\|x\| \to \infty$, then it is coercive and therefore attains a global minimum. The minimizer need not be unique: for

$$f(x) = (x^2 - 1)^2,$$

both $x = 1$ and $x = -1$ are global minimizers.

1.6 Worked Examples

Example (A stationary point with semidefinite Hessian that is not a minimum)**.** Consider

$$f(x_1, x_2) = x_1^2 + x_2^3.$$

The gradient is

$$\nabla f(x_1, x_2) = \begin{bmatrix} 2x_1 \\ 3x_2^2 \end{bmatrix},$$

so the only stationary point is $(0, 0)$. The Hessian is

$$\nabla^2 f(x_1, x_2) = \begin{bmatrix} 2 & 0 \\ 0 & 6x_2 \end{bmatrix},$$

and therefore

$$\nabla^2 f(0, 0) = \begin{bmatrix} 2 & 0 \\ 0 & 0 \end{bmatrix} \succeq 0.$$

Nevertheless $(0,0)$ is not a local minimum, because along the curve $x_1 = 0$ we have

$$f(0, x_2) = x_2^3,$$

which is negative for $x_2 < 0$ and positive for $x_2 > 0$. Hence $(0,0)$ is a saddle point. This is a standard multivariable counterexample to the incorrect claim that a positive semidefinite Hessian at a stationary point guarantees local optimality.

Example (Classifying a quadratic critical point)**.** Consider

$$f(x_1, x_2) = 2x_1^2 + x_2^2 - 2x_1x_2 + 4x_1 - 6x_2.$$

The gradient is

$$\nabla f(x_1, x_2) = \begin{bmatrix} 4x_1 - 2x_2 + 4 \\ 2x_2 - 2x_1 - 6 \end{bmatrix}.$$

Solving $\nabla f = 0$ gives

$$2x_1 - x_2 = -2, \qquad -x_1 + x_2 = 3,$$

so

$$x^\star = (-1, 2).$$

The Hessian is constant:

$$\nabla^2 f = \begin{bmatrix} 4 & -2 \\ -2 & 2 \end{bmatrix}.$$

Its leading principal minors are $4 > 0$ and $4 \cdot 2 - (-2)^2 = 4 > 0$, so the Hessian is positive definite. Therefore $x^\star$ is a strict local minimum. Because f is a convex quadratic, this minimum is automatically global. The optimal value is

$$f(-1, 2) = 2(1) + 4 - 2(-1)(2) + 4(-1) - 6(2) = -6.$$

Example (A quartic with two global minima)**.** Let

$$f(x) = x^4 - 4x^2 + 3.$$

Then

$$f'(x) = 4x^3 - 8x = 4x(x^2 - 2),$$

so the critical points are

$$x = 0, \qquad x = \pm\sqrt{2}.$$

Further,

$$f''(x) = 12x^2 - 8.$$

Thus

$$f''(0) = -8 < 0,$$

so $x = 0$ is a strict local maximum, while

$$f''(\pm\sqrt{2}) = 16 > 0,$$

so $x = \pm\sqrt{2}$ are strict local minima. Since the leading term is x^4, the function is coercive, so these local minima are in fact global minima. Their common value is

$$f(\pm\sqrt{2}) = 4 - 8 + 3 = -1.$$

Example (General quadratic optimization)**.** Consider

$$f(x) = \frac{1}{2}x^\top Qx - b^\top x,$$

where $Q \in \mathbb{R}^{n\times n}$ is symmetric. Then

$$\nabla f(x) = Qx - b, \qquad \nabla^2 f(x) = Q.$$

A stationary point satisfies

$$Qx = b.$$

If $Q \succ 0$, then Q is invertible, the stationary point is unique,

$$x^\star = Q^{-1}b,$$

and positive definiteness of the Hessian shows that $x^\star$ is the unique global minimizer. Substituting back,

$$f(x^\star) = \frac{1}{2}b^\top Q^{-1}b - b^\top Q^{-1}b = -\frac{1}{2}b^\top Q^{-1}b.$$

If $Q \succeq 0$ but not positive definite, then the function is convex but may fail to have a unique minimizer. It has a minimizer exactly when $b \in \operatorname{range}(Q)$; in that case the set of minimizers is an affine space

$$x^\star + \operatorname{null}(Q).$$

Example (An analytic scalar minimization). For

$$f(x) = e^x - 2x,$$

we compute

$$f'(x) = e^x - 2, \qquad f''(x) = e^x.$$

The first-order condition gives the unique stationary point

$$e^x = 2 \qquad \Longrightarrow \qquad x^\star = \ln 2.$$

Since $f''(x^\star) = 2 > 0$, the point $x^\star$ is a strict local minimum. In fact f is strictly convex on all of $\mathbb{R}$ because $f''(x) > 0$ everywhere, so the local minimum is global. The minimal value is

$$f(\ln 2) = 2 - 2\ln 2.$$

Remark. Lecture 1 establishes the three structural facts used repeatedly later:

(i) convexity turns local statements into global ones,

(ii) strong convexity gives uniqueness and quantitative curvature,

(iii) Hessian information is decisive when it is strictly positive or genuinely indefinite, but semidefinite cases require extra analysis.

These themes return in line search, Newton methods, Riccati theory, and model predictive control.

Lecture 2: Numerical Methods for Unconstrained Optimization

Intuition. Lecture 1 answered the question "what does optimality look like?" Lecture 2 addresses the harder computational question: "how do we actually get there?" The answer is not a single algorithm but a hierarchy:

- line search supplies a generic descent framework,
- gradient methods give robust but often slow progress,
- Newton methods use curvature for rapid local convergence,

- quasi-Newton methods aim for most of Newton's speed without explicit Hessians.

The theoretical thread running through all of them is simple: obtain a provable decrease at each step, then convert that decrease into a convergence statement.

2.1 Descent Directions and the Line Search Framework

Definition 2.1 (Descent direction). Let $f : U \to \mathbb{R}$ be differentiable on an open set $U \subseteq \mathbb{R}^n$, and let $x \in U$. A vector $d \in \mathbb{R}^n$ is called a *descent direction* for f at x if there exists $\bar{\alpha} > 0$ such that

$$f(x + \alpha d) < f(x) \qquad \text{for all } \alpha \in (0, \bar{\alpha}].$$

Proposition 2.1 (Characterization of descent directions). Let f be differentiable at x. A vector d is a descent direction at x if and only if

$$\nabla f(x)^\top d < 0.$$

Proof. Define the one-dimensional restriction

$$\phi(\alpha) = f(x + \alpha d).$$

Then

$$\phi'(0) = \nabla f(x)^\top d.$$

If $\nabla f(x)^\top d < 0$, then $\phi'(0) < 0$, so for sufficiently small $\alpha > 0$ we have $\phi(\alpha) < \phi(0)$; hence d is a descent direction.

Conversely, if d is a descent direction, then $\phi(\alpha) < \phi(0)$ for all sufficiently small positive α. Therefore

$$\frac{\phi(\alpha) - \phi(0)}{\alpha} < 0$$

for such α, and letting $\alpha \downarrow 0$ gives

$$\phi'(0) = \nabla f(x)^\top d \leq 0.$$

Equality cannot occur, because if $\phi'(0) = 0$ then the function cannot decrease linearly to first order along d. Thus strict descent requires $\nabla f(x)^\top d < 0$. ■

Notation (Generic line search iteration). The standard line search update has the form

$$x_{k+1} = x_k + \alpha_k d_k,$$

where two choices must be made at iteration k:

- a direction d_k, which should point toward lower objective values,
- a step length $\alpha_k > 0$, which controls how far we move along that direction.

A good direction with a terrible step length can stagnate, and a good step length along a poor direction cannot repair a bad search direction.

Algorithm 1: Generic Line Search Method

Input: Initial point x_0, direction rule, step-length rule, tolerance $\varepsilon > 0$

```
for k = 0, 1, 2, ... do
    Compute g_k = ∇f(x_k)
    if ||g_k|| ≤ ε then
        return x_k
    Choose a descent direction d_k such that g_k^⊤ d_k < 0
    Choose a step length α_k > 0
    Set x_{k+1} = x_k + α_k d_k
```

Definition 2.2 (Armijo condition). Given constants $c_1 \in (0,1)$, the step length $\alpha_k > 0$ satisfies the *Armijo* or *sufficient decrease* condition if

$$f(x_k + \alpha_k d_k) \leq f(x_k) + c_1 \alpha_k \nabla f(x_k)^\top d_k.$$

Intuition. Since $\nabla f(x_k)^\top d_k < 0$, the right-hand side lies below $f(x_k)$. The Armijo condition therefore demands not merely a decrease, but a decrease proportional to the first-order model prediction. It rejects steps that are so tiny that they make negligible progress.

Definition 2.3 (Wolfe conditions). Let $0 < c_1 < c_2 < 1$. A step length $\alpha_k > 0$ satisfies the *Wolfe conditions* if

$$\begin{aligned} f(x_k + \alpha_k d_k) &\leq f(x_k) + c_1 \alpha_k \nabla f(x_k)^\top d_k, \\ \nabla f(x_k + \alpha_k d_k)^\top d_k &\geq c_2 \nabla f(x_k)^\top d_k. \end{aligned}$$

The first inequality is Armijo; the second is the *curvature condition*.

Remark (Why Armijo alone is not enough). Armijo guarantees decrease, but it does not prevent *arbitrarily small* accepted steps. If one keeps taking vanishingly small steps, the iterates may decrease the objective without making meaningful geometric progress. The curvature condition blocks this pathology by insisting that the directional derivative after the step is not too negative, which means the step has advanced far enough along the chosen direction.

Remark (Why c_1 is chosen small). The Armijo condition compares actual decrease to the linear prediction

$$f(x_k) + \alpha \nabla f(x_k)^\top d_k.$$

Since the linear model is accurate only for small α, one typically asks for only a small fraction of that predicted decrease, hence values such as $c_1 = 10^{-4}$. If c_1 were close to 1, the condition would demand an unrealistically large fraction of the linear-model decrease and would reject many reasonable steps. At $c_1 = 1$, the inequality would ask the actual objective value to lie below the full first-order prediction, which is generally impossible once second-order curvature is taken into account.

Algorithm 2: Backtracking Line Search

Input: Current iterate x_k, descent direction d_k, initial trial step $\alpha_0 > 0$, reduction factor $\rho \in (0,1)$, Armijo constant $c_1 \in (0,1)$

```
α ← α_0
while f(x_k + α d_k) > f(x_k) + c_1 α ∇f(x_k)^⊤ d_k do
    α ← ρα
```

Output: Accepted step size $\alpha_k = \alpha$

Proposition 2.2 (Finite termination of backtracking). Suppose ∇f is Lipschitz continuous with constant L on a neighborhood of the line segment $\{x_k + \alpha d_k : 0 \leq \alpha \leq \alpha_0\}$, and suppose d_k is a descent direction. Then the backtracking line search terminates after finitely many reductions.

Proof. By the descent lemma proved below,

$$f(x_k + \alpha d_k) \leq f(x_k) + \alpha \nabla f(x_k)^\top d_k + \frac{L}{2}\alpha^2 \|d_k\|^2.$$

Therefore Armijo is guaranteed whenever

$$\alpha \nabla f(x_k)^\top d_k + \frac{L}{2}\alpha^2 \|d_k\|^2 \leq c_1 \alpha \nabla f(x_k)^\top d_k,$$

or equivalently

$$\frac{L}{2}\alpha\|d_k\|^2 \le -(1-c_1)\nabla f(x_k)^\top d_k.$$

Because d_k is a descent direction, the right-hand side is strictly positive. Hence every step size satisfying

$$0 < \alpha \le \frac{2(1-c_1)\big(-\nabla f(x_k)^\top d_k\big)}{L\|d_k\|^2}$$

is accepted. Repeated multiplication by $\rho \in (0,1)$ eventually makes α fall into this interval, so the algorithm terminates in finitely many steps. ■

Example (Exact line search in one dimension). (a) Let $f(x) = (x-3)^2$, $x_0 = 0$, and $d_0 = 1$. Then

$$\phi(\alpha) = f(x_0 + \alpha d_0) = (\alpha - 3)^2.$$

Differentiating,

$$\phi'(\alpha) = 2(\alpha - 3),$$

so the exact line search step is

$$\alpha_0^* = 3.$$

(b) Let $f(x) = x^4 - 4x^2$, $x_0 = 3$, and $d_0 = -1$. Then

$$\phi(\alpha) = f(3-\alpha) = (3-\alpha)^4 - 4(3-\alpha)^2.$$

Differentiating,

$$\phi'(\alpha) = -4(3-\alpha)^3 + 8(3-\alpha) = -4(3-\alpha)\big((3-\alpha)^2 - 2\big).$$

Hence the stationary points are

$$\alpha = 3, \qquad \alpha = 3 - \sqrt{2}, \qquad \alpha = 3 + \sqrt{2}.$$

Evaluating the objective,

$$f(0) = 0, \qquad f(\pm\sqrt{2}) = -4.$$

Therefore the exact line search minimizers on the ray are

$$\alpha^* \in \{3 - \sqrt{2},\, 3 + \sqrt{2}\}.$$

The smaller choice reaches the first global minimizer encountered along the ray.

Proposition 2.3 (Exact line search for convex quadratics). Consider

$$f(x) = \frac{1}{2}x^\top Qx - b^\top x, \qquad Q \succ 0,$$

and a search direction d with $\nabla f(x)^\top d < 0$. Then the exact line search step size along d is

$$\alpha^* = -\frac{\nabla f(x)^\top d}{d^\top Q d}.$$

Proof. Define

$$\phi(\alpha) = f(x + \alpha d).$$

Expanding,

$$\phi(\alpha) = f(x) + \alpha\nabla f(x)^\top d + \frac{\alpha^2}{2}d^\top Q d.$$

Since $Q \succ 0$, we have $d^\top Q d > 0$ for every nonzero d, so ϕ is a strictly convex quadratic in α. Its

unique minimizer is obtained from

$$\phi'(\alpha) = \nabla f(x)^\top d + \alpha d^\top Q d = 0,$$

which yields

$$\alpha^* = -\frac{\nabla f(x)^\top d}{d^\top Q d}.$$

■

2.2 Steepest Descent

Definition 2.4 (Steepest descent iteration)**.** The *steepest descent* direction at x_k is

$$d_k = -\nabla f(x_k).$$

The resulting iteration is

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k).$$

In optimization literature, this is also called *gradient descent*.

Proposition 2.4 (Why the negative gradient is the steepest descent direction)**.** Among all unit vectors d with $\|d\| = 1$, the quantity $\nabla f(x_k)^\top d$ is minimized by

$$d = -\frac{\nabla f(x_k)}{\|\nabla f(x_k)\|}.$$

In particular, $-\nabla f(x_k)$ points in the direction of maximal first-order decrease.

Proof. By Cauchy–Schwarz,

$$\nabla f(x_k)^\top d \geq -\|\nabla f(x_k)\| \, \|d\| = -\|\nabla f(x_k)\|.$$

Equality holds if and only if d is the negative unit vector in the gradient direction, namely

$$d = -\frac{\nabla f(x_k)}{\|\nabla f(x_k)\|}.$$

■

Algorithm 3: Steepest Descent with Line Search

Input: Initial point x_0, step selection rule, tolerance $\varepsilon > 0$

```
for k = 0, 1, 2, ... do
    g_k ← ∇f(x_k)
    if ‖g_k‖ ≤ ε then
        return x_k
    d_k ← −g_k
    Choose α_k > 0 by exact line search, backtracking, or Wolfe line search
    x_{k+1} ← x_k + α_k d_k
```

Theorem 2.1 (Exact line search step for a quadratic)**.** Let

$$f(x) = \frac{1}{2} x^\top Q x - b^\top x, \qquad Q \succ 0,$$

and let

$$g_k = \nabla f(x_k) = Q x_k - b.$$

For steepest descent with exact line search,

$$\alpha_k = \frac{g_k^\top g_k}{g_k^\top Q g_k}.$$

Proof. In steepest descent, $d_k = -g_k$. The general quadratic line-search formula gives

$$\alpha_k = -\frac{g_k^\top d_k}{d_k^\top Q d_k} = -\frac{g_k^\top(-g_k)}{(-g_k)^\top Q(-g_k)} = \frac{g_k^\top g_k}{g_k^\top Q g_k}.$$

■

Example (Two steepest descent steps and the zig-zag phenomenon). Consider

$$f(x_1, x_2) = 5x_1^2 + x_2^2 = \frac{1}{2}\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}^\top \begin{bmatrix} 10 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix},$$

starting from

$$x_0 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

The gradient is

$$\nabla f(x) = \begin{bmatrix} 10x_1 \\ 2x_2 \end{bmatrix}.$$

Thus

$$g_0 = \begin{bmatrix} 10 \\ 2 \end{bmatrix}, \qquad d_0 = -g_0 = \begin{bmatrix} -10 \\ -2 \end{bmatrix}.$$

The exact line-search step is

$$\alpha_0 = \frac{g_0^\top g_0}{g_0^\top Q g_0} = \frac{10^2 + 2^2}{10 \cdot 10^2 + 2 \cdot 2^2} = \frac{104}{1008} = \frac{13}{126}.$$

Hence

$$x_1 = x_0 - \alpha_0 g_0 = \begin{bmatrix} 1 \\ 1 \end{bmatrix} - \frac{13}{126}\begin{bmatrix} 10 \\ 2 \end{bmatrix} = \begin{bmatrix} -\frac{2}{63} \\ \frac{50}{63} \end{bmatrix}.$$

Now

$$g_1 = \begin{bmatrix} -\frac{20}{63} \\ \frac{100}{63} \end{bmatrix}, \qquad d_1 = -g_1.$$

The second exact step is

$$\alpha_1 = \frac{g_1^\top g_1}{g_1^\top Q g_1} = \frac{13}{30}.$$

Therefore

$$x_2 = x_1 - \alpha_1 g_1 = \begin{bmatrix} \frac{20}{189} \\ \frac{20}{189} \end{bmatrix}.$$

The first step moves mostly in the x_1 direction because curvature there is much larger; the next step swings toward the x_2 direction. This alternating behavior is the classical zig-zag pattern.

Proposition 2.5 (Orthogonality of consecutive gradients). For steepest descent with exact line search,

$$g_{k+1}^\top g_k = 0 \qquad \text{for all } k.$$

Proof. Define the line-search function

$$\phi_k(\alpha) = f(x_k - \alpha g_k).$$

Exact line search chooses α_k so that $\phi_k'(\alpha_k) = 0$. By the chain rule,

$$\phi_k'(\alpha) = \nabla f(x_k - \alpha g_k)^\top(-g_k).$$

Evaluating at $\alpha = \alpha_k$ gives

$$0 = \phi_k'(\alpha_k) = -\nabla f(x_{k+1})^\top g_k = -g_{k+1}^\top g_k.$$

Hence $g_{k+1}^\top g_k = 0$. ■

Lemma 2.1 (Kantorovich inequality in scalar form)**.** Let $Q \succ 0$ with eigenvalues in the interval $[m, M]$, where $0 < m \leq M$. Then for every nonzero vector z,

$$\frac{(z^\top z)^2}{(z^\top Qz)(z^\top Q^{-1}z)} \geq \frac{4mM}{(m+M)^2}.$$

Proof. Write the spectral decomposition $Q = U\Lambda U^\top$, where $\Lambda = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$ and $\lambda_i \in [m, M]$. Let $w = U^\top z$ and define

$$p_i = \frac{w_i^2}{\sum_j w_j^2}.$$

Then $p_i \geq 0$ and $\sum_i p_i = 1$, while

$$\frac{(z^\top Qz)(z^\top Q^{-1}z)}{(z^\top z)^2} = \left(\sum_i p_i\lambda_i\right)\left(\sum_i p_i\frac{1}{\lambda_i}\right).$$

For each $\lambda_i \in [m, M]$ we have

$$(M - \lambda_i)(\lambda_i - m) \geq 0.$$

After dividing by $\lambda_i > 0$, this becomes

$$\lambda_i + \frac{mM}{\lambda_i} \leq m + M.$$

Multiply by p_i and sum:

$$\sum_i p_i\lambda_i + mM\sum_i p_i\frac{1}{\lambda_i} \leq m + M.$$

Now apply the arithmetic-geometric mean inequality:

$$2\sqrt{mM\left(\sum_i p_i\lambda_i\right)\left(\sum_i p_i\frac{1}{\lambda_i}\right)} \leq \sum_i p_i\lambda_i + mM\sum_i p_i\frac{1}{\lambda_i} \leq m + M.$$

Squaring yields

$$\left(\sum_i p_i\lambda_i\right)\left(\sum_i p_i\frac{1}{\lambda_i}\right) \leq \frac{(m+M)^2}{4mM}.$$

Taking reciprocals proves the stated inequality. ■

Theorem 2.2 (Linear convergence rate for steepest descent on quadratics)**.** Let

$$f(x) = \frac{1}{2}x^\top Qx - b^\top x, \qquad Q \succ 0,$$

and let $x^\star = Q^{-1}b$. Denote the condition number by

$$\kappa(Q) = \frac{\lambda_{\max}(Q)}{\lambda_{\min}(Q)}.$$

Then steepest descent with exact line search satisfies

$$\frac{f(x_{k+1}) - f(x^\star)}{f(x_k) - f(x^\star)} \le \left(\frac{\kappa(Q) - 1}{\kappa(Q) + 1}\right)^2.$$

In particular, the method converges linearly.

Proof. Let

$$e_k = x_k - x^\star.$$

Because $Qx^\star = b$,

$$g_k = \nabla f(x_k) = Qe_k.$$

Also,

$$f(x_k) - f(x^\star) = \frac{1}{2} e_k^\top Q e_k = \frac{1}{2} g_k^\top Q^{-1} g_k.$$

Using the exact line-search step,

$$x_{k+1} = x_k - \alpha_k g_k, \qquad \alpha_k = \frac{g_k^\top g_k}{g_k^\top Q g_k}.$$

Substituting into the quadratic expression for the objective,

$$f(x_{k+1}) - f(x^\star) = f(x_k) - f(x^\star) - \frac{(g_k^\top g_k)^2}{2 g_k^\top Q g_k}.$$

Divide by $f(x_k) - f(x^\star) = \frac{1}{2} g_k^\top Q^{-1} g_k$:

$$\frac{f(x_{k+1}) - f(x^\star)}{f(x_k) - f(x^\star)} = 1 - \frac{(g_k^\top g_k)^2}{(g_k^\top Q g_k)(g_k^\top Q^{-1} g_k)}.$$

Now apply the Kantorovich inequality with $m = \lambda_{\min}(Q)$ and $M = \lambda_{\max}(Q)$:

$$\frac{(g_k^\top g_k)^2}{(g_k^\top Q g_k)(g_k^\top Q^{-1} g_k)} \ge \frac{4mM}{(m+M)^2} = \frac{4\kappa}{(\kappa+1)^2}.$$

Hence

$$\frac{f(x_{k+1}) - f(x^\star)}{f(x_k) - f(x^\star)} \le 1 - \frac{4\kappa}{(\kappa+1)^2} = \left(\frac{\kappa - 1}{\kappa + 1}\right)^2.$$

■

Remark (Condition number and practical behavior). If $\kappa(Q) = 1$, then Q is a scalar multiple of the identity and the convergence factor is 0: one steepest-descent step reaches the minimizer. As $\kappa(Q) \to \infty$, the factor approaches 1, so progress becomes arbitrarily slow. This is the precise mathematical explanation for the zig-zagging seen on elongated quadratic level sets.

Remark (Why steepest descent is rarely the final algorithm). Steepest descent is globally robust and easy to implement, but in ill-conditioned problems it is typically too slow for serious computation. Modern practice therefore treats it as

- a baseline method,
- a first step inside more sophisticated updates,
- or a fallback when curvature information is unreliable.

Conjugate gradients, Newton, quasi-Newton, proximal, and trust-region methods exist precisely because raw steepest descent wastes curvature information.

2.3 Global Convergence Theory

Definition 2.5 (Lipschitz continuous gradient)**.** The gradient ∇f is *Lipschitz continuous* with constant $L > 0$ on a set D if

$$\|\nabla f(x) - \nabla f(y)\| \le L\|x - y\| \qquad \text{for all } x, y \in D.$$

Remark. This assumption controls how fast the gradient can change. It is the natural smoothness condition behind most descent estimates because it replaces the exact Taylor remainder by a uniform quadratic upper bound.

Theorem 2.3 (Descent lemma / quadratic upper bound)**.** If ∇f is Lipschitz continuous with constant L on a convex set containing the segment from x to $x + \alpha d$, then

$$f(x + \alpha d) \le f(x) + \alpha \nabla f(x)^\top d + \frac{L}{2}\alpha^2 \|d\|^2.$$

Proof. Define

$$\phi(t) = f(x + t\alpha d), \qquad t \in [0, 1].$$

By the fundamental theorem of calculus,

$$f(x + \alpha d) - f(x) = \int_0^1 \phi'(t)\, dt = \int_0^1 \nabla f(x + t\alpha d)^\top (\alpha d)\, dt.$$

Add and subtract $\nabla f(x)$:

$$\begin{aligned} f(x + \alpha d) - f(x) &= \alpha \nabla f(x)^\top d + \alpha \int_0^1 \big(\nabla f(x + t\alpha d) - \nabla f(x)\big)^\top d\, dt \\ &\le \alpha \nabla f(x)^\top d + \alpha \int_0^1 \|\nabla f(x + t\alpha d) - \nabla f(x)\| \|d\|\, dt \\ &\le \alpha \nabla f(x)^\top d + \alpha \int_0^1 Lt\alpha \|d\|^2\, dt \\ &= \alpha \nabla f(x)^\top d + \frac{L}{2}\alpha^2 \|d\|^2. \end{aligned}$$

■

Theorem 2.4 (Gradient descent with step $1/L$ gives sufficient decrease)**.** If ∇f is Lipschitz continuous with constant L, then the fixed-step gradient descent iteration

$$x_{k+1} = x_k - \frac{1}{L}\nabla f(x_k)$$

satisfies

$$f(x_{k+1}) \le f(x_k) - \frac{1}{2L}\|\nabla f(x_k)\|^2.$$

Proof. Apply the descent lemma with

$$d_k = -\nabla f(x_k), \qquad \alpha = \frac{1}{L}.$$

Then

$$\begin{aligned} f(x_{k+1}) &\leq f(x_k) + \frac{1}{L}\nabla f(x_k)^\top(-\nabla f(x_k)) + \frac{L}{2}\frac{1}{L^2}\|\nabla f(x_k)\|^2 \\ &= f(x_k) - \frac{1}{L}\|\nabla f(x_k)\|^2 + \frac{1}{2L}\|\nabla f(x_k)\|^2 \\ &= f(x_k) - \frac{1}{2L}\|\nabla f(x_k)\|^2. \end{aligned}$$

■

Theorem 2.5 (Zoutendijk condition). Consider a line-search method

$$x_{k+1} = x_k + \alpha_k d_k$$

with the following assumptions:

(i) f is continuously differentiable and bounded below on the level set

$$\mathcal{L} = \{x : f(x) \leq f(x_0)\},$$

(ii) ∇f is Lipschitz continuous with constant L on an open neighborhood of $\mathcal{L}$,

(iii) each d_k is a descent direction,

(iv) each α_k satisfies the Wolfe conditions.

Let θ_k be the angle between d_k and $-\nabla f(x_k)$, defined by

$$\cos\theta_k = \frac{-\nabla f(x_k)^\top d_k}{\|\nabla f(x_k)\|\,\|d_k\|}.$$

Then

$$\sum_{k=0}^{\infty} \cos^2\theta_k\,\|\nabla f(x_k)\|^2 < \infty.$$

Proof. Write $g_k = \nabla f(x_k)$. By the curvature condition,

$$g_{k+1}^\top d_k \geq c_2 g_k^\top d_k.$$

Since $g_k^\top d_k < 0$, we rearrange this as

$$(g_{k+1} - g_k)^\top d_k \geq (c_2 - 1)g_k^\top d_k = (1 - c_2)(-g_k^\top d_k).$$

By Lipschitz continuity of ∇f,

$$\|g_{k+1} - g_k\| \leq L\|x_{k+1} - x_k\| = L\alpha_k\|d_k\|.$$

Hence

$$(g_{k+1} - g_k)^\top d_k \leq \|g_{k+1} - g_k\|\|d_k\| \leq L\alpha_k\|d_k\|^2.$$

Combining the two inequalities gives

$$\alpha_k \geq \frac{(1 - c_2)(-g_k^\top d_k)}{L\|d_k\|^2}.$$

Now apply Armijo:

$$f(x_k) - f(x_{k+1}) \geq -c_1\alpha_k g_k^\top d_k.$$

Substitute the lower bound on α_k:

$$f(x_k) - f(x_{k+1}) \geq \frac{c_1(1-c_2)(g_k^\top d_k)^2}{L\|d_k\|^2}.$$

Since

$$(g_k^\top d_k)^2 = \|g_k\|^2\|d_k\|^2 \cos^2\theta_k,$$

we obtain

$$f(x_k) - f(x_{k+1}) \geq \frac{c_1(1-c_2)}{L}\cos^2\theta_k\|g_k\|^2.$$

Summing from $k = 0$ to N gives

$$\frac{c_1(1-c_2)}{L}\sum_{k=0}^{N}\cos^2\theta_k\|g_k\|^2 \leq f(x_0) - f(x_{N+1}).$$

Because all iterates remain in the initial level set, the right-hand side is bounded above by $f(x_0) - \inf_{\mathcal{L}} f < \infty$. Letting $N \to \infty$ proves

$$\sum_{k=0}^{\infty}\cos^2\theta_k\|g_k\|^2 < \infty.$$

■

Corollary 2.1 (A standard consequence of Zoutendijk). If, in addition to the assumptions of the theorem, there exists $\delta > 0$ such that

$$\cos\theta_k \geq \delta \qquad \text{for all } k,$$

then

$$\liminf_{k\to\infty}\|\nabla f(x_k)\| = 0.$$

Proof. Suppose instead that there exists $\varepsilon > 0$ and K such that

$$\|\nabla f(x_k)\| \geq \varepsilon \qquad \text{for all } k \geq K.$$

Then

$$\cos^2\theta_k\|\nabla f(x_k)\|^2 \geq \delta^2\varepsilon^2 \qquad \text{for all } k \geq K.$$

The series

$$\sum_{k=K}^{\infty}\cos^2\theta_k\|\nabla f(x_k)\|^2$$

would therefore diverge, contradicting Zoutendijk's theorem. Hence the assumption was false, and

$$\liminf_{k\to\infty}\|\nabla f(x_k)\| = 0.$$

■

Proposition 2.6 (Angle bounds for steepest descent and Newton). (i) For steepest descent,

$$\cos\theta_k = 1 \qquad \text{for every } k.$$

(ii) For Newton's method, if

$$mI \preceq \nabla^2 f(x_k) \preceq MI \qquad \text{with } 0 < m \leq M,$$

then the Newton direction $d_k = -[\nabla^2 f(x_k)]^{-1}\nabla f(x_k)$ satisfies

$$\cos\theta_k \geq \frac{m}{M}.$$

Proof. For steepest descent,

$$d_k = -g_k,$$

so

$$\cos\theta_k = \frac{-g_k^\top(-g_k)}{\|g_k\|\|-g_k\|} = 1.$$

For Newton's method, let $H_k = \nabla^2 f(x_k)$ and $g_k = \nabla f(x_k)$. Then

$$d_k = -H_k^{-1}g_k.$$

Hence

$$\cos\theta_k = \frac{g_k^\top H_k^{-1}g_k}{\|g_k\|\,\|H_k^{-1}g_k\|}.$$

From $mI \preceq H_k \preceq MI$ we get

$$\frac{1}{M}I \preceq H_k^{-1} \preceq \frac{1}{m}I.$$

Therefore

$$g_k^\top H_k^{-1}g_k \geq \frac{1}{M}\|g_k\|^2$$

and

$$\|H_k^{-1}g_k\| \leq \frac{1}{m}\|g_k\|.$$

Substituting these estimates gives

$$\cos\theta_k \geq \frac{(1/M)\|g_k\|^2}{\|g_k\|\,(1/m)\|g_k\|} = \frac{m}{M}.$$

■

Remark (What Zoutendijk does not say). Zoutendijk's theorem is a global convergence *framework*, not a full problem solution. It does *not* imply:

- convergence to a local minimum rather than a saddle point,
- any explicit convergence rate,
- applicability to nonsmooth objectives,
- convergence to a global minimum without extra convexity assumptions.

What it does say is more modest but still powerful: if every step is sufficiently well aligned with the negative gradient and Wolfe line search is used, then the method cannot keep producing large gradients forever.

2.4 Convergence Rates for Gradient Descent

Theorem 2.6 (Sublinear $O(1/k)$ rate for L-smooth convex functions). Let $f : \mathbb{R}^n \to \mathbb{R}$ be convex and differentiable, and assume ∇f is Lipschitz continuous with constant L. Let $x^\star$ be a global minimizer, and consider

$$x_{k+1} = x_k - \frac{1}{L}\nabla f(x_k).$$

Then

$$f(x_k) - f(x^\star) \leq \frac{L\|x_0 - x^\star\|^2}{2k} \qquad \text{for all } k \geq 1.$$

Proof. Write $g_k = \nabla f(x_k)$. Convexity gives

$$f(x^\star) \geq f(x_k) + g_k^\top (x^\star - x_k),$$

so

$$f(x_k) - f(x^\star) \leq g_k^\top (x_k - x^\star).$$

On the other hand, since

$$x_{k+1} = x_k - \frac{1}{L} g_k,$$

we have

$$\|x_k - x^\star\|^2 - \|x_{k+1} - x^\star\|^2 = 2\frac{1}{L} g_k^\top (x_k - x^\star) - \frac{1}{L^2}\|g_k\|^2.$$

Multiplying by $L/2$ yields

$$\frac{L}{2}\big(\|x_k - x^\star\|^2 - \|x_{k+1} - x^\star\|^2\big) = g_k^\top (x_k - x^\star) - \frac{1}{2L}\|g_k\|^2.$$

Using the sufficient decrease estimate from the previous subsection,

$$f(x_{k+1}) \leq f(x_k) - \frac{1}{2L}\|g_k\|^2.$$

Combining these relations gives

$$f(x_{k+1}) - f(x^\star) \leq \frac{L}{2}\big(\|x_k - x^\star\|^2 - \|x_{k+1} - x^\star\|^2\big).$$

Summing from $k = 0$ to $N - 1$ produces

$$\sum_{k=0}^{N-1} \big(f(x_{k+1}) - f(x^\star)\big) \leq \frac{L}{2}\|x_0 - x^\star\|^2.$$

Because the objective values decrease monotonically under the method,

$$N\big(f(x_N) - f(x^\star)\big) \leq \sum_{k=0}^{N-1} \big(f(x_{k+1}) - f(x^\star)\big).$$

Therefore

$$f(x_N) - f(x^\star) \leq \frac{L\|x_0 - x^\star\|^2}{2N}.$$

■

Lemma 2.2 (Strong convexity implies a gradient lower bound)**.** If f is differentiable and m-strongly convex with minimizer $x^\star$, then for every x,

$$f(x) - f(x^\star) \leq \frac{1}{2m}\|\nabla f(x)\|^2.$$

Proof. Strong convexity gives

$$f(x^\star) \geq f(x) + \nabla f(x)^\top (x^\star - x) + \frac{m}{2}\|x^\star - x\|^2.$$

Rearranging,

$$f(x) - f(x^\star) \leq \nabla f(x)^\top (x - x^\star) - \frac{m}{2}\|x - x^\star\|^2.$$

Now maximize the right-hand side over $z = x - x^\star$:

$$\nabla f(x)^\top z - \frac{m}{2}\|z\|^2 \le \max_z \left\{ \nabla f(x)^\top z - \frac{m}{2}\|z\|^2 \right\} = \frac{1}{2m}\|\nabla f(x)\|^2.$$

This proves the bound. ■

Theorem 2.7 (Linear convergence for strongly convex smooth functions). Assume f is differentiable, L-smooth, and m-strongly convex. Then gradient descent with step size $1/L$ satisfies

$$f(x_{k+1}) - f(x^\star) \le \left(1 - \frac{m}{L}\right)\big(f(x_k) - f(x^\star)\big),$$

and consequently

$$f(x_k) - f(x^\star) \le \left(1 - \frac{m}{L}\right)^k \big(f(x_0) - f(x^\star)\big).$$

Proof. From the sufficient decrease estimate,

$$f(x_{k+1}) \le f(x_k) - \frac{1}{2L}\|\nabla f(x_k)\|^2.$$

By the lemma,

$$f(x_k) - f(x^\star) \le \frac{1}{2m}\|\nabla f(x_k)\|^2,$$

or equivalently

$$\|\nabla f(x_k)\|^2 \ge 2m\big(f(x_k) - f(x^\star)\big).$$

Substituting into the decrease estimate gives

$$\begin{aligned} f(x_{k+1}) - f(x^\star) &\le f(x_k) - f(x^\star) - \frac{1}{2L}\|\nabla f(x_k)\|^2 \\ &\le f(x_k) - f(x^\star) - \frac{m}{L}\big(f(x_k) - f(x^\star)\big) \\ &= \left(1 - \frac{m}{L}\right)\big(f(x_k) - f(x^\star)\big). \end{aligned}$$

Iterating proves the geometric rate. ■

Remark (Condition number in the strongly convex case). Writing $\kappa = L/m$, the factor becomes

$$1 - \frac{1}{\kappa}.$$

Large condition number means weak curvature in some directions relative to others, which again translates directly into slow convergence.

2.5 Newton's Method

Intuition. Steepest descent uses only first-order information, so it cannot distinguish a narrow valley from a wide bowl. Newton's method corrects this by fitting a local quadratic model and minimizing that model exactly. Near a nondegenerate minimizer, this makes the remaining error square itself from one step to the next.

Definition 2.6 (Newton direction). Given x_k, form the second-order Taylor model

$$m_k(d) = f(x_k) + \nabla f(x_k)^\top d + \frac{1}{2} d^\top \nabla^2 f(x_k) d.$$

If $\nabla^2 f(x_k)$ is invertible, the minimizer of this quadratic model is the *Newton direction*

$$d_k^N = -[\nabla^2 f(x_k)]^{-1} \nabla f(x_k).$$

Proposition 2.7 (Newton direction is a descent direction under positive definiteness)**.** If $\nabla^2 f(x_k) \succ 0$ and $\nabla f(x_k) \neq 0$, then the Newton direction satisfies

$$\nabla f(x_k)^\top d_k^N < 0.$$

Proof. Let $g_k = \nabla f(x_k)$ and $H_k = \nabla^2 f(x_k)$. Then

$$d_k^N = -H_k^{-1} g_k.$$

Therefore

$$g_k^\top d_k^N = -g_k^\top H_k^{-1} g_k.$$

Since $H_k \succ 0$, its inverse is also positive definite. Hence

$$g_k^\top H_k^{-1} g_k > 0$$

whenever $g_k \neq 0$. Thus

$$g_k^\top d_k^N < 0.$$

■

Algorithm 4: Damped Newton Method

Input: Initial point x_0, tolerance $\varepsilon > 0$, line-search parameters

```
for k = 0, 1, 2, ... do
    g_k ← ∇f(x_k)
    if ‖g_k‖ ≤ ε then
        return x_k
    H_k ← ∇²f(x_k)
    Solve H_k d_k = −g_k
    Choose α_k > 0 by backtracking or Wolfe line search
    x_{k+1} ← x_k + α_k d_k
```

Example (One Newton step on a quadratic)**.** Consider

$$f(x_1, x_2) = x_1^2 + 2x_2^2 + x_1 x_2 - 4x_1.$$

Then

$$\nabla f(x_1, x_2) = \begin{bmatrix} 2x_1 + x_2 - 4 \\ x_1 + 4x_2 \end{bmatrix}, \qquad \nabla^2 f(x_1, x_2) = \begin{bmatrix} 2 & 1 \\ 1 & 4 \end{bmatrix}.$$

Starting from

$$x_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix},$$

we have

$$g_0 = \begin{bmatrix} -4 \\ 0 \end{bmatrix}.$$

The Newton step solves

$$\begin{bmatrix} 2 & 1 \\ 1 & 4 \end{bmatrix} d_0 = \begin{bmatrix} 4 \\ 0 \end{bmatrix},$$

which yields

$$d_0 = \begin{bmatrix} 16/7 \\ -4/7 \end{bmatrix}.$$

Hence

$$x_1 = x_0 + d_0 = \begin{bmatrix} 16/7 \\ -4/7 \end{bmatrix}.$$

Substituting into the gradient shows

$$\nabla f(x_1) = 0.$$

So one Newton step reaches the exact minimizer. This is not an accident; it is a general fact for quadratic objectives.

Proposition 2.8 (One-step convergence on quadratics)**.** For

$$f(x) = \frac{1}{2}x^\top Qx - b^\top x, \qquad Q \succ 0,$$

Newton's method reaches the minimizer

$$x^\star = Q^{-1}b$$

in exactly one iteration from any starting point.

Proof. Here

$$\nabla f(x) = Qx - b, \qquad \nabla^2 f(x) = Q.$$

The Newton update is therefore

$$x_{k+1} = x_k - Q^{-1}(Qx_k - b) = Q^{-1}b = x^\star.$$

So the minimizer is reached in one step, regardless of the starting point. ■

Theorem 2.8 (Local quadratic convergence of Newton's method)**.** Let $x^\star$ satisfy

$$\nabla f(x^\star) = 0, \qquad \nabla^2 f(x^\star) \succ 0.$$

Assume $\nabla^2 f$ is Lipschitz continuous with constant M on a neighborhood of $x^\star$. Then there exist constants $r > 0$ and $C > 0$ such that whenever $\|x_k - x^\star\| < r$, pure Newton's method is well defined and

$$\|x_{k+1} - x^\star\| \le C\|x_k - x^\star\|^2.$$

Proof. Let

$$e_k = x_k - x^\star, \qquad H_k = \nabla^2 f(x_k).$$

Since $\nabla^2 f(x^\star) \succ 0$, continuity of the Hessian implies that for x_k sufficiently close to $x^\star$, the matrices H_k remain uniformly positive definite. Thus there exists $r > 0$ and a constant $\beta > 0$ such that

$$\left\|H_k^{-1}\right\| \le \beta$$

whenever $\|e_k\| < r$.

The Newton update gives

$$e_{k+1} = x_k - H_k^{-1}\nabla f(x_k) - x^\star = H_k^{-1}\big(H_k e_k - \nabla f(x_k)\big),$$

where we used $\nabla f(x^\star) = 0$.

Now use the integral form of the gradient:

$$\nabla f(x_k) - \nabla f(x^\star) = \int_0^1 \nabla^2 f(x^\star + te_k)e_k\, dt.$$

Hence

$$H_k e_k - \nabla f(x_k) = \int_0^1 \big(H_k - \nabla^2 f(x^\star + te_k)\big)e_k\, dt.$$

Taking norms and using Lipschitz continuity of the Hessian,

$$\begin{aligned}\|H_k e_k - \nabla f(x_k)\| &\le \int_0^1 \left\|H_k - \nabla^2 f(x^\star + te_k)\right\| \|e_k\| \, dt \\ &\le \int_0^1 M(1-t)\|e_k\|^2 \, dt \\ &= \frac{M}{2}\|e_k\|^2.\end{aligned}$$

Therefore

$$\|e_{k+1}\| \le \left\|H_k^{-1}\right\| \|H_k e_k - \nabla f(x_k)\| \le \beta \frac{M}{2}\|e_k\|^2.$$

This is the desired quadratic estimate, with

$$C = \frac{\beta M}{2}.$$

■

Remark (Why pure Newton can fail globally)**.** Newton's method is only a *local* fast method.

- Each iteration requires solving a linear system with the Hessian, which can be expensive.
- Far from the minimizer, the quadratic model may be poor and the step may increase the objective.
- If the Hessian is singular or indefinite, the Newton direction may be undefined or may fail to be a descent direction.

This is why practical optimization almost always uses *damped Newton*, namely Newton steps embedded inside a line search or trust-region globalization strategy.

Remark (Newton can converge to a saddle point)**.** Consider

$$f(x) = x^3.$$

For $x \neq 0$,

$$f'(x) = 3x^2, \qquad f''(x) = 6x.$$

Newton's method gives

$$x_{k+1} = x_k - \frac{3x_k^2}{6x_k} = \frac{x_k}{2}.$$

So every nonzero starting point converges to $x = 0$. But $x = 0$ is not a minimizer; it is an inflection point. This shows clearly that Newton is not intrinsically an optimization method until one adds globalization and curvature safeguards.

2.6 Quasi-Newton Methods and BFGS

Intuition. Newton's method is fast because it uses curvature, but expensive because it needs the Hessian. Quasi-Newton methods try to keep the good part and avoid the bad part: they build curvature information incrementally from gradients, updating an approximate Hessian or inverse Hessian at each step.

Definition 2.7 (Secant condition)**.** Let

$$s_k = x_{k+1} - x_k, \qquad y_k = \nabla f(x_{k+1}) - \nabla f(x_k).$$

A quasi-Newton matrix B_{k+1} is required to satisfy the *secant condition*

$$B_{k+1}s_k = y_k.$$

Remark (Why the secant condition is natural). By the fundamental theorem of calculus,

$$y_k = \int_0^1 \nabla^2 f(x_k + ts_k)s_k \, dt.$$

So y_k is the exact action of the *average Hessian along the step* on s_k. The secant condition therefore asks the new Hessian approximation to reproduce the true curvature at least in the most recently explored direction.

Definition 2.8 (BFGS inverse-Hessian update). Let H_k approximate the inverse Hessian. Define

$$\rho_k = \frac{1}{y_k^\top s_k}.$$

The BFGS update is

$$H_{k+1} = \big(I - \rho_k s_k y_k^\top\big) H_k \big(I - \rho_k y_k s_k^\top\big) + \rho_k s_k s_k^\top.$$

Algorithm 5: BFGS with Line Search

Input: Initial point x_0, initial inverse-Hessian approximation $H_0 \succ 0$, tolerance $\varepsilon > 0$

```
for k = 0, 1, 2, ... do
    g_k ← ∇f(x_k)
    if ‖g_k‖ ≤ ε then
        return x_k
    d_k ← −H_k g_k
    Choose α_k > 0 using a Wolfe line search
    x_{k+1} ← x_k + α_k d_k
    s_k ← x_{k+1} − x_k
    y_k ← ∇f(x_{k+1}) − ∇f(x_k)
    ρ_k ← 1/(y_k^⊤ s_k)
    H_{k+1} ← (I − ρ_k s_k y_k^⊤) H_k (I − ρ_k y_k s_k^⊤) + ρ_k s_k s_k^⊤
```

Theorem 2.9 (Two basic BFGS properties). Suppose $H_k \succ 0$ and $y_k^\top s_k > 0$. Then the BFGS update satisfies:

(i) the secant condition in inverse form,

$$H_{k+1}y_k = s_k,$$

(ii) positive definiteness preservation,

$$H_{k+1} \succ 0.$$

Proof. Let

$$\rho = \frac{1}{y_k^\top s_k}, \qquad V = I - \rho s_k y_k^\top.$$

Then

$$H_{k+1} = V H_k V^\top + \rho s_k s_k^\top.$$

To prove the secant condition, note that

$$V^\top y_k = \big(I - \rho y_k s_k^\top\big) y_k = y_k - \rho y_k (s_k^\top y_k) = 0.$$

Therefore

$$H_{k+1}y_k = VH_kV^\top y_k + \rho s_k s_k^\top y_k = \rho s_k(s_k^\top y_k) = s_k.$$

For positive definiteness, take any nonzero vector z and set

$$w = V^\top z.$$

Then

$$z^\top H_{k+1}z = w^\top H_k w + \rho(s_k^\top z)^2.$$

The first term is nonnegative because $H_k \succ 0$, and the second term is nonnegative because $\rho > 0$. If $s_k^\top z = 0$, then $w = V^\top z = z$, so the first term is strictly positive. If $s_k^\top z \neq 0$, then the second term is strictly positive. Hence

$$z^\top H_{k+1}z > 0$$

for all nonzero z, proving $H_{k+1} \succ 0$. ■

Remark (Why the first BFGS step is steepest descent). If $H_0 = I$, then the first search direction is

$$d_0 = -H_0\nabla f(x_0) = -\nabla f(x_0).$$

So the first BFGS step is exactly a steepest-descent step. This is not a defect; it is the natural starting point before any curvature information has been learned from the iterates.

Theorem 2.10 (Superlinear convergence of BFGS). Suppose:

- $f \in C^2$ in a neighborhood of a minimizer $x^\star$,
- $\nabla^2 f(x^\star) \succ 0$,
- $\nabla^2 f$ is Lipschitz continuous near $x^\star$,
- the BFGS iterates generated with a Wolfe line search converge to $x^\star$.

Then BFGS converges *superlinearly*, meaning

$$\frac{\|x_{k+1} - x^\star\|}{\|x_k - x^\star\|} \to 0.$$

Theorem 2.11 (Dennis–More condition). Let B_k denote a quasi-Newton Hessian approximation and let d_k be the corresponding search direction. Under the standard assumptions for local convergence, the method is superlinearly convergent if and only if

$$\lim_{k\to\infty} \frac{\|(B_k - \nabla^2 f(x^\star))d_k\|}{\|d_k\|} = 0.$$

Remark (Interpretation of the Dennis–More condition). The quantity

$$(B_k - \nabla^2 f(x^\star))d_k$$

measures the error in the Hessian model *along the step that the algorithm actually takes*. The condition does not require B_k to converge to the true Hessian in every matrix entry; it requires the approximation to become asymptotically exact in the directions that matter for the iterates. That is precisely the amount of accuracy needed to recover Newton-like superlinear behavior.

Remark (Self-concordance and the quadratic convergence region). For self-concordant functions, one can quantify the region in which Newton's method transitions from a damped global phase to a pure quadratic local phase. This is important in interior-point methods, where self-concordance

supplies a rigorous globalization theory beyond the local result proved earlier.

2.7 Comparison of Major Methods

Method	Rate type	Condition-number dependence	Per-iteration cost
Steepest descent	Linear on SPD quadratics; often slow	Strongly affected by κ	One gradient + line search
Newton	Quadratic near $x^\star$	Essentially insensitive locally	Gradient + Hessian + linear solve
BFGS	Superlinear under standard assumptions	Much milder than steepest descent	Gradient + matrix update + line search

Remark. The right way to read the table is not "which method is best in the abstract?" but "which information can I afford to compute, and what structure does my problem provide?" In large-scale optimal control, that question leads naturally to structure-exploiting second-order methods, Riccati recursions, shooting methods, and SQP variants.

Lecture 3: Constrained Optimization

Intuition. Unconstrained optimization is the exception, not the rule. In optimal control, dynamics, actuator bounds, state limits, terminal conditions, and collision-avoidance requirements all appear as constraints. Once constraints are present, the gradient of the objective need no longer vanish at an optimum; instead, it must be balanced against the gradients of the active constraints. This lecture builds the static constrained-optimization theory that later reappears in infinite-dimensional form inside Pontryagin's principle, trajectory transcription, and model predictive control.

3.1 Problem Formulation

Definition 3.1 (General constrained optimization problem)**.** A finite-dimensional constrained optimization problem has the form

$$\begin{aligned} \min_{x\in\mathbb{R}^n} \quad & f(x) \\ \text{subject to} \quad & h(x) = 0, \\ & g(x) \le 0, \end{aligned}$$

where

$$f : \mathbb{R}^n \to \mathbb{R}, \qquad h : \mathbb{R}^n \to \mathbb{R}^m, \qquad g : \mathbb{R}^n \to \mathbb{R}^p.$$

Here the equality constraints are written componentwise as

$$h_i(x) = 0, \qquad i = 1, \dots, m,$$

and the inequality constraints as

$$g_j(x) \le 0, \qquad j = 1, \dots, p.$$

Definition 3.2 (Feasible set, active set, inactive set)**.** The *feasible set* is

$$\mathcal{F} = \{x \in \mathbb{R}^n : h(x) = 0,\ g(x) \le 0\}.$$

A feasible point $x \in \mathcal{F}$ is:

- *active* for inequality constraint j if
$$g_j(x) = 0,$$
- *inactive* for inequality constraint j if
$$g_j(x) < 0.$$

The active-set index set is
$$\mathcal{A}(x) = \{j \in \{1, \dots, p\} : g_j(x) = 0\}.$$

Remark. Only active constraints can influence first-order optimality conditions. Inactive constraints are locally irrelevant because a sufficiently small perturbation remains feasible with respect to them.

Example (Two-dimensional geometry). (a) The set
$$\{x \in \mathbb{R}^2 : x_1^2 + x_2^2 \leq 1\}$$
is the closed unit disk. At an interior point, no inequality constraint is active. At a boundary point with $x_1^2 + x_2^2 = 1$, the boundary inequality is active.

(b) The set
$$\{x \in \mathbb{R}^2 : x_1 + x_2 = 1\}$$
is an affine line. Every feasible perturbation must stay tangent to that line.

(c) The set
$$\{x \in \mathbb{R}^2 : x_1 \geq 0,\ x_2 \geq 0,\ x_1 + x_2 \leq 1\}$$
is a triangle. At an interior point no inequalities are active; at a point on an edge exactly one is active; at a vertex, two constraints may be active simultaneously.

Definition 3.3 (Feasible direction). Let $x \in \mathcal{F}$. A vector $d \in \mathbb{R}^n$ is a *feasible direction* at x if there exists $\bar{t} > 0$ and a curve $\gamma : [0, \bar{t}) \to \mathcal{F}$ such that
$$\gamma(0) = x, \qquad \gamma'(0) = d.$$

Remark. For equality constraints, feasible directions are tangent to the constraint manifold. For inequality constraints, feasible directions must point into the feasible region or stay tangent to the active boundary.

3.2 Equality-Constrained Optimization and Lagrange Multipliers

Intuition. In unconstrained optimization, stationarity means that the objective cannot decrease along any direction, so the gradient must vanish. Under equality constraints, only *feasible* directions are allowed. The gradient need not vanish; it must only vanish on the tangent space of the feasible manifold. That forces the gradient of the objective to lie in the span of the gradients of the constraints.

Definition 3.4 (Lagrangian for equality constraints). For the problem
$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & f(x) \\ \text{subject to} \quad & h(x) = 0, \end{aligned}$$
the *Lagrangian* is
$$\mathcal{L}(x, \lambda) = f(x) + \lambda^\top h(x), \qquad \lambda \in \mathbb{R}^m.$$

Definition 3.5 (LICQ for equality constraints). The *linear independence constraint qualification* (LICQ) at a feasible point x means that the gradients

$$\nabla h_1(x), \dots, \nabla h_m(x)$$

are linearly independent. Equivalently, the Jacobian

$$J_h(x) \in \mathbb{R}^{m \times n}$$

has full row rank m.

Theorem 3.1 (Lagrange multiplier theorem). Assume $f, h_1, \dots, h_m$ are continuously differentiable, and let $x^\star$ be a local minimizer of

$$\begin{aligned} \min_x \quad & f(x) \\ \text{subject to} \quad & h(x) = 0. \end{aligned}$$

If LICQ holds at $x^\star$, then there exists a multiplier $\lambda^\star \in \mathbb{R}^m$ such that

$$\nabla_x \mathcal{L}(x^\star, \lambda^\star) = \nabla f(x^\star) + J_h(x^\star)^\top \lambda^\star = 0.$$

Proof. Because LICQ holds, the implicit function theorem implies that near $x^\star$ the feasible set

$$\{x : h(x) = 0\}$$

is a smooth manifold whose tangent space at $x^\star$ is

$$T_{x^\star}\mathcal{F} = \{d \in \mathbb{R}^n : J_h(x^\star)d = 0\}.$$

Since $x^\star$ is a local minimizer restricted to this manifold, the directional derivative of f must vanish along every tangent direction:

$$\nabla f(x^\star)^\top d = 0 \qquad \text{for all } d \in T_{x^\star}\mathcal{F}.$$

In other words,

$$\nabla f(x^\star) \in \left(T_{x^\star}\mathcal{F}\right)^\perp.$$

But

$$T_{x^\star}\mathcal{F} = \operatorname{null}(J_h(x^\star)),$$

and for every matrix A,

$$\operatorname{null}(A)^\perp = \operatorname{range}(A^\top).$$

Hence

$$\nabla f(x^\star) \in \operatorname{range}(J_h(x^\star)^\top).$$

Therefore there exists $\lambda^\star$ such that

$$\nabla f(x^\star) + J_h(x^\star)^\top \lambda^\star = 0.$$

■

Remark (Geometric interpretation). When there is a single equality constraint $h(x) = 0$, the condition becomes

$$\nabla f(x^\star) = -\lambda^\star \nabla h(x^\star).$$

Thus the gradient of the objective is parallel to the gradient of the constraint. Since $\nabla h(x^\star)$ is normal to the feasible surface, the objective gradient has no component along the tangent space. This is the constrained analogue of stationarity.

Example (Minimizing distance to the origin on a line). Consider

$$\begin{aligned} \min_{x_1,x_2} \quad & x_1^2 + x_2^2 \\ \text{subject to} \quad & x_1 + x_2 = 1. \end{aligned}$$

The Lagrangian is

$$\mathcal{L}(x_1, x_2, \lambda) = x_1^2 + x_2^2 + \lambda(x_1 + x_2 - 1).$$

The stationarity equations are

$$2x_1 + \lambda = 0, \qquad 2x_2 + \lambda = 0,$$

so

$$x_1 = x_2.$$

Using the constraint, we get

$$x_1^\star = x_2^\star = \frac{1}{2}.$$

The multiplier is

$$\lambda^\star = -1.$$

Geometrically, this is the orthogonal projection of the origin onto the affine line $x_1 + x_2 = 1$.

Remark (What can go wrong without a constraint qualification). Constraint qualifications are not technical decoration; they prevent degenerate constraint geometry. Consider

$$\begin{aligned} \min_{x\in\mathbb{R}} \quad & x \\ \text{subject to} \quad & x^2 = 0. \end{aligned}$$

The only feasible point is $x^\star = 0$, so it is trivially a global minimizer. But the gradient of the constraint is

$$\nabla h(0) = 0,$$

so there is no multiplier λ satisfying

$$\nabla f(0) + \lambda \nabla h(0) = 1 + 0 = 0.$$

LICQ fails, and the Lagrange multiplier theorem correspondingly fails.

3.3 Inequality Constraints and KKT Conditions

Definition 3.6 (Lagrangian for mixed equality and inequality constraints). For the problem

$$\begin{aligned} \min_{x} \quad & f(x) \\ \text{subject to} \quad & h(x) = 0, \\ & g(x) \le 0, \end{aligned}$$

the Lagrangian is

$$\mathcal{L}(x, \lambda, \mu) = f(x) + \lambda^\top h(x) + \mu^\top g(x),$$

where

$$\lambda \in \mathbb{R}^m, \qquad \mu \in \mathbb{R}^p.$$

Definition 3.7 (LICQ for mixed constraints). LICQ holds at a feasible point x if the vectors

$$\nabla h_1(x), \dots, \nabla h_m(x), \qquad \nabla g_j(x) \ \text{ for } j \in \mathcal{A}(x),$$

are linearly independent.

Lemma 3.1 (Linearized alternative for constrained stationarity). Let

$$a_1, \dots, a_m, b_1, \dots, b_q, c \in \mathbb{R}^n.$$

The following are equivalent:

(i) $c^\top d \geq 0$ for every d satisfying

$$a_i^\top d = 0, \quad i = 1, \dots, m, \qquad b_j^\top d \leq 0, \quad j = 1, \dots, q.$$

(ii) There exist multipliers $\lambda_1, \dots, \lambda_m \in \mathbb{R}$ and $\mu_1, \dots, \mu_q \geq 0$ such that

$$c + \sum_{i=1}^{m} \lambda_i a_i + \sum_{j=1}^{q} \mu_j b_j = 0.$$

Remark. This is a standard consequence of Farkas' lemma. It is the linear-algebraic mechanism behind the Karush–Kuhn–Tucker theorem.

Theorem 3.2 (KKT necessary conditions). Assume f, h, g are continuously differentiable. Let $x^\star$ be a local minimizer of

$$\begin{aligned} \min_x \quad & f(x) \\ \text{subject to} \quad & h(x) = 0, \\ & g(x) \leq 0, \end{aligned}$$

and assume LICQ holds at $x^\star$. Then there exist multipliers $\lambda^\star \in \mathbb{R}^m$ and $\mu^\star \in \mathbb{R}^p$ such that:

$$\begin{aligned} \nabla_x \mathcal{L}(x^\star, \lambda^\star, \mu^\star) &= 0 && \text{(stationarity)}, \\ h(x^\star) = 0, \quad g(x^\star) &\leq 0 && \text{(primal feasibility)}, \\ \mu^\star &\geq 0 && \text{(dual feasibility)}, \\ \mu_j^\star g_j(x^\star) = 0, \quad j &= 1, \dots, p && \text{(complementary slackness)}. \end{aligned}$$

Proof. Let

$$\mathcal{A}^\star = \mathcal{A}(x^\star)$$

denote the active set at the minimizer. Any feasible direction d at $x^\star$ must satisfy the linearized constraints

$$J_h(x^\star) d = 0, \qquad \nabla g_j(x^\star)^\top d \leq 0 \quad \text{for } j \in \mathcal{A}^\star.$$

Since $x^\star$ is a local minimizer, the objective cannot decrease in any feasible direction, so

$$\nabla f(x^\star)^\top d \geq 0$$

for every such d.

Apply the linearized alternative lemma with

$$c = \nabla f(x^\star), \qquad a_i = \nabla h_i(x^\star), \qquad b_j = \nabla g_j(x^\star), \;\; j \in \mathcal{A}^\star.$$

We obtain multipliers $\lambda^\star$ and $\mu_j^\star \geq 0$ for $j \in \mathcal{A}^\star$ such that

$$\nabla f(x^\star) + J_h(x^\star)^\top \lambda^\star + \sum_{j \in \mathcal{A}^\star} \mu_j^\star \nabla g_j(x^\star) = 0.$$

Define

$$\mu_j^\star = 0 \qquad \text{for } j \notin \mathcal{A}^\star.$$

Then stationarity becomes

$$\nabla f(x^\star) + J_h(x^\star)^\top \lambda^\star + J_g(x^\star)^\top \mu^\star = 0.$$

Primal feasibility holds because $x^\star$ is feasible. Dual feasibility holds because the multipliers of active inequalities are nonnegative and the inactive ones were set to zero. Finally, if j is inactive then $g_j(x^\star) < 0$ and $\mu_j^\star = 0$, while if j is active then $g_j(x^\star) = 0$. In both cases

$$\mu_j^\star g_j(x^\star) = 0.$$

■

Remark (Complementary slackness). The identity

$$\mu_j^\star g_j(x^\star) = 0$$

means that each inequality constraint falls into one of two categories:

- either the constraint is inactive, so it does not influence stationarity and its multiplier vanishes,
- or the constraint is active, in which case it may carry a positive multiplier.

There is no third possibility.

Example (KKT analysis in a simplex). Consider

$$\begin{aligned} \min_{x_1,x_2} \quad & (x_1-2)^2 + (x_2-1)^2 \\ \text{subject to} \quad & x_1 + x_2 - 1 \le 0, \\ & -x_1 \le 0, \\ & -x_2 \le 0. \end{aligned}$$

The feasible set is the triangle

$$x_1 \ge 0, \quad x_2 \ge 0, \quad x_1 + x_2 \le 1.$$

Since the objective is the squared distance to $(2,1)$, the minimizer should be the closest feasible point, namely

$$x^\star = (1,0).$$

Let

$$g_1(x) = x_1 + x_2 - 1, \quad g_2(x) = -x_1, \quad g_3(x) = -x_2.$$

Then

$$\nabla f(x) = \begin{bmatrix} 2(x_1-2) \\ 2(x_2-1) \end{bmatrix},$$

so

$$\nabla f(1,0) = \begin{bmatrix} -2 \\ -2 \end{bmatrix}.$$

The active constraints at $(1,0)$ are g_1 and g_3. Stationarity becomes

$$\begin{bmatrix} -2 \\ -2 \end{bmatrix} + \mu_1 \begin{bmatrix} 1 \\ 1 \end{bmatrix} + \mu_2 \begin{bmatrix} -1 \\ 0 \end{bmatrix} + \mu_3 \begin{bmatrix} 0 \\ -1 \end{bmatrix} = 0.$$

Complementary slackness forces $\mu_2 = 0$ because $g_2(1,0) = -1 < 0$. Hence

$$-2 + \mu_1 = 0, \qquad -2 + \mu_1 - \mu_3 = 0.$$

So

$$\mu_1 = 2, \qquad \mu_3 = 0.$$

All KKT conditions are satisfied, and because the problem is convex, this point is globally optimal.

Theorem 3.3 (KKT sufficient conditions for convex problems)**.** Assume:

- f is convex,
- each equality constraint is affine,
- each inequality function g_j is convex.

If a feasible point $x^\star$ admits multipliers $(\lambda^\star, \mu^\star)$ satisfying the KKT conditions, then $x^\star$ is a global minimizer.

Proof. By convexity of f,

$$f(x) \geq f(x^\star) + \nabla f(x^\star)^\top (x - x^\star) \qquad \text{for all } x.$$

Use stationarity:

$$\nabla f(x^\star) = -J_h(x^\star)^\top \lambda^\star - J_g(x^\star)^\top \mu^\star.$$

Thus

$$f(x) \geq f(x^\star) - \lambda^{\star\top} J_h(x^\star)(x - x^\star) - \mu^{\star\top} J_g(x^\star)(x - x^\star).$$

Since the equality constraints are affine,

$$h(x) = h(x^\star) + J_h(x^\star)(x - x^\star) = J_h(x^\star)(x - x^\star)$$

because $h(x^\star) = 0$. Therefore, for every feasible x,

$$J_h(x^\star)(x - x^\star) = 0.$$

By convexity of each g_j,

$$g_j(x) \geq g_j(x^\star) + \nabla g_j(x^\star)^\top (x - x^\star),$$

so

$$\nabla g_j(x^\star)^\top (x - x^\star) \leq g_j(x) - g_j(x^\star).$$

Multiply by $\mu_j^\star \geq 0$ and sum:

$$\mu^{\star\top} J_g(x^\star)(x - x^\star) \leq \mu^{\star\top} g(x) - \mu^{\star\top} g(x^\star).$$

If x is feasible, then $g(x) \leq 0$. Complementary slackness gives

$$\mu^{\star\top} g(x^\star) = 0.$$

Hence

$$\mu^{\star\top} J_g(x^\star)(x - x^\star) \leq 0.$$

Substituting into the lower bound for $f(x)$ yields

$$f(x) \geq f(x^\star)$$

for every feasible x. Therefore $x^\star$ is a global minimizer. ■

3.4 Second-Order Conditions for Constrained Problems

Definition 3.8 (Critical cone)**.** Let $(x^\star, \lambda^\star, \mu^\star)$ be a KKT point, and let

$$\mathcal{A}^\star = \mathcal{A}(x^\star).$$

The *critical cone* at $x^\star$ is

$$\mathcal{C}(x^\star, \lambda^\star, \mu^\star) = \left\{ d \in \mathbb{R}^n : \begin{array}{ll} J_h(x^\star)d = 0, & \\ \nabla g_j(x^\star)^\top d = 0 & \text{if } j \in \mathcal{A}^\star \text{ and } \mu_j^\star > 0, \\ \nabla g_j(x^\star)^\top d \le 0 & \text{if } j \in \mathcal{A}^\star \text{ and } \mu_j^\star = 0, \\ \nabla f(x^\star)^\top d = 0 & \end{array} \right\}.$$

Remark. The critical cone consists of first-order feasible directions along which the objective has zero first-order change. These are precisely the directions in which second-order information matters.

Theorem 3.4 (Second-order necessary condition)**.** Assume f, h, g are twice continuously differentiable, and let $(x^\star, \lambda^\star, \mu^\star)$ satisfy the KKT conditions. If $x^\star$ is a local minimizer, then

$$d^\top \nabla^2_{xx} \mathcal{L}(x^\star, \lambda^\star, \mu^\star) d \ge 0 \qquad \text{for all } d \in \mathcal{C}(x^\star, \lambda^\star, \mu^\star).$$

Proof. Fix a direction d in the critical cone. Under LICQ one may construct a feasible curve

$$x(t) = x^\star + td + o(t)$$

that respects the active constraints to first order. Because $x^\star$ is a local minimizer and because d lies in the critical cone, the first-order term in the Taylor expansion of the Lagrangian vanishes. Hence the second-order term must be nonnegative:

$$0 \le \frac{d^2}{dt^2}\Big|_{t=0} \mathcal{L}(x(t), \lambda^\star, \mu^\star) = d^\top \nabla^2_{xx} \mathcal{L}(x^\star, \lambda^\star, \mu^\star) d.$$

This is the constrained analogue of the unconstrained positive-semidefinite Hessian test. ■

Theorem 3.5 (Second-order sufficient condition)**.** Assume f, h, g are twice continuously differentiable, and let $(x^\star, \lambda^\star, \mu^\star)$ satisfy the KKT conditions. If

$$d^\top \nabla^2_{xx} \mathcal{L}(x^\star, \lambda^\star, \mu^\star) d > 0 \qquad \text{for every nonzero } d \in \mathcal{C}(x^\star, \lambda^\star, \mu^\star),$$

then $x^\star$ is a strict local minimizer.

Proof. Assume by contradiction that $x^\star$ is not a strict local minimizer. Then there exists a sequence of distinct feasible points $x_k \to x^\star$ with

$$f(x_k) \le f(x^\star).$$

Normalize the directions

$$d_k = \frac{x_k - x^\star}{\|x_k - x^\star\|}$$

and extract a convergent subsequence $d_k \to d$. Standard first-order feasibility arguments show that d belongs to the critical cone. Expanding the Lagrangian along the sequence and using the KKT conditions shows that the second-order term must be nonpositive along d, contradicting the assumed positive definiteness on the critical cone. Therefore $x^\star$ is a strict local minimizer. ■

3.5 Lagrangian Duality

Definition 3.9 (Dual function)**.** The *Lagrange dual function* is

$$q(\lambda, \mu) = \inf_{x \in \mathbb{R}^n} \mathcal{L}(x, \lambda, \mu) = \inf_x \big(f(x) + \lambda^\top h(x) + \mu^\top g(x) \big),$$

with the convention that the dual variables must satisfy

$$\mu \ge 0.$$

Proposition 3.1 (The dual function is concave). The function $q(\lambda, \mu)$ is concave in (λ, μ), even if the primal problem is not convex.

Proof. For each fixed x, the map

$$(\lambda, \mu) \mapsto f(x) + \lambda^\top h(x) + \mu^\top g(x)$$

is affine in (λ, μ). The dual function is the pointwise infimum of this family of affine functions. A pointwise infimum of affine functions is concave. Explicitly, for any two dual points (λ_1, μ_1) and (λ_2, μ_2) and any $\theta \in [0, 1]$,

$$\begin{aligned} q(\theta(\lambda_1, \mu_1) + (1-\theta)(\lambda_2, \mu_2)) &= \inf_x \Big[\theta \mathcal{L}(x, \lambda_1, \mu_1) + (1-\theta)\mathcal{L}(x, \lambda_2, \mu_2)\Big] \\ &\geq \theta \inf_x \mathcal{L}(x, \lambda_1, \mu_1) + (1-\theta) \inf_x \mathcal{L}(x, \lambda_2, \mu_2) \\ &= \theta q(\lambda_1, \mu_1) + (1-\theta) q(\lambda_2, \mu_2). \end{aligned}$$

■

Definition 3.10 (Dual problem). The *dual optimization problem* is

$$\begin{aligned} \max_{\lambda, \mu} \quad & q(\lambda, \mu) \\ \text{subject to} \quad & \mu \geq 0. \end{aligned}$$

Theorem 3.6 (Weak duality). Let x be any primal-feasible point and (λ, μ) any dual-feasible point. Then

$$q(\lambda, \mu) \leq f(x).$$

In particular,

$$\sup_{\mu \geq 0, \lambda} q(\lambda, \mu) \leq \inf_{x \in \mathcal{F}} f(x).$$

Proof. If x is primal feasible, then

$$h(x) = 0, \qquad g(x) \leq 0.$$

If (λ, μ) is dual feasible, then $\mu \geq 0$, so

$$\mu^\top g(x) \leq 0.$$

Hence

$$\mathcal{L}(x, \lambda, \mu) = f(x) + \lambda^\top h(x) + \mu^\top g(x) \leq f(x).$$

By definition of the infimum,

$$q(\lambda, \mu) = \inf_z \mathcal{L}(z, \lambda, \mu) \leq \mathcal{L}(x, \lambda, \mu) \leq f(x).$$

This proves weak duality. ■

Definition 3.11 (Duality gap). The *duality gap* is the difference

$$p^\star - d^\star,$$

where

$$p^\star = \inf_{x \in \mathcal{F}} f(x), \qquad d^\star = \sup_{\mu \geq 0, \lambda} q(\lambda, \mu).$$

By weak duality, the duality gap is always nonnegative.

Theorem 3.7 (Strong duality under Slater's condition)**.** Consider the convex problem

$$\begin{aligned} \min_x \quad & f(x) \\ \text{subject to} \quad & h(x) = 0, \\ & g(x) \leq 0, \end{aligned}$$

where f and each g_j are convex and the equality constraints are affine. Assume there exists a strictly feasible point $\bar{x}$ such that

$$h(\bar{x}) = 0, \qquad g_j(\bar{x}) < 0 \quad \text{for all } j.$$

Then

$$p^\star = d^\star.$$

Proof sketch. The strict feasibility assumption is Slater's condition. It implies that the convex feasible set has nonempty relative interior, which rules out pathological boundary degeneracies. One then applies a supporting-hyperplane theorem to the convex set of achievable pairs

$$\{(g(x), f(x)) : h(x) = 0\}.$$

The supporting hyperplane separating the primal optimum from the strictly infeasible region produces nonnegative multipliers $\mu^\star$ and equality multipliers $\lambda^\star$, yielding a saddle point of the Lagrangian and hence equality of primal and dual optimal values. ■

Remark. Strong duality is one of the central reasons convex optimization is tractable. When it holds, solving the dual is not merely a lower-bounding exercise; it recovers the exact primal optimum.

3.6 Quadratic Programming

Definition 3.12 (Quadratic program). A *quadratic program* (QP) has the form

$$\begin{aligned} \min_{x} \quad & \frac{1}{2}x^\top Hx + c^\top x \\ \text{subject to} \quad & Ax = b, \\ & Gx \leq h, \end{aligned}$$

where $H = H^\top$. If $H \succeq 0$, then the objective is convex.

Remark. QPs are the workhorse subproblems of modern optimal control. Linear MPC is a QP solved online at every time step, and SQP methods for nonlinear optimal control repeatedly solve structured QPs.

Proposition 3.2 (Equality-constrained QP KKT system). Consider the equality-constrained QP

$$\begin{aligned} \min_{x} \quad & \frac{1}{2}x^\top Hx + c^\top x \\ \text{subject to} \quad & Ax = b, \end{aligned}$$

where $H = H^\top$. Any KKT point $(x^\star, \lambda^\star)$ satisfies the linear system

$$\begin{bmatrix} H & A^\top \\ A & 0 \end{bmatrix} \begin{bmatrix} x^\star \\ \lambda^\star \end{bmatrix} = \begin{bmatrix} -c \\ b \end{bmatrix}.$$

Proof. The Lagrangian is

$$\mathcal{L}(x, \lambda) = \frac{1}{2}x^\top Hx + c^\top x + \lambda^\top (Ax - b).$$

Stationarity gives

$$Hx + c + A^\top \lambda = 0.$$

Primal feasibility gives

$$Ax = b.$$

Writing these equations together yields the stated block linear system. ■

Example (A small QP solved by hand). Consider

$$\begin{aligned} \min_{x_1, x_2} \quad & \frac{1}{2}(x_1^2 + 2x_2^2) - x_1 - 2x_2 \\ \text{subject to} \quad & x_1 + x_2 = 1. \end{aligned}$$

Here

$$H = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}, \qquad c = \begin{bmatrix} -1 \\ -2 \end{bmatrix}, \qquad A = \begin{bmatrix} 1 & 1 \end{bmatrix}.$$

The KKT system is

$$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 2 & 1 \\ 1 & 1 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \lambda \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}.$$

From stationarity,

$$x_1 + \lambda = 1, \qquad 2x_2 + \lambda = 2.$$

So

$$x_1 = 1 - \lambda, \qquad x_2 = 1 - \frac{\lambda}{2}.$$

Using $x_1 + x_2 = 1$,

$$1 - \lambda + 1 - \frac{\lambda}{2} = 1 \qquad \Longrightarrow \qquad \lambda = \frac{2}{3}.$$

Hence

$$x_1^\star = \frac{1}{3}, \qquad x_2^\star = \frac{2}{3}.$$

Since $H \succ 0$, the problem is strictly convex, so this feasible KKT point is the unique global minimizer.

3.7 Interior-Point Methods

Remark. KKT conditions describe optimal points, but they do not yet constitute a numerically robust algorithm for inequality-constrained problems. One must still cope with complementarity,

$$\mu_i g_i(x) = 0,$$

which is nonsmooth as an equation in the pair (g_i, μ_i). Interior-point methods replace this singular condition by a smooth perturbed relation and then drive the perturbation to zero.

Definition 3.13 (Logarithmic barrier formulation)**.** Consider the problem

$$\begin{aligned} \min_x \quad & f(x) \\ \text{subject to} \quad & c(x) = 0, \\ & g(x) \le 0, \end{aligned}$$

with $g = (g_1, \dots, g_m)^\top$. For a barrier parameter $\mu > 0$, the logarithmic barrier subproblem is

$$\begin{aligned} \min_x \quad & f(x) - \mu \sum_{i=1}^m \log\big(-g_i(x)\big) \\ \text{subject to} \quad & c(x) = 0, \end{aligned}$$

defined only on the strict interior

$$g_i(x) < 0, \qquad i = 1, \dots, m.$$

Remark (Slack-variable form)**.** It is often cleaner to introduce slack variables

$$s \in \mathbb{R}^m, \qquad s > 0,$$

and rewrite the inequalities as equalities

$$g(x) + s = 0.$$

The barrier problem then becomes

$$\begin{aligned} \min_{x,s} \quad & f(x) - \mu \sum_{i=1}^m \log(s_i) \\ \text{subject to} \quad & c(x) = 0, \\ & g(x) + s = 0. \end{aligned}$$

This formulation is the standard entry point for primal-dual interior-point methods because positivity is handled entirely through the barrier term.

Definition 3.14 (Central path). A family of primal-dual points

$$(x(\mu), s(\mu), \lambda(\mu), \nu(\mu)), \qquad \mu > 0,$$

is called the *central path* if it satisfies

$$\begin{aligned} \nabla f(x) + \nabla c(x)^\top \lambda + \nabla g(x)^\top \nu &= 0, \\ c(x) &= 0, \\ g(x) + s &= 0, \\ S\nu &= \mu \mathbf{1}, \\ s &> 0, \\ \nu &> 0, \end{aligned}$$

where $S = \operatorname{diag}(s_1, \ldots, s_m)$.

Proposition 3.3 (Perturbed KKT system for the barrier problem). The KKT conditions of the slack-variable barrier problem are exactly the central-path equations from Definition 3.14.

Proof. Form the Lagrangian

$$\mathcal{L}_\mu(x, s, \lambda, \nu) = f(x) - \mu \sum_{i=1}^{m} \log(s_i) + \lambda^\top c(x) + \nu^\top (g(x) + s).$$

Stationarity with respect to x gives

$$\nabla f(x) + \nabla c(x)^\top \lambda + \nabla g(x)^\top \nu = 0.$$

Stationarity with respect to s gives, componentwise,

$$-\frac{\mu}{s_i} + \nu_i = 0,$$

or equivalently

$$s_i \nu_i = \mu.$$

The equality constraints of the barrier problem are

$$c(x) = 0, \qquad g(x) + s = 0.$$

Finally, the logarithm forces $s_i > 0$, and the relation $\nu_i = \mu / s_i$ then implies $\nu_i > 0$. These are exactly the central-path equations. ■

Theorem 3.8 (Barrier path converges to KKT as $\mu \downarrow 0$). Suppose $(x(\mu), s(\mu), \lambda(\mu), \nu(\mu))$ is a bounded family of central-path points for $\mu \downarrow 0$. Then every accumulation point $(x^\star, s^\star, \lambda^\star, \nu^\star)$ satisfies the KKT conditions of the original nonlinear program:

$$\begin{aligned} \nabla f(x^\star) + \nabla c(x^\star)^\top \lambda^\star + \nabla g(x^\star)^\top \nu^\star &= 0, \\ c(x^\star) &= 0, \\ g(x^\star) + s^\star &= 0, \\ s^\star &\geq 0, \\ \nu^\star &\geq 0, \\ S^\star \nu^\star &= 0. \end{aligned}$$

Proof. Stationarity and primal feasibility hold for every μ, so they pass to the limit by continuity. Since each $s_i(\mu) > 0$ and $\nu_i(\mu) > 0$, the limit satisfies

$$s_i^\star \geq 0, \qquad \nu_i^\star \geq 0.$$

Finally,

$$s_i(\mu)\nu_i(\mu) = \mu$$

for every component, and therefore

$$s_i^\star \nu_i^\star = \lim_{\mu \downarrow 0} \mu = 0.$$

This is complementary slackness. Hence any accumulation point is a KKT point of the original problem. ■

Algorithm 6: Primal-Dual Interior-Point Method

Input: Initial strictly feasible or nearly feasible guess (x, s, λ, ν) with $s > 0$, $\nu > 0$; barrier reduction rule; tolerances

while *KKT residuals and complementarity are not sufficiently small* **do**
- Choose a target complementarity level $\mu > 0$
- Linearize the perturbed KKT equations around the current iterate
- Solve the resulting primal-dual Newton system for $(\Delta x, \Delta s, \Delta\lambda, \Delta\nu)$
- Choose primal and dual step lengths so that

$$s + \alpha_p \Delta s > 0, \qquad \nu + \alpha_d \Delta \nu > 0$$

  remain strictly positive
- Update the iterate and reduce μ according to the centering strategy

Output: Approximate primal-dual KKT point

Remark (Why interior-point methods matter for optimal control)**.** When direct collocation or multiple shooting transcribes an optimal-control problem into a large sparse NLP, interior-point methods are often the default choice. Their Newton systems respect the banded time structure, and the central-path viewpoint handles large numbers of inequality constraints without explicit active-set combinatorics.

3.8 Regularization of KKT Systems

Remark. Newton steps for equality-constrained QPs and SQP subproblems require solving saddle-point systems. Even when the original optimization problem is well posed, these linear systems can become singular or can have the wrong inertia if the Hessian model is indefinite on the constraint tangent space.

Definition 3.15 (Regularized KKT matrix)**.** For an equality-constrained quadratic model

$$\begin{aligned} \min_p \quad & \frac{1}{2} p^\top W p + g^\top p \\ \text{subject to} \quad & Ap + c = 0, \end{aligned}$$

a standard regularized KKT system is

$$\begin{bmatrix} W + \delta I & A^\top \\ A & -\gamma I \end{bmatrix} \begin{bmatrix} p \\ y \end{bmatrix} = - \begin{bmatrix} g \\ c \end{bmatrix},$$

where $\delta \geq 0$ and $\gamma \geq 0$ are regularization parameters.

Proposition 3.4 (Quasi-definiteness under regularization)**.** If

$$W + \delta I \succ 0$$

and $\gamma > 0$, then the regularized KKT matrix in Definition 3.15 is nonsingular. More precisely, it has positive definite $(1, 1)$ block and negative definite Schur complement in the dual variables.

Proof. Since $W + \delta I \succ 0$, the primal block is invertible. Taking its Schur complement gives

$$-\gamma I - A(W + \delta I)^{-1} A^\top.$$

The matrix

$$A(W + \delta I)^{-1} A^\top$$

is positive semidefinite, so the Schur complement is strictly negative definite because $\gamma > 0$. Therefore both the primal block and its Schur complement are nonsingular, which implies nonsingularity of the full block matrix. ■

Remark (Inertia correction). Modern NLP solvers do not merely want an invertible KKT matrix; they want one whose inertia matches that of a local constrained minimization problem. In practice, one increases δ and, when necessary, γ until the factorization has the desired pivot structure. This is called *inertia correction*. It is the constrained analogue of adding Levenberg–Marquardt damping in unconstrained Newton methods.

3.9 Merit Functions and Line Search for Constrained Problems

Definition 3.16 (ℓ_1 exact-penalty merit function). For the nonlinear program

$$\begin{aligned} \min_x \quad & f(x) \\ \text{subject to} \quad & c(x) = 0, \\ & g(x) \le 0, \end{aligned}$$

a standard globalization device is the merit function

$$\Phi_\rho(x) = f(x) + \rho \|c(x)\|_1 + \rho \| \max\{0, g(x)\}\|_1,$$

where $\rho > 0$ is a penalty parameter.

Remark (Why a merit function is needed). A Newton or SQP step may improve stationarity while worsening feasibility, or vice versa. A merit function converts this vector-valued notion of progress into a scalar quantity suitable for line search. The price is that one must choose ρ large enough that feasibility is not underweighted.

Proposition 3.5 (Merit decrease for linearized feasible SQP steps). Consider an SQP step p_k obtained from a QP subproblem whose linearized constraints are satisfied:

$$c(x_k) + \nabla c(x_k) p_k = 0,$$

and whose active linearized inequalities are enforced analogously. If the Hessian model is positive definite on the linearized tangent space and ρ is chosen sufficiently large relative to the multiplier estimates, then p_k is a descent direction for the merit function Φ_ρ from Definition 3.16.

Proof sketch. The directional derivative of Φ_ρ splits into an objective part and a feasibility part. The QP step is constructed to reduce the quadratic model of the Lagrangian, so on the tangent space it gives negative first-order change in the stationarity part. The exact-penalty term contributes

$$\rho\, D(\|c(x)\|_1 + \| \max\{0, g(x)\}\|_1)\, [p_k].$$

Because the QP step also reduces the linearized constraint residuals, the feasibility contribution is nonpositive. Choosing ρ larger than the current multiplier magnitudes ensures that any possible objective increase caused by imperfect multiplier estimates is dominated by the penalty on infeasibility. Hence the total directional derivative is negative. ■

Algorithm 7: SQP with Merit-Function Line Search

Input: Initial guess x_0, multiplier estimate (λ_0, μ_0), Hessian model W_0, penalty parameter $\rho > 0$

for $k = 0, 1, 2, \ldots$ *until convergence* **do**

- Build the QP subproblem by quadratic approximation of the Lagrangian and linearization of the constraints
- Regularize the QP KKT system if needed and solve for the step p_k and multiplier update
- Choose a step size $\alpha_k \in (0, 1]$ by backtracking until

$$\Phi_\rho(x_k + \alpha_k p_k) \leq \Phi_\rho(x_k) + \sigma \alpha_k D\Phi_\rho(x_k)[p_k]$$

  holds for some $\sigma \in (0, 1)$
- Update

$$x_{k+1} = x_k + \alpha_k p_k$$

  and refresh the Hessian model and multiplier estimates

Output: Approximate KKT point

Remark (Augmented Lagrangian versus pure penalty globalization)**.** The ℓ_1 merit function is simple and effective, but many implementations instead use augmented-Lagrangian merit functions. The reason is conditioning: pure penalties require large ρ values, while multiplier terms let the solver enforce feasibility without forcing the quadratic model to become extremely stiff.

Remark. Lecture 3 gives the static constrained-optimization language used repeatedly later:

- Lagrange multipliers become costates in optimal control,
- KKT systems become the algebraic core of QP and NLP solvers,
- duality becomes the conceptual bridge between primal trajectory optimization and multiplier-based methods.

In that sense, constrained optimization is not a side topic; it is the algebraic backbone of modern optimal control computation.

Part II
Dynamic Optimal Control

Lecture 4: Dynamic Programming and the Calculus of Optimal Control

Intuition. Static optimization chooses a point. Optimal control chooses an entire signal or trajectory. That single change transforms the problem fundamentally: decisions are now coupled through dynamics, information flows over time, and optimality can be analyzed either by backward induction on value functions or by forward-backward necessary conditions. Bellman and Pontryagin are therefore not competing viewpoints but complementary ones:

- dynamic programming gives a sufficient characterization through the value function,
- Pontryagin's minimum principle gives necessary conditions through a Hamiltonian system.

This lecture introduces both viewpoints at the conceptual level required for the later computational lectures.

4.1 Optimal Control Problem Formulation

Definition 4.1 (Continuous-time optimal control problem in Bolza form)**.** A continuous-time optimal control problem (OCP) in Bolza form is

$$\begin{aligned} \min_{u(\cdot)} \quad & \phi(x(t_f), t_f) + \int_{t_0}^{t_f} L(t, x(t), u(t))\, dt \\ \text{subject to} \quad & \dot{x}(t) = f(t, x(t), u(t)), \qquad t \in [t_0, t_f], \\ & x(t_0) = x_0, \\ & u(t) \in \mathcal{U}. \end{aligned}$$

Here

$$x(t) \in \mathbb{R}^n \qquad \text{and} \qquad u(t) \in \mathbb{R}^m$$

are the state and control, L is the running cost, and ϕ is the terminal cost.

Definition 4.2 (Discrete-time optimal control problem)**.** A finite-horizon discrete-time OCP is

$$\begin{aligned} \min_{\{u_k\}_{k=0}^{N-1}} \quad & \phi_N(x_N) + \sum_{k=0}^{N-1} \ell_k(x_k, u_k) \\ \text{subject to} \quad & x_{k+1} = f_k(x_k, u_k), \qquad k = 0, \dots, N-1, \\ & x_0 \text{ given}, \\ & u_k \in \mathcal{U}_k. \end{aligned}$$

Notation (Bolza, Mayer, and Lagrange forms)**.** The three standard objective forms are:

- *Bolza:*
$$\phi(x(t_f), t_f) + \int_{t_0}^{t_f} L(t, x, u)\, dt,$$
- *Mayer:* terminal-cost only, i.e. $L \equiv 0$,
- *Lagrange:* running-cost only, i.e. $\phi \equiv 0$.

These forms are equivalent up to state augmentation. For example, Bolza can be converted into

Mayer form by introducing an auxiliary state

$$\dot{z}(t) = L(t, x(t), u(t)), \qquad z(t_0) = 0,$$

and then minimizing

$$\phi(x(t_f), t_f) + z(t_f).$$

Remark. In robotics, the running cost typically encodes tracking error, control effort, collision penalties, or smoothness, while the dynamics encode rigid-body motion, actuator dynamics, or learned models. The terminal cost enforces the quality of the final state or approximates the tail cost beyond the planning horizon.

4.2 Dynamic Programming in Discrete Time

Intuition. Dynamic programming is based on a brutally simple observation: if a trajectory is optimal from time k onward, then its suffix from time $k+1$ onward must already be optimal for the state reached at time $k+1$. Otherwise one could replace that suffix by a cheaper one and improve the whole trajectory. This recursive structure is what makes backward solution possible.

Theorem 4.1 (Bellman's principle of optimality)**.** Consider a finite-horizon discrete-time OCP. Let

$$\{x_k^\star, u_k^\star\}_{k=0}^{N-1}$$

be an optimal state-control sequence for the initial condition x_0. Then for every intermediate index $j \in \{0, \dots, N-1\}$, the truncated sequence

$$\{x_k^\star, u_k^\star\}_{k=j}^{N-1}$$

is optimal for the subproblem starting from the state $x_j^\star$ at time j.

Proof sketch. Assume the tail from time j onward were not optimal for the state $x_j^\star$. Then there would exist another admissible tail with smaller cost and the same initial state $x_j^\star$. Replacing the original tail by this cheaper one would preserve feasibility of the whole trajectory and strictly reduce the total cost, contradicting optimality of the original solution. Therefore every optimal trajectory has optimal tails. ■

Definition 4.3 (Value function)**.** The *value function* at stage k is

$$V_k(x) = \inf_{\{u_i\}_{i=k}^{N-1}} \left[\phi_N(x_N) + \sum_{i=k}^{N-1} \ell_i(x_i, u_i) \right],$$

where the infimum is taken over all admissible control sequences satisfying

$$x_{i+1} = f_i(x_i, u_i), \qquad x_k = x.$$

The terminal value is

$$V_N(x) = \phi_N(x).$$

Theorem 4.2 (Bellman equation)**.** For the finite-horizon discrete-time OCP,

$$V_N(x) = \phi_N(x),$$

and for $k = N-1, \dots, 0$,

$$V_k(x) = \min_{u \in \mathcal{U}_k} \Big\{ \ell_k(x, u) + V_{k+1}(f_k(x, u)) \Big\}.$$

Proof sketch. Fix the state x at time k. Any admissible control choice first incurs the immediate stage cost $\ell_k(x, u)$, after which the system moves to the next state

$$x^+ = f_k(x, u).$$

From that state onward, the optimal remaining cost is by definition $V_{k+1}(x^+)$. Therefore the total cost generated by choosing u now and behaving optimally afterward is

$$\ell_k(x, u) + V_{k+1}(f_k(x, u)).$$

Minimizing over the current control produces the Bellman recursion. ■

Example (A scalar dynamic-programming calculation). Consider the discrete-time system

$$x_{k+1} = x_k + u_k$$

with horizon $N = 2$ and cost

$$J = x_2^2 + u_0^2 + u_1^2.$$

Then

$$V_2(x) = x^2.$$

At stage $k = 1$,

$$V_1(x) = \min_u \left\{u^2 + (x + u)^2\right\}.$$

Differentiating with respect to u,

$$2u + 2(x + u) = 0 \qquad \Longrightarrow \qquad u_1^\star(x) = -\frac{x}{2}.$$

Substituting back gives

$$V_1(x) = \frac{x^2}{2}.$$

At stage $k = 0$,

$$V_0(x) = \min_u \left\{u^2 + \frac{1}{2}(x + u)^2\right\}.$$

Differentiating,

$$2u + (x + u) = 0 \qquad \Longrightarrow \qquad u_0^\star(x) = -\frac{x}{3}.$$

Hence

$$V_0(x) = \frac{x^2}{3}.$$

This example shows the characteristic backward structure of dynamic programming: first solve the last stage, then substitute that solution into the previous stage.

Remark (Curse of dimensionality). If the state dimension is n and one discretizes each state coordinate using M grid points, then storing the value function requires on the order of M^n entries. This exponential dependence on dimension is Bellman's curse of dimensionality. Dynamic programming is exact in principle, but rapidly becomes intractable for high-dimensional nonlinear systems.

4.3 The Hamilton–Jacobi–Bellman Equation

Definition 4.4 (Continuous-time value function). For the continuous-time Bolza problem, the value function is

$$V(t, x) = \inf_{u(\cdot)} \left[\phi(x(t_f), t_f) + \int_t^{t_f} L(\tau, x(\tau), u(\tau))\, d\tau\right],$$

where the infimum is over admissible controls starting from the initial condition

$$x(t) = x.$$

Theorem 4.3 (Hamilton–Jacobi–Bellman equation). Assume the value function is sufficiently smooth. Then it satisfies

$$\partial_t V(t,x) + \min_{u \in \mathcal{U}} \left\{ L(t,x,u) + \nabla_x V(t,x)^\top f(t,x,u) \right\} = 0$$

with terminal condition

$$V(t_f, x) = \phi(x, t_f).$$

Proof sketch. Apply the dynamic-programming principle over a short interval $[t, t+\Delta t]$:

$$V(t,x) = \min_u \left\{ \int_t^{t+\Delta t} L(\tau, x(\tau), u(\tau))\, d\tau + V(t+\Delta t, x(t+\Delta t)) \right\}.$$

For small Δt, approximate

$$\int_t^{t+\Delta t} L(\tau, x(\tau), u(\tau))\, d\tau \approx L(t,x,u)\Delta t$$

and

$$x(t+\Delta t) \approx x + f(t,x,u)\Delta t.$$

Expand $V(t+\Delta t, x(t+\Delta t))$ to first order:

$$V(t+\Delta t, x + f\Delta t) \approx V(t,x) + \partial_t V\, \Delta t + \nabla_x V^\top f\, \Delta t.$$

Subtract $V(t,x)$ from both sides, divide by Δt, and let $\Delta t \to 0$. The result is the HJB equation. ■

Theorem 4.4 (Verification theorem). Suppose a smooth function $W(t,x)$ satisfies the HJB equation with the correct terminal condition, and suppose there exists a feedback law

$$u^\star(t,x) \in \arg\min_{u \in \mathcal{U}} \left\{ L(t,x,u) + \nabla_x W(t,x)^\top f(t,x,u) \right\}.$$

Then

$$W(t,x) = V(t,x),$$

and the feedback law $u^\star$ is optimal.

Proof sketch. Let $x(\cdot)$ be any admissible trajectory. Along that trajectory,

$$\frac{d}{dt} W(t, x(t)) = \partial_t W(t, x(t)) + \nabla_x W(t, x(t))^\top f(t, x(t), u(t)).$$

By the HJB equation, the right-hand side is bounded below by $-L(t, x(t), u(t))$, with equality for the minimizing control $u^\star$. Integrate from t to t_f, use the terminal condition, and rearrange. This shows that every admissible control has cost at least $W(t,x)$, while the minimizing feedback attains exactly that value. Therefore $W = V$ and $u^\star$ is optimal. ■

Remark (HJB is a sufficient theory). HJB does more than characterize candidate minimizers: if one can solve the PDE and identify the minimizer in the Hamiltonian minimization, then one has already solved the optimal control problem globally. This is much stronger than first-order necessary conditions. The cost is that solving the PDE is usually impossible in high dimensions.

Remark (Preview: HJB and LQR). For linear dynamics

$$\dot{x} = Ax + Bu$$

and quadratic cost, the ansatz

$$V(t,x) = x^\top P(t) x$$

reduces HJB to a matrix differential equation for $P(t)$, namely the Riccati equation. This is the origin of LQR.

4.4 Pontryagin's Minimum Principle

Definition 4.5 (Hamiltonian). For the continuous-time OCP

$$J = \phi(x(t_f), t_f) + \int_{t_0}^{t_f} L(t, x(t), u(t))\, dt,$$

the *Hamiltonian* is

$$H(t,x,u,p) = L(t,x,u) + p^\top f(t,x,u),$$

where

$$p(t) \in \mathbb{R}^n$$

is the *costate* or *adjoint* variable.

Intuition. The costate plays exactly the role that Lagrange multipliers played in Theorem 3.2. There the constraint was algebraic, such as $Ax = b$ or $g(x) \le 0$. Here the constraint is differential:

$$\dot{x} - f(t,x,u) = 0.$$

The multiplier for this infinite family of constraints is no longer a vector but a time-varying function $p(t)$.

Theorem 4.5 (Pontryagin's Minimum Principle). Consider the fixed-final-time problem

$$\begin{aligned} \min_{u(\cdot)} \quad & \phi(x(t_f)) + \int_{t_0}^{t_f} L(t, x(t), u(t))\, dt \\ \text{subject to} \quad & \dot{x}(t) = f(t, x(t), u(t)), \\ & x(t_0) = x_0. \end{aligned}$$

If $u^\star(\cdot)$ is optimal with corresponding state $x^\star(\cdot)$, then there exists a costate $p(\cdot)$ such that:

$$\begin{aligned} \dot{x}^\star(t) &= \partial_p H(t, x^\star(t), u^\star(t), p(t)), \\ \dot{p}(t) &= -\partial_x H(t, x^\star(t), u^\star(t), p(t)), \\ u^\star(t) &\in \arg\min_{u \in \mathcal{U}} H(t, x^\star(t), u, p(t)), \\ p(t_f) &= \nabla_x \phi(x^\star(t_f)). \end{aligned}$$

Additional transversality conditions appear when the terminal state or terminal time is free.

Proof outline. One writes an augmented functional by appending the dynamics constraint with a multiplier $p(t)$:

$$\phi(x(t_f)) + \int_{t_0}^{t_f} \Big[L(t,x,u) + p^\top (f(t,x,u) - \dot{x}) \Big]\, dt.$$

Then one computes the first variation with respect to x, u, and the multiplier. Integration by parts moves the derivative off the state variation, which produces the adjoint equation and the terminal transversality term. Stationarity with respect to the control gives the Hamiltonian minimization condition. A fully rigorous proof is usually carried out using needle variations or an infinite-dimensional multiplier theorem. ■

Remark (PMP is necessary, not sufficient). Pontryagin's principle characterizes optimal trajectories by first-order conditions. Like KKT conditions, these may also be satisfied at nonoptimal extremals or saddle-type solutions. Additional convexity or verification arguments are required for sufficiency.

Example (Minimum-energy control of an integrator). Consider

$$\begin{aligned} \min_{u(\cdot)} \quad & \frac{1}{2}\int_0^T u(t)^2\,dt \\ \text{subject to} \quad & \dot{x}(t) = u(t), \\ & x(0) = x_0, \quad x(T) = 0. \end{aligned}$$

The Hamiltonian is

$$H(x,u,p) = \frac{1}{2}u^2 + pu.$$

The optimality condition gives

$$\partial_u H = u + p = 0 \qquad \Longrightarrow \qquad u^\star(t) = -p(t).$$

The costate equation is

$$\dot{p}(t) = -\partial_x H = 0,$$

so $p(t) \equiv c$ is constant and therefore

$$u^\star(t) \equiv -c.$$

The state then evolves according to

$$x(t) = x_0 - ct.$$

Imposing the terminal condition $x(T) = 0$ gives

$$c = \frac{x_0}{T},$$

so

$$u^\star(t) = -\frac{x_0}{T}.$$

The minimum-energy solution is thus constant control.

Example (A bang-bang minimum-time problem). Consider

$$\begin{aligned} \min_{u(\cdot),T} \quad & T \\ \text{subject to} \quad & \dot{x}(t) = u(t), \\ & |u(t)| \le 1, \\ & x(0) = x_0 > 0, \quad x(T) = 0. \end{aligned}$$

The running cost is $L \equiv 1$, so the Hamiltonian is

$$H(x,u,p) = 1 + pu.$$

The costate equation gives

$$\dot{p}(t) = 0,$$

so p is constant. Minimizing $1 + pu$ over $u \in [-1,1]$ yields

$$u^\star(t) = \begin{cases} -1, & p > 0, \\ +1, & p < 0. \end{cases}$$

To drive a positive initial state to the origin in minimum time, the optimal control is

$$u^\star(t) \equiv -1,$$

which lies on the boundary of the control set at all times. This is the defining bang-bang phenomenon.

4.5 The Fundamental Connection Between Dynamic Programming and PMP

Remark. At a conceptual level, HJB and PMP appear to solve different problems. HJB seeks the value function itself and is therefore globally sufficient but computationally intractable in high dimension. PMP introduces a costate and gives a two-point boundary-value problem whose solution is only locally necessary. The two theories meet when the value function is smooth: the costate is then the gradient of the value function evaluated along the optimal trajectory.

Theorem 4.6 (Costate equals gradient of the value function)**.** Suppose the value function

$$V : [t_0, t_f] \times \mathbb{R}^n \to \mathbb{R}$$

is continuously differentiable, and suppose an optimal feedback $u^\star(t, x)$ exists and is sufficiently regular that the optimal trajectory $x^\star(\cdot)$ is classical. Define

$$p(t) := \nabla_x V(t, x^\star(t)).$$

Then $p(\cdot)$ satisfies the PMP adjoint equation and terminal condition:

$$\begin{aligned} \dot{p}(t) &= -\partial_x H(t, x^\star(t), u^\star(t), p(t)), \\ p(t_f) &= \nabla_x \phi(x^\star(t_f)). \end{aligned}$$

Moreover,

$$u^\star(t) \in \arg\min_{u \in \mathcal{U}} H(t, x^\star(t), u, p(t)).$$

Proof sketch. Because

$$V(t_f, x) = \phi(x),$$

the terminal condition is immediate:

$$p(t_f) = \nabla_x V(t_f, x^\star(t_f)) = \nabla_x \phi(x^\star(t_f)).$$

Next, the HJB equation reads

$$\partial_t V(t, x) + \min_{u \in \mathcal{U}} \left\{ L(t, x, u) + \nabla_x V(t, x)^\top f(t, x, u) \right\} = 0.$$

Along the optimal trajectory, the minimizing control is $u^\star(t) = u^\star(t, x^\star(t))$, so

$$\partial_t V(t, x^\star(t)) + H(t, x^\star(t), u^\star(t), p(t)) = 0.$$

Differentiate this identity with respect to x along the optimal feedback branch. Because $u^\star$ minimizes the Hamiltonian, the envelope principle removes the derivative of the minimizer itself and yields

$$\partial_t \nabla_x V(t, x^\star(t)) + \partial_x H(t, x^\star(t), u^\star(t), p(t)) + V_{xx}(t, x^\star(t))\, \dot{x}^\star(t) = 0.$$

Now use the chain rule

$$\dot{p}(t) = \frac{d}{dt} \nabla_x V(t, x^\star(t)) = \partial_t \nabla_x V(t, x^\star(t)) + V_{xx}(t, x^\star(t))\, \dot{x}^\star(t),$$

and substitute

$$\dot{x}^\star(t) = f(t, x^\star(t), u^\star(t)).$$

The terms combine to give

$$\dot{p}(t) = -\partial_x H(t, x^\star(t), u^\star(t), p(t)).$$

Finally, the minimizing control in the HJB Hamiltonian is exactly the minimizer in the PMP Hamiltonian once we identify

$$p(t) = \nabla_x V(t, x^\star(t)).$$

■

Remark (Discrete-time analogue). In discrete time, the same argument leads to

$$\lambda_k = \nabla_x V_k(x_k^\star),$$

where λ_k is the multiplier on the dynamics constraint at stage k. Section 6.3 will make this identity explicit in the linear-quadratic setting, where the relation becomes

$$\lambda_k = P_k x_k^\star.$$

4.6 Calculus of Variations

Definition 4.6 (Variational problem). A basic problem in the calculus of variations seeks a function $x(\cdot)$ minimizing

$$J[x] = \int_{t_0}^{t_1} F(t, x(t), \dot{x}(t))\, dt$$

subject to fixed endpoint conditions

$$x(t_0) = x_0, \qquad x(t_1) = x_1.$$

Definition 4.7 (Admissible variation). An *admissible variation* is a smooth perturbation

$$x_\varepsilon(t) = x(t) + \varepsilon\eta(t)$$

where the variation function satisfies

$$\eta(t_0) = \eta(t_1) = 0.$$

Theorem 4.7 (Euler–Lagrange equation). If $x^\star(\cdot)$ is a smooth minimizer of the variational problem, then it satisfies

$$\frac{d}{dt}\left(\partial_{\dot{x}} F(t, x^\star(t), \dot{x}^\star(t))\right) = \partial_x F(t, x^\star(t), \dot{x}^\star(t)).$$

Proof sketch. Insert the varied path $x_\varepsilon = x^\star + \varepsilon\eta$ into the functional and differentiate with respect to ε at $\varepsilon = 0$:

$$0 = \frac{d}{d\varepsilon}\Big|_{\varepsilon=0} J[x_\varepsilon] = \int_{t_0}^{t_1} (\partial_x F\, \eta + \partial_{\dot{x}} F\, \dot{\eta})\, dt.$$

Integrate the second term by parts. The boundary term vanishes because η vanishes at both endpoints. This leaves

$$\int_{t_0}^{t_1} \left(\partial_x F - \frac{d}{dt}\partial_{\dot{x}} F\right) \eta(t)\, dt = 0.$$

Since this holds for every admissible variation η, the fundamental lemma of the calculus of variations implies the Euler–Lagrange equation. ■

Remark. The Euler–Lagrange equation is the special case of PMP obtained when the control enters only as the velocity variable and no explicit control constraints are present. Pontryagin's principle is therefore a true generalization of classical variational calculus.

Lecture 5: Numerical Methods for Trajectory Optimization

Intuition. Section 4.3 and Section 4.4 supplied the theory: HJB gives a PDE, and PMP gives a coupled two-point boundary value problem. Those descriptions are conceptually complete but rarely computationally friendly. Real trajectory optimization therefore proceeds by *numerical transcriptions*: either solve the PMP boundary value problem indirectly, or discretize the trajectories and solve a finite-dimensional nonlinear program directly. This lecture explains the main families of methods used in practice.

5.1 The Two-Point Boundary Value Problem from PMP

Remark. For a fixed-final-time problem, Pontryagin's principle produces the coupled system

$$\begin{aligned}\dot{x}(t) &= \partial_p H(t, x(t), u(t), p(t)),\\ \dot{p}(t) &= -\partial_x H(t, x(t), u(t), p(t)),\\ u(t) &\in \arg\min_{v \in \mathcal{U}} H(t, x(t), v, p(t)).\end{aligned}$$

The state has an initial condition

$$x(t_0) = x_0,$$

while the costate typically has a terminal condition

$$p(t_f) = \nabla_x \phi(x(t_f))$$

or some transversality variant. This is a *two-point boundary value problem* (TPBVP): part of the boundary data is specified at t_0, part at t_f.

Definition 5.1 (TPBVP structure)**.** A generic TPBVP arising from optimal control can be written abstractly as

$$\begin{aligned}\dot{z}(t) &= F(t, z(t)),\\ \Psi(z(t_0), z(t_f)) &= 0,\end{aligned}$$

where

$$z(t) = \begin{bmatrix} x(t) \\ p(t) \end{bmatrix}.$$

The difficulty is that the initial state is known, but the initial costate is usually not.

Remark (Why TPBVPs are difficult)**.** The difficulty comes from three simultaneous sources:

- the boundary conditions are split between the beginning and end of the horizon,
- the state and costate dynamics are coupled through the control law,
- the resulting system is frequently nonlinear and sensitive to the unknown costate initialization.

That sensitivity is exactly why indirect shooting can be powerful on easy problems and fragile on hard ones.

5.2 Indirect Shooting Methods

Intuition. Indirect shooting converts the TPBVP into root-finding. One guesses the missing boundary data, integrates the ODEs forward, measures the terminal mismatch, and iterates on the guess until the mismatch vanishes.

Definition 5.2 (Single shooting for a TPBVP). Suppose $x(t_0) = x_0$ is known but $p(t_0)$ is unknown. Let the unknown initial costate be denoted by

$$s = p(t_0).$$

Given a guess s, solve the initial value problem

$$\begin{aligned}
\dot{x}(t) &= \partial_p H(t, x(t), u(t), p(t)), \\
\dot{p}(t) &= -\partial_x H(t, x(t), u(t), p(t)), \\
u(t) &\in \arg\min_{v \in \mathcal{U}} H(t, x(t), v, p(t)),
\end{aligned}$$

with initial conditions

$$x(t_0) = x_0, \qquad p(t_0) = s.$$

The terminal mismatch defines the *shooting residual*

$$R(s) = \Psi(z_s(t_0), z_s(t_f)).$$

The TPBVP is solved when

$$R(s) = 0.$$

Algorithm 8: Indirect Single Shooting

Input: Initial state x_0, horizon $[t_0, t_f]$, initial guess $s^{(0)}$ for the unknown costate, tolerance ε

```
for r = 0, 1, 2, ... do
    Set p(t_0) = s^(r) and solve the PMP state-costate ODEs forward in time
    Evaluate the terminal residual R(s^(r))
    if ||R(s^(r))|| ≤ ε then
        return the associated trajectory
    Update the guess using Newton, Gauss–Newton, or a quasi-Newton step on R(s) = 0
```

Remark. Single shooting reduces a boundary value problem to a nonlinear system in the unknown initial costate (and possibly final time). The price of this reduction is sensitivity: a small perturbation in the initial guess can cause a large perturbation in the terminal state for unstable or long-horizon systems.

Example (Indirect shooting for minimum-energy control of a double integrator). Consider the fixed-time problem

$$\begin{aligned}
\min_{u(\cdot)} \quad & \frac{1}{2}\int_0^T u(t)^2\, dt \\
\text{subject to} \quad & \dot{x}_1 = x_2, \\
& \dot{x}_2 = u, \\
& x(0) = \begin{bmatrix} x_{10} \\ x_{20} \end{bmatrix}, \qquad x(T) = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.
\end{aligned}$$

The Hamiltonian is

$$H = \frac{1}{2}u^2 + p_1 x_2 + p_2 u.$$

PMP gives

$$u^\star = -p_2, \qquad \dot{p}_1 = 0, \qquad \dot{p}_2 = -p_1.$$

Write

$$p_1(t) = a, \qquad p_2(t) = -at + b,$$

where (a, b) are the unknown initial costate parameters. Then

$$u^\star(t) = at - b.$$

Integrating the state equations yields

$$x_2(t) = x_{20} + \frac{a}{2}t^2 - bt,$$

$$x_1(t) = x_{10} + x_{20}t + \frac{a}{6}t^3 - \frac{b}{2}t^2.$$

The terminal conditions $x_1(T) = 0$ and $x_2(T) = 0$ define the shooting equations

$$R(a, b) = \begin{bmatrix} x_{10} + x_{20}T + \frac{a}{6}T^3 - \frac{b}{2}T^2 \\ x_{20} + \frac{a}{2}T^2 - bT \end{bmatrix} = 0.$$

This is a 2×2 nonlinear root-finding problem, which in this case is actually linear in (a, b). The point of the example is structural: indirect shooting converts the optimal-control boundary conditions into a residual map in the unknown initial costate.

5.3 Direct Single Shooting

Intuition. Direct methods optimize the controls themselves rather than solving the PMP boundary value problem. In direct single shooting, the controls are parameterized and the dynamics are used to eliminate the states entirely by forward simulation.

Definition 5.3 (Direct single shooting)**.** Choose a time grid

$$t_0 < t_1 < \cdots < t_N$$

and parameterize the controls by a finite-dimensional vector

$$w = (w_0, \ldots, w_{N-1}),$$

for example by taking piecewise-constant controls

$$u(t) = w_k \qquad \text{for } t \in [t_k, t_{k+1}).$$

Using a numerical integrator, define the discrete propagation

$$x_{k+1} = F_k(x_k, w_k), \qquad x_0 \text{ given.}$$

The OCP is thereby transcribed to the finite-dimensional problem

$$\min_w \; J_{\text{disc}}(w),$$

where the states are implicit functions of the control parameters.

Algorithm 9: Direct Single Shooting

Input: Initial state x_0, control parameterization w, integrator, discretized objective, NLP tolerance

```
for outer NLP iterations do
    Given w, simulate the dynamics forward to obtain x_1, ..., x_N
    Evaluate the discretized objective and any path/terminal constraint residuals
    Compute derivatives by sensitivities, adjoints, or automatic differentiation
    Update the control parameter vector using an NLP step
```

Remark (Advantages and disadvantages). Direct single shooting has several attractive features:

- very few decision variables if only controls are optimized,
- exact satisfaction of the discretized dynamics by construction,
- easy implementation when a simulator already exists.

It also has serious limitations:

- the map from controls to terminal state can be badly conditioned,
- state constraints are awkward because states are not decision variables,
- numerical integration errors are buried inside the objective and constraints.

5.4 Direct Multiple Shooting

Intuition. Multiple shooting keeps the simulation-based flavor of shooting but introduces intermediate state variables. Instead of forcing one long simulation to carry all information, it breaks the trajectory into shorter segments and enforces continuity between them.

Definition 5.4 (Multiple shooting transcription). Given a grid $t_0 < \cdots < t_N$, introduce decision variables

$$x_0, \ldots, x_N, \qquad u_0, \ldots, u_{N-1}.$$

On each interval $[t_k, t_{k+1}]$, integrate the dynamics with initial value x_k and control parameter u_k to produce the segment endpoint

$$\Phi_k(x_k, u_k).$$

The transcription enforces the continuity constraints

$$x_{k+1} - \Phi_k(x_k, u_k) = 0, \qquad k = 0, \ldots, N-1.$$

Algorithm 10: Direct Multiple Shooting

Input: Initial guess for node states and controls, segment integrator, discretized objective

for *outer NLP iterations* **do**
- For each interval, integrate from x_k under u_k to compute $\Phi_k(x_k, u_k)$
- Assemble continuity defects $x_{k+1} - \Phi_k(x_k, u_k)$
- Assemble the NLP objective and all constraints
- Solve the structured NLP step and update node states and controls

Remark. Multiple shooting is often markedly better conditioned than single shooting because each integration window is short. The state variables at grid points also make path and boundary constraints much easier to impose directly.

Proposition 5.1 (Sparsity structure of multiple shooting). The Jacobian of the multiple-shooting continuity constraints is block sparse. Specifically, each constraint

$$c_k(x_k, x_{k+1}, u_k) = x_{k+1} - \Phi_k(x_k, u_k)$$

depends only on the triplet (x_k, u_k, x_{k+1}).

Proof. By construction, the kth continuity equation compares the next node state x_{k+1} with the endpoint of the segment launched from x_k under u_k. No other node states or controls enter this equation. Therefore the derivative of c_k with respect to any variable outside (x_k, u_k, x_{k+1}) is zero. Stacking the constraints over all intervals produces a block-banded Jacobian. ■

Remark. This block sparsity is one of the reasons multiple shooting is a standard transcription in software such as CasADi- and ACADOS-based pipelines. Sparse linear algebra is not a minor implementation detail here; it is what makes large problems tractable.

5.5 Direct Collocation

Intuition. Collocation abandons explicit forward simulation and instead approximates the state trajectory by low-order polynomials on each interval. The dynamics are enforced at selected collocation points, turning differential equations into algebraic equality constraints.

Definition 5.5 (Trapezoidal collocation)**.** Let the step size be

$$h_k = t_{k+1} - t_k.$$

With node variables (x_k, u_k) and (x_{k+1}, u_{k+1}), trapezoidal collocation enforces

$$x_{k+1} - x_k - \frac{h_k}{2}\big(f(x_k, u_k) + f(x_{k+1}, u_{k+1})\big) = 0.$$

Definition 5.6 (Hermite–Simpson collocation)**.** Hermite–Simpson collocation introduces a midpoint state and control:

$$u_{k+\frac{1}{2}} = \frac{u_k + u_{k+1}}{2},$$

$$x_{k+\frac{1}{2}} = \frac{x_k + x_{k+1}}{2} + \frac{h_k}{8}\big(f(x_k, u_k) - f(x_{k+1}, u_{k+1})\big).$$

The defect constraint is

$$x_{k+1} - x_k - \frac{h_k}{6}\Big(f(x_k, u_k) + 4f(x_{k+\frac{1}{2}}, u_{k+\frac{1}{2}}) + f(x_{k+1}, u_{k+1})\Big) = 0.$$

Remark (Why Hermite–Simpson is more than a formula)**.** The Hermite–Simpson rule is not merely a higher-order defect equation inserted into the NLP. It encodes a local cubic model of the state trajectory and a Simpson-type quadrature of the integral dynamics. That viewpoint explains both its accuracy and its algebraic sparsity.

Proposition 5.2 (Hermite–Simpson midpoint construction)**.** On the interval $[t_k, t_{k+1}]$, Hermite–Simpson uses a cubic Hermite interpolant for the state determined by

$$x_k, \quad x_{k+1}, \quad f(x_k, u_k), \quad f(x_{k+1}, u_{k+1}).$$

The midpoint state formula

$$x_{k+\frac{1}{2}} = \frac{x_k + x_{k+1}}{2} + \frac{h_k}{8}\big(f(x_k, u_k) - f(x_{k+1}, u_{k+1})\big)$$

is exactly the value of this cubic interpolant at the midpoint.

Proof. A cubic Hermite interpolant on $[t_k, t_{k+1}]$ is uniquely specified by endpoint values and endpoint derivatives. Writing the interpolant in normalized time

$$\tau = \frac{t - t_k}{h_k} \in [0, 1],$$

and evaluating the standard Hermite basis functions at

$$\tau = \frac{1}{2},$$

one obtains

$$x\left(t_k + \frac{h_k}{2}\right) = \frac{x_k + x_{k+1}}{2} + \frac{h_k}{8}(\dot{x}_k - \dot{x}_{k+1}).$$

Approximating the endpoint derivatives by the dynamics,

$$\dot{x}_k = f(x_k, u_k), \qquad \dot{x}_{k+1} = f(x_{k+1}, u_{k+1}),$$

yields the stated midpoint formula. ■

Remark (Implementation detail). In many software implementations one does not store a separate midpoint state variable for Hermite–Simpson. Instead, one reconstructs

$$x_{k+\frac{1}{2}}$$

algebraically from the node values and endpoint dynamics. This keeps the NLP smaller while preserving the same local approximation order. Other implementations do introduce midpoint variables, especially when midpoint path constraints or contact conditions must be imposed explicitly.

Remark (Accuracy versus Euler or trapezoidal transcription). Hermite–Simpson is substantially more accurate per interval than a forward-Euler shooting constraint and typically more accurate than the trapezoidal rule on the same mesh. The practical consequence is not merely better asymptotic order: for a fixed accuracy target, one often needs many fewer mesh intervals, which can more than offset the extra algebra per interval.

Algorithm 11: Direct Collocation Transcription

Input: Time grid, initial guess for node states and controls, collocation rule

1 Choose optimization variables containing all node states and controls
2 For each interval, form the collocation defect equations
3 Approximate the continuous cost by a quadrature rule consistent with the transcription
4 Add boundary, path, and control constraints at nodes (and midpoints if needed)
5 Solve the resulting sparse NLP with SQP or an interior-point method

Remark. In direct collocation, both states and controls are explicit decision variables. This is the opposite design choice from single shooting. The reward is flexibility: state constraints, path constraints, and stiff dynamics are often handled more robustly.

Proposition 5.3 (Sparsity of direct collocation NLPs). Direct collocation produces a large but sparse nonlinear program. Each defect constraint couples only neighboring nodes and, in midpoint formulas, a small number of auxiliary midpoint variables.

Proof. The collocation equation on interval k uses only the variables attached to that interval, such as

$$(x_k, u_k, x_{k+1}, u_{k+1})$$

for trapezoidal collocation, or the same tuple together with midpoint data for Hermite–Simpson. Consequently, the Jacobian row corresponding to interval k has nonzero entries only in a small local block. After stacking all intervals, the overall Jacobian is banded or nearly banded, and the Hessian of the Lagrangian inherits comparable locality. ■

Remark. Collocation methods often behave better than explicit shooting on stiff systems because the dynamics are enforced implicitly through algebraic constraints rather than through one long explicit integration chain.

5.6 Transcription to Nonlinear Programming

Definition 5.7 (General NLP form from direct transcription). After discretization, most direct trajectory-optimization methods produce an NLP of the form

$$\begin{aligned} \min_z \quad & F(z) \\ \text{subject to} \quad & c(z) = 0, \\ & d(z) \le 0, \end{aligned}$$

where z stacks the discretized state, control, and possibly time variables.

Remark (Connection to Lecture 3). Once transcribed, the problem is a standard finite-dimensional constrained optimization problem. The KKT conditions, Lagrange multipliers, SQP subproblems, interior-point barrier terms, and dual variables are exactly those of Theorem 3.2. Dynamic optimal control becomes large-scale nonlinear programming after transcription.

Remark (Sparsity and band structure). If the grid is ordered in time, then most constraints only couple nearby time indices. This yields banded Jacobians and structured Hessians of the Lagrangian. Efficient solvers exploit this structure through sparse factorizations, condensing, partial condensing, or Riccati-type eliminations.

Remark (Warm starts and real-time use). Optimal control is often solved repeatedly with slowly changing initial conditions or references. Warm-starting from the previous solution can dramatically reduce solve time. This is one of the key reasons direct methods become viable in real-time MPC and nonlinear receding-horizon control.

5.7 Sequential Quadratic Programming for Transcribed Optimal Control

Remark. Once the trajectory problem has been discretized as in Definition 5.7, one of the most important solver classes is sequential quadratic programming (SQP). SQP should be viewed as Newton's method applied to the KKT system of the transcribed NLP.

Definition 5.8 (SQP subproblem for trajectory optimization). Let

$$\mathcal{L}(z, \lambda, \mu) = F(z) + \lambda^\top c(z) + \mu^\top d(z)$$

be the Lagrangian of the transcribed NLP, where $\mu \ge 0$ corresponds to the inequality constraints. At iterate (z_k, λ_k, μ_k), an SQP step p_k is obtained by solving the quadratic program

$$\begin{aligned} \min_p \quad & \nabla F(z_k)^\top p + \frac{1}{2} p^\top W_k p \\ \text{subject to} \quad & c(z_k) + \nabla c(z_k) p = 0, \\ & d(z_k) + \nabla d(z_k) p \le 0, \end{aligned}$$

where W_k approximates

$$\nabla^2_{zz} \mathcal{L}(z_k, \lambda_k, \mu_k).$$

Remark (Where the sparsity enters). The vector z is ordered by time. Therefore the Jacobians

$$\nabla c(z_k), \qquad \nabla d(z_k)$$

and the Hessian model W_k inherit the block-banded locality of the transcription. The resulting QP is large but highly structured. Sparse SQP codes exploit that structure rather than treating the problem as a generic dense nonlinear program.

Proposition 5.4 (Why SQP is a natural solver for direct transcription). If the transcription mesh is fixed, then every SQP iteration replaces the nonlinear trajectory-optimization problem by a structured QP whose variables are corrections to the state, control, and possibly time variables. The linearized defect constraints preserve exactly the same nearest-neighbor coupling pattern as the original transcription.

Proof. Each defect constraint in single shooting, multiple shooting, or collocation involves only local time-neighbor variables; see Proposition 5.1 and Proposition 5.3. Linearizing such a constraint does not enlarge its support: the Jacobian row remains nonzero only in the same local variable blocks. Therefore the SQP QP has the same banded coupling graph as the original NLP. The QP interpretation is thus faithful to the time-structured geometry of the control problem. ■

Algorithm 12: SQP for a Transcribed Trajectory-Optimization NLP

Input: Initial guess z_0, multiplier estimate (λ_0, μ_0), Hessian model W_0

for $k = 0, 1, 2, \ldots$ *until convergence* **do**

  Linearize the dynamics defects, boundary constraints, and path constraints at z_k

  Build the structured QP subproblem from Definition 5.8

  Solve the QP using sparse linear algebra that respects the time ordering

  Globalize with a merit function or filter line search as in algorithm 7

  Update the primal variables, multipliers, and Hessian model

Output: Approximate KKT point of the transcribed NLP

Remark (SQP versus interior-point methods). Both SQP and interior-point methods are standard for trajectory NLPs. Interior-point methods solve a sequence of perturbed KKT systems along a central path, while SQP solves a sequence of explicit QP approximations. The two families often use similar sparse factorizations internally, but their globalization, multiplier updates, and treatment of inequalities differ.

5.8 Free-Time and Minimum-Time Transcriptions

Definition 5.9 (Free-final-time optimal control problem). A free-final-time problem augments the usual state and control variables with the terminal time T. In continuous time,

$$\begin{aligned} \min_{u(\cdot),T} \quad & \phi(x(T), T) + \int_0^T L(x(t), u(t), t)\, dt \\ \text{subject to} \quad & \dot{x}(t) = f(x(t), u(t), t), \\ & x(0) = x_0, \end{aligned}$$

with T itself unknown.

Remark (Minimum-time as a special case). Minimum-time control is obtained by choosing

$$L \equiv 1, \qquad \phi \equiv 0,$$

so that the objective is exactly

$$J = T.$$

The minimum-time example in Section 4.4 was analyzed indirectly through PMP. Direct transcription treats the same problem by making T or the time steps themselves decision variables.

Definition 5.10 (Time normalization for free-time transcription). Introduce normalized time

$$\tau = \frac{t}{T} \in [0, 1].$$

Then

$$\frac{dx}{d\tau} = Tf(x(\tau), u(\tau), T\tau),$$

and the cost becomes

$$J = \phi(x(1), T) + T \int_0^1 L(x(\tau), u(\tau), T\tau)\, d\tau.$$

After normalization, the mesh in τ is fixed while the scalar T becomes an optimization variable.

Remark (Variable step sizes). An equivalent discrete formulation introduces interval lengths

$$h_k = t_{k+1} - t_k$$

as decision variables. Then the discrete dynamics become

$$x_{k+1} = \Phi(x_k, u_k, h_k)$$

and the total horizon is

$$T = \sum_{k=0}^{N-1} h_k.$$

This formulation is especially natural in collocation methods when mesh adaptation is also desired.

Algorithm 13: Direct Transcription for a Free-Time Problem

Input: State/control boundary data, fixed normalized mesh, initial guesses for (x_k, u_k, T)

1 Introduce either the terminal time T or interval lengths $\{h_k\}$ as optimization variables

2 Rewrite the dynamics on a fixed normalized grid using

$$\frac{dx}{d\tau} = Tf(x, u, T\tau)$$

(or the equivalent discrete step map depending on h_k)

3 Scale every integral stage cost by the local time interval

4 Add positivity constraints

$$T > 0 \qquad \text{or} \qquad h_k > 0$$

so that time does not reverse

5 Transcribe the resulting free-time OCP as an NLP and solve with SQP or an interior-point method

Output: State, control, and time variables describing the optimized trajectory

Remark (A subtle but important scaling issue). When the horizon length is variable, the stage cost must scale with the corresponding interval length. Otherwise the discretized objective is not a consistent approximation of the continuous-time integral, and the optimizer may exploit the mismatch by stretching or shrinking time in unphysical ways.

5.9 Comparison of Major Trajectory-Optimization Methods

Method	Decision variables	Strengths	Weaknesses
Indirect shooting	Initial costate (and possibly final time)	Uses exact PMP structure; low dimension when it works	Sensitive, nonconvex, hard to initialize, awkward with constraints
Direct single shooting	Control parameters only	Simple to implement; few variables; reuses simulators	Poor conditioning; state constraints awkward; long-horizon sensitivity
Direct multiple shooting	Node states and controls	Better conditioning; path constraints natural; sparse NLP	More variables than single shooting; requires continuity constraints
Direct collocation	Node states and controls (plus possible midpoint data)	Very sparse; handles path constraints and stiff dynamics well	Larger NLP; transcription design affects accuracy and conditioning

Remark. In modern robotics practice, direct multiple shooting and direct collocation dominate general-purpose nonlinear trajectory optimization. Indirect methods remain valuable when an accurate analytical model is available and the structure is favorable, but they are usually less robust in the presence of path constraints, hybrid effects, or weak initial guesses.

Remark. Lecture 5 closes the loop from theory to computation:

- Section 4.3 and Section 4.4 described optimality through HJB and PMP,
- Section 5.2 through Section 5.6 described how those problems are actually solved numerically,
- later lectures will show how special structure, especially linear-quadratic structure, leads to much more specialized and efficient solvers.

In particular, iLQR can be viewed as a specialized trajectory-optimization method for problems whose local subproblems are solved by Riccati recursions rather than generic sparse NLP algebra.

Lecture 6: Linear Quadratic Regulator

Intuition. LQR is the canonical optimal-control problem because it is rich enough to produce a nontrivial feedback law and structured enough to solve exactly. The value function is quadratic, the optimal policy is linear, and both dynamic programming and Pontryagin's principle collapse to the same Riccati equation. This lecture develops that structure carefully and keeps the presentation strictly within the finite-horizon, infinite-horizon, and Riccati themes laid out in the plan.

6.1 Problem Statement and Structure

Definition 6.1 (Finite-horizon discrete-time LQR)**.** The finite-horizon discrete-time linear quadratic regulator problem is

$$\begin{aligned} \min_{\{u_k\}_{k=0}^{N-1}} \quad & x_N^\top Q_f x_N + \sum_{k=0}^{N-1} \left(x_k^\top Q x_k + u_k^\top R u_k\right) \\ \text{subject to} \quad & x_{k+1} = Ax_k + Bu_k, \qquad k = 0, \dots, N-1, \\ & x_0 \text{ given.} \end{aligned}$$

Here

$$A \in \mathbb{R}^{n\times n}, \qquad B \in \mathbb{R}^{n\times m},$$

and the weight matrices satisfy

$$Q = Q^\top \succeq 0, \qquad R = R^\top \succ 0, \qquad Q_f = Q_f^\top \succeq 0.$$

Remark (Why the assumptions on Q, R, and Q_f matter). The conditions

$$Q \succeq 0, \qquad Q_f \succeq 0$$

ensure that state deviations are penalized but not rewarded. The condition

$$R \succ 0$$

ensures that the stage cost is strictly convex in the control input. This strict convexity is what makes the Bellman minimization produce a unique control law and guarantees that the matrix

$$R + B^\top P B$$

is invertible when $P \succeq 0$.

Remark (Role of Q versus R). The matrix Q prices state error, while R prices control effort.

- Increasing Q makes state deviations more expensive, so the controller becomes more aggressive.
- Increasing R makes actuation more expensive, so the controller becomes more conservative.

LQR therefore formalizes a precise tradeoff between regulation performance and control energy.

Remark (What goes wrong if R is only positive semidefinite). If R is merely positive semidefinite, then some control directions may have zero cost. In such directions, the Bellman stage minimization need not be strictly convex, the minimizing control may fail to be unique, and the matrix

$$R + B^\top P B$$

may become singular. Algebraically, the Riccati recursion may cease to be well defined in the standard inverse form. Intuitively, the problem is no longer penalizing all actuation channels.

Definition 6.2 (Linear regulator structure). A feedback law is called a *linear regulator* if it has the form

$$u_k = -K_k x_k$$

for matrices $K_k \in \mathbb{R}^{m \times n}$. In the infinite-horizon time-invariant case, the gain becomes constant:

$$u_k = -K x_k.$$

6.2 Finite-Horizon LQR via Dynamic Programming

Theorem 6.1 (Quadratic value function and linear optimal control). For finite-horizon discrete-time LQR, the value function at stage k has the quadratic form

$$V_k(x) = x^\top P_k x,$$

where the matrices P_k satisfy the backward Riccati recursion

$$P_N = Q_f,$$

$$P_k = Q + A^\top P_{k+1} A - A^\top P_{k+1} B \left(R + B^\top P_{k+1} B\right)^{-1} B^\top P_{k+1} A, \qquad k = N-1, \ldots, 0.$$

The optimal control law is

$$u_k^\star = -K_k x_k,$$

with

$$K_k = \left(R + B^\top P_{k+1} B\right)^{-1} B^\top P_{k+1} A.$$

Proof. We prove the result by backward induction on k using Bellman's equation.

At the terminal stage,

$$V_N(x) = x^\top Q_f x,$$

so the claim holds with

$$P_N = Q_f.$$

Now assume that for some $k+1 \le N$,

$$V_{k+1}(x) = x^\top P_{k+1} x$$

with $P_{k+1} = P_{k+1}^\top \succeq 0$. Bellman's equation gives

$$\begin{aligned} V_k(x) &= \min_{u \in \mathbb{R}^m} \left\{ x^\top Q x + u^\top R u + V_{k+1}(Ax + Bu) \right\} \\ &= \min_{u \in \mathbb{R}^m} \left\{ x^\top Q x + u^\top R u + (Ax + Bu)^\top P_{k+1}(Ax + Bu) \right\}. \end{aligned}$$

Expanding the quadratic form in u yields

$$\begin{aligned} V_k(x) = \min_u \Big\{ & x^\top (Q + A^\top P_{k+1} A) x + 2u^\top B^\top P_{k+1} A x \\ & + u^\top (R + B^\top P_{k+1} B) u \Big\}. \end{aligned}$$

Since

$$R + B^\top P_{k+1} B \succ 0,$$

the minimization is strictly convex, and the unique minimizer is obtained by setting the gradient with respect to u equal to zero:

$$2(R + B^\top P_{k+1} B)u + 2B^\top P_{k+1} A x = 0.$$

Hence

$$u_k^\star = -\left(R + B^\top P_{k+1} B\right)^{-1} B^\top P_{k+1} A x = -K_k x.$$

Substituting this minimizing control back into the Bellman expression gives

$$V_k(x) = x^\top P_k x,$$

with

$$P_k = Q + A^\top P_{k+1} A - A^\top P_{k+1} B \left(R + B^\top P_{k+1} B\right)^{-1} B^\top P_{k+1} A.$$

This completes the induction. ■

Definition 6.3 (Riccati gain matrix)**.** The matrix

$$K_k = \left(R + B^\top P_{k+1} B\right)^{-1} B^\top P_{k+1} A$$

is called the *finite-horizon LQR gain* at stage k. The optimal input is the state feedback

$$u_k^\star = -K_k x_k.$$

Proposition 6.1 (Equivalent completed-square form)**.** The Riccati recursion can be rewritten as

$$x^\top P_k x = \min_u \left\{ x^\top Q x + u^\top R u + (Ax + Bu)^\top P_{k+1}(Ax + Bu) \right\},$$

and the minimizing expression admits the completed-square representation

$$\begin{aligned} & x^\top Q x + u^\top R u + (Ax + Bu)^\top P_{k+1}(Ax + Bu) \\ & \quad = x^\top P_k x + (u + K_k x)^\top (R + B^\top P_{k+1} B)(u + K_k x). \end{aligned}$$

Proof. Expand the right-hand side:

$$x^\top P_k x + u^\top (R + B^\top P_{k+1} B)u + 2u^\top (R + B^\top P_{k+1} B)K_k x$$
$$+ x^\top K_k^\top (R + B^\top P_{k+1} B)K_k x.$$

By the definition of K_k,

$$(R + B^\top P_{k+1} B)K_k = B^\top P_{k+1} A.$$

Also,

$$K_k^\top (R + B^\top P_{k+1} B)K_k = A^\top P_{k+1} B(R + B^\top P_{k+1} B)^{-1} B^\top P_{k+1} A.$$

Combining these identities with the definition of P_k reproduces the expanded Bellman expression. ■

Algorithm 14: Finite-Horizon Discrete-Time LQR

Input: A, B, $Q \succeq 0$, $R \succ 0$, terminal weight $Q_f \succeq 0$, horizon N, initial state x_0

Set $P_N \leftarrow Q_f$

for $k = N-1, N-2, \ldots, 0$ **do**

  Compute

$$K_k \leftarrow \left(R + B^\top P_{k+1} B\right)^{-1} B^\top P_{k+1} A$$

  and

$$P_k \leftarrow Q + A^\top P_{k+1} A - A^\top P_{k+1} B K_k$$

Set x_0 to the given initial state

for $k = 0, 1, \ldots, N-1$ **do**

  Apply the optimal input $u_k \leftarrow -K_k x_k$

  Propagate the state $x_{k+1} \leftarrow A x_k + B u_k$

Output: Gains $\{K_k\}_{k=0}^{N-1}$, Riccati matrices $\{P_k\}_{k=0}^{N}$, optimal state-control trajectory

Example (Scalar finite-horizon LQR solved by hand)**.** Consider the scalar system

$$x_{k+1} = x_k + u_k,$$

with horizon $N = 2$ and cost

$$J = x_2^2 + \sum_{k=0}^{1} (x_k^2 + u_k^2).$$

Here

$$A = 1, \qquad B = 1, \qquad Q = 1, \qquad R = 1, \qquad Q_f = 1.$$

The Riccati recursion gives

$$P_2 = 1,$$
$$P_1 = 1 + 1 - \frac{1 \cdot 1 \cdot 1}{1+1} = \frac{3}{2},$$
$$P_0 = 1 + \frac{3}{2} - \frac{(\frac{3}{2})^2}{1 + \frac{3}{2}} = \frac{8}{5}.$$

The gains are

$$K_1 = \frac{1}{1+1} = \frac{1}{2}, \qquad K_0 = \frac{\frac{3}{2}}{1 + \frac{3}{2}} = \frac{3}{5}.$$

If

$$x_0 = 1,$$

then

$$u_0^\star = -\frac{3}{5}, \qquad x_1 = \frac{2}{5},$$

and

$$u_1^\star = -\frac{1}{2} \cdot \frac{2}{5} = -\frac{1}{5}, \qquad x_2 = \frac{1}{5}.$$

This example shows concretely how the backward Riccati sweep produces the forward optimal trajectory.

Remark (Finite versus infinite horizon). In finite-horizon LQR, the gain K_k depends on time because the remaining horizon length changes with k. In infinite-horizon LQR, the problem is time invariant, the value function becomes stationary, and the gain converges to a constant matrix under the standard stabilizability and detectability assumptions.

6.3 Finite-Horizon LQR as a Structured Quadratic Program

Remark. The dynamic-programming derivation from Section 6.2 is conceptually fundamental, but it is not the only exact derivation. One can also write the entire finite-horizon problem as a single equality-constrained quadratic program and then solve its KKT system. The Riccati recursion then appears as a sparse block elimination of that KKT system.

Definition 6.4 (Stacked QP form of finite-horizon LQR). Let

$$z = \begin{bmatrix} u_0 \\ x_1 \\ u_1 \\ x_2 \\ \vdots \\ u_{N-1} \\ x_N \end{bmatrix}.$$

With x_0 fixed, the finite-horizon LQR problem can be written as

$$\begin{aligned} \min_z \quad & \frac{1}{2} z^\top H z + \ell^\top z + \text{const} \\ \text{subject to} \quad & Cz = d, \end{aligned}$$

where H is block diagonal with copies of R and Q (and Q_f at the final state block), while C is block bidiagonal and represents the dynamics

$$x_{k+1} - Ax_k - Bu_k = 0.$$

Remark (Structure of the matrices). The important point is not the exact bookkeeping of H, C, and d, but their sparsity:

- H is block diagonal because the stage cost couples only variables from the same time index,
- C is block banded because each dynamics constraint couples only x_k, u_k, and x_{k+1}.

This structure is precisely what later allows Riccati-style elimination and, more generally, sparse optimal-control solvers.

Proposition 6.2 (KKT system for stacked LQR). The first-order optimality conditions of the stacked QP from Definition 6.4 are the linear saddle-point system

$$\begin{bmatrix} H & C^\top \\ C & 0 \end{bmatrix} \begin{bmatrix} z^\star \\ \lambda^\star \end{bmatrix} = \begin{bmatrix} -\ell \\ d \end{bmatrix}.$$

The multiplier block $\lambda^\star$ consists of one vector per dynamics constraint and is the discrete-time costate sequence.

Proof. This is the equality-constrained QP KKT system from Proposition 3.2 applied to the stacked LQR matrices. Since the constraints are exactly the stagewise dynamics equations, each multiplier corresponds to one dynamics equation and therefore plays the role of a costate. ■

Theorem 6.2 (Riccati recursion from block elimination of the KKT system)**.** Eliminating the variables of the KKT system in reverse time recovers the same feedback gains and Riccati matrices as in Theorem 6.1. In particular, if the multiplier attached to the constraint

$$x_{k+1} - Ax_k - Bu_k = 0$$

is denoted by λ_{k+1}, then

$$\lambda_k = P_k x_k,$$

and backward elimination yields

$$K_k = \left(R + B^\top P_{k+1} B\right)^{-1} B^\top P_{k+1} A$$

and

$$P_k = Q + A^\top P_{k+1} A - A^\top P_{k+1} B \left(R + B^\top P_{k+1} B\right)^{-1} B^\top P_{k+1} A.$$

Proof. Write the Lagrangian in stagewise form:

$$\mathcal{L} = x_N^\top Q_f x_N + \sum_{k=0}^{N-1} \Big(x_k^\top Q x_k + u_k^\top R u_k + \lambda_{k+1}^\top (Ax_k + Bu_k - x_{k+1}) \Big).$$

Stationarity with respect to the terminal state gives

$$2Q_f x_N - \lambda_N = 0.$$

Up to the purely conventional factor of 2 coming from whether one writes the cost with or without $\frac{1}{2}$, this says that the final multiplier is linear in the final state:

$$\lambda_N = P_N x_N, \qquad P_N = Q_f.$$

Now fix a stage k. Stationarity with respect to u_k yields

$$2Ru_k + B^\top \lambda_{k+1} = 0.$$

If we already know

$$\lambda_{k+1} = P_{k+1} x_{k+1},$$

then substituting

$$x_{k+1} = Ax_k + Bu_k$$

into the stationarity condition gives

$$\left(R + B^\top P_{k+1} B\right) u_k = -B^\top P_{k+1} A x_k,$$

again up to the same harmless factor convention. Hence

$$u_k = -K_k x_k$$

with the usual Riccati gain

$$K_k = \left(R + B^\top P_{k+1} B\right)^{-1} B^\top P_{k+1} A.$$

Next, stationarity with respect to x_k gives

$$2Qx_k + A^\top \lambda_{k+1} - \lambda_k = 0.$$

Substitute

$$\lambda_{k+1} = P_{k+1}(Ax_k + Bu_k)$$

and then replace u_k by the feedback law just derived. The result is

$$\lambda_k = \Big(Q + A^\top P_{k+1} A - A^\top P_{k+1} B K_k\Big) x_k,$$

again with the same normalization convention. Identifying the coefficient of x_k gives

$$P_k = Q + A^\top P_{k+1} A - A^\top P_{k+1} B K_k,$$

which is exactly the Riccati recursion. ■

Remark (Why this derivation matters computationally). The dynamic-programming derivation explains optimality through value functions; the KKT derivation explains why sparse linear algebra is so effective. Riccati recursion is not magic. It is the specialized sparse elimination one gets when the QP has the chain structure induced by time.

6.4 Infinite-Horizon LQR and the Algebraic Riccati Equation

Definition 6.5 (Infinite-horizon discrete-time LQR). The infinite-horizon discrete-time LQR problem is

$$\begin{aligned} \min_{\{u_k\}_{k=0}^{\infty}} \quad & \sum_{k=0}^{\infty} \left(x_k^\top Q x_k + u_k^\top R u_k\right) \\ \text{subject to} \quad & x_{k+1} = Ax_k + Bu_k, \\ & x_0 \text{ given.} \end{aligned}$$

Remark (Stabilizability and detectability). The infinite-horizon problem is meaningful only if the system can be stabilized and the cost can detect unstable directions. The standard hypotheses are:

- (A, B) is stabilizable,
- $(Q^{1/2}, A)$ is detectable.

The first prevents uncontrollable unstable modes from remaining in the closed loop. The second prevents unstable directions from carrying zero cost.

Theorem 6.3 (Discrete-time algebraic Riccati equation and optimal gain). Assume (A, B) is stabilizable, $(Q^{1/2}, A)$ is detectable, and

$$Q \succeq 0, \qquad R \succ 0.$$

Then there exists a unique symmetric matrix $P \succeq 0$ such that

$$P = Q + A^\top P A - A^\top P B \left(R + B^\top P B\right)^{-1} B^\top P A,$$

and the feedback law

$$u_k^\star = -K x_k, \qquad K = \left(R + B^\top P B\right)^{-1} B^\top P A$$

is optimal. The matrix P is called the *stabilizing solution* of the discrete-time algebraic Riccati equation (DARE).

Proof idea. The finite-horizon Riccati matrices form a monotone sequence under the standard assumptions and converge to a limit P. Passing to the limit in the Riccati recursion shows that P solves the fixed-point equation, namely the DARE. The stabilizability and detectability hypotheses rule out nonstabilizing or nonminimal fixed points and single out the unique stabilizing solution. Since the finite-horizon gains converge to the limiting gain

$$K = \left(R + B^\top PB\right)^{-1} B^\top PA,$$

the infinite-horizon optimal controller is stationary. ■

Theorem 6.4 (Closed-loop stability of infinite-horizon LQR)**.** Let $P \succeq 0$ be the stabilizing solution of the DARE, and define

$$K = \left(R + B^\top PB\right)^{-1} B^\top PA, \qquad A_c = A - BK.$$

Under the assumptions that (A, B) is stabilizable and $(Q^{1/2}, A)$ is detectable, the closed-loop matrix A_c is Schur stable; equivalently, all eigenvalues of $A - BK$ lie strictly inside the unit disk.

Proof. Starting from the DARE and the definition of K, one verifies the identity

$$P = Q + K^\top RK + A_c^\top PA_c.$$

Let

$$V(x) = x^\top Px.$$

Then along the closed-loop dynamics $x_{k+1} = A_c x_k$,

$$\begin{aligned} V(x_{k+1}) - V(x_k) &= x_k^\top (A_c^\top PA_c - P)x_k \\ &= -x_k^\top (Q + K^\top RK)x_k \\ &\le 0. \end{aligned}$$

So V is a Lyapunov function candidate.

Now suppose that A_c had an eigenvalue λ with $|\lambda| \ge 1$, with corresponding eigenvector $x \neq 0$. Substituting the closed-loop trajectory

$$x_k = \lambda^k x$$

into the decrease identity yields

$$x^\top (Q + K^\top RK)x = 0.$$

Since both summands are positive semidefinite and $R \succ 0$, we obtain

$$Q^{1/2}x = 0, \qquad Kx = 0.$$

The second identity implies

$$A_c x = Ax = \lambda x.$$

So x is an eigenvector of A with $|\lambda| \ge 1$ that is unobservable through $Q^{1/2}$. This contradicts detectability of $(Q^{1/2}, A)$. Hence no such eigenvalue exists, and A_c is Schur stable. ■

Example (Setting up a 2×2 algebraic Riccati equation)**.** Consider

$$A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \qquad B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \qquad Q = I, \qquad R = 1.$$

Write

$$P = \begin{bmatrix} p_{11} & p_{12} \\ p_{12} & p_{22} \end{bmatrix}.$$

Then

$$B^\top PB = p_{22}, \qquad B^\top PA = \begin{bmatrix} p_{12} & p_{12} + p_{22} \end{bmatrix}.$$

Also,

$$A^\top PA = \begin{bmatrix} p_{11} & p_{11}+p_{12} \\ p_{11}+p_{12} & p_{11}+2p_{12}+p_{22} \end{bmatrix}.$$

Therefore the DARE becomes

$$P = I + \begin{bmatrix} p_{11} & p_{11}+p_{12} \\ p_{11}+p_{12} & p_{11}+2p_{12}+p_{22} \end{bmatrix} - \frac{1}{1+p_{22}} \begin{bmatrix} p_{12} \\ p_{12}+p_{22} \end{bmatrix} \begin{bmatrix} p_{12} & p_{12}+p_{22} \end{bmatrix}.$$

Equating the entries yields a coupled nonlinear algebraic system for (p_{11}, p_{12}, p_{22}).

6.5 The Riccati Equation – Properties and Structure

Definition 6.6 (DARE versus CARE)**.** The *discrete-time algebraic Riccati equation* (DARE) is

$$P = Q + A^\top PA - A^\top PB\left(R + B^\top PB\right)^{-1} B^\top PA.$$

The *continuous-time algebraic Riccati equation* (CARE) is

$$A^\top P + PA - PBR^{-1}B^\top P + Q = 0.$$

Both equations characterize stationary quadratic value functions for linear-quadratic optimal control, but one comes from discrete-time dynamics and the other from continuous-time dynamics.

Definition 6.7 (Riccati operator)**.** For $P \succeq 0$, define the discrete Riccati operator

$$\mathcal{T}(P) = Q + A^\top PA - A^\top PB\left(R + B^\top PB\right)^{-1} B^\top PA.$$

Theorem 6.5 (Monotonicity and convergence of the Riccati iteration)**.** Assume

$$Q \succeq 0, \qquad R \succ 0.$$

Then the Riccati operator is order preserving:

$$P \succeq S \succeq 0 \quad \Longrightarrow \quad \mathcal{T}(P) \succeq \mathcal{T}(S).$$

If one defines the sequence

$$\Pi_0 = 0, \qquad \Pi_{j+1} = \mathcal{T}(\Pi_j),$$

then $\{\Pi_j\}$ is monotone nondecreasing. If, in addition, the sequence is bounded above, then it converges to a fixed point of the DARE.

Proof. For any $P \succeq 0$,

$$x^\top \mathcal{T}(P)x = \min_u \left\{x^\top Qx + u^\top Ru + (Ax+Bu)^\top P(Ax+Bu)\right\}.$$

If $P \succeq S$, then for every (x, u),

$$(Ax+Bu)^\top P(Ax+Bu) \geq (Ax+Bu)^\top S(Ax+Bu).$$

Adding the common stage cost and minimizing over u gives

$$x^\top \mathcal{T}(P)x \geq x^\top \mathcal{T}(S)x$$

for all x, hence

$$\mathcal{T}(P) \succeq \mathcal{T}(S).$$

Next, since

$$\Pi_1 = \mathcal{T}(0) = Q \succeq 0 = \Pi_0,$$

order preservation implies inductively that

$$\Pi_{j+1} = \mathcal{T}(\Pi_j) \succeq \mathcal{T}(\Pi_{j-1}) = \Pi_j.$$

So the sequence is monotone nondecreasing.

If the sequence is bounded above by some $\bar{P}$, then each matrix entry converges monotonically, and the limit

$$\Pi_\infty = \lim_{j\to\infty} \Pi_j$$

exists in the cone of symmetric positive semidefinite matrices. Continuity of $\mathcal{T}$ on that cone implies

$$\Pi_\infty = \mathcal{T}(\Pi_\infty),$$

so Π_∞ is a fixed point of the DARE. ■

Proposition 6.3 (Symmetry of the Riccati matrices)**.** If the terminal weight Q_f is symmetric, then every finite-horizon Riccati matrix P_k is symmetric.

Proof. We argue by backward induction. By assumption,

$$P_N = Q_f = Q_f^\top.$$

Assume $P_{k+1} = P_{k+1}^\top$. Then

$$R + B^\top P_{k+1} B$$

is symmetric, hence so is its inverse. Therefore

$$A^\top P_{k+1} B \left(R + B^\top P_{k+1} B\right)^{-1} B^\top P_{k+1} A$$

is symmetric because it has the form $M^\top S^{-1} M$ with $S = S^\top$. Since both

$$Q \qquad \text{and} \qquad A^\top P_{k+1} A$$

are symmetric, the recursion shows that P_k is symmetric. ■

Proposition 6.4 (Invertibility of $R + B^\top PB$)**.** If

$$R \succ 0 \qquad \text{and} \qquad P \succeq 0,$$

then

$$R + B^\top PB \succ 0,$$

hence it is invertible.

Proof. Let $z \neq 0$. Then

$$\begin{aligned} z^\top (R + B^\top PB) z &= z^\top R z + z^\top B^\top PBz \\ &= z^\top R z + (Bz)^\top P(Bz). \end{aligned}$$

The first term is strictly positive because $R \succ 0$, and the second is nonnegative because $P \succeq 0$. Therefore

$$z^\top (R + B^\top PB) z > 0$$

for all $z \neq 0$. ■

Theorem 6.6 (Hamiltonian matrix connection for continuous-time LQR)**.** Consider the continuous-time Hamiltonian matrix

$$\mathcal{H} = \begin{bmatrix} A & -BR^{-1}B^\top \\ -Q & -A^\top \end{bmatrix}.$$

If P solves the CARE

$$A^\top P + PA - PBR^{-1}B^\top P + Q = 0,$$

then the graph subspace

$$\mathcal{G}(P) = \left\{ \begin{bmatrix} x \\ Px \end{bmatrix} : x \in \mathbb{R}^n \right\}$$

is invariant under $\mathcal{H}$, and

$$\mathcal{H} \begin{bmatrix} I \\ P \end{bmatrix} = \begin{bmatrix} I \\ P \end{bmatrix} (A - BR^{-1}B^\top P).$$

Proof. Compute

$$\mathcal{H} \begin{bmatrix} I \\ P \end{bmatrix} = \begin{bmatrix} A - BR^{-1}B^\top P \\ -Q - A^\top P \end{bmatrix}.$$

On the other hand,

$$\begin{bmatrix} I \\ P \end{bmatrix} (A - BR^{-1}B^\top P) = \begin{bmatrix} A - BR^{-1}B^\top P \\ PA - PBR^{-1}B^\top P \end{bmatrix}.$$

These are equal precisely when

$$-Q - A^\top P = PA - PBR^{-1}B^\top P,$$

which is equivalent to the CARE. ■

Remark (Schur-complement interpretation). The discrete Riccati equation is equivalent to the statement that the Schur complement of the block matrix

$$\begin{bmatrix} R + B^\top PB & B^\top PA \\ A^\top PB & Q + A^\top PA - P \end{bmatrix}$$

with respect to the upper-left block vanishes. Indeed, because $R + B^\top PB$ is invertible, the Schur complement is

$$Q + A^\top PA - P - A^\top PB(R + B^\top PB)^{-1}B^\top PA,$$

which is exactly zero when P satisfies the DARE.

Example (Scalar DARE). For the scalar system

$$x_{k+1} = ax_k + bu_k$$

with cost weights

$$q \geq 0, \qquad r > 0,$$

the DARE becomes

$$P = q + a^2P - \frac{a^2b^2P^2}{r + b^2P}.$$

Multiplying through by $r + b^2P$ gives the quadratic equation

$$b^2P^2 + \big(r(1 - a^2) - qb^2\big)P - qr = 0.$$

The relevant LQR solution is the nonnegative root that yields a stabilizing gain

$$K = \frac{baP}{r + b^2P}.$$

Remark (Cheap control and expensive control limits). As

$$R \to 0^+,$$

control becomes cheap, so the optimal gain typically grows and the controller acts more aggressively. In controllable directions, the closed loop tends to regulate the state rapidly, and the value matrix P decreases because state error can be removed at lower relative cost.

As

$$R \to \infty,$$

control becomes expensive, so the optimal gain tends to zero. The controller then approaches open-loop behavior. If the open-loop matrix A is already stable, the limiting value matrix approaches the uncontrolled cost. If A has unstable modes, the infinite-horizon cost may diverge because the controller is no longer willing to spend enough effort to stabilize the system.

6.6 Continuous-Time LQR

Definition 6.8 (Continuous-time LQR). The finite-horizon continuous-time LQR problem is

$$\begin{aligned} \min_{u(\cdot)} \quad & x(t_f)^\top S_f x(t_f) + \int_{t_0}^{t_f} \left(x(t)^\top Q x(t) + u(t)^\top R u(t) \right) dt \\ \text{subject to} \quad & \dot{x}(t) = Ax(t) + Bu(t), \\ & x(t_0) = x_0, \end{aligned}$$

where

$$Q = Q^\top \succeq 0, \qquad R = R^\top \succ 0, \qquad S_f = S_f^\top \succeq 0.$$

Theorem 6.7 (Differential Riccati equation). For continuous-time finite-horizon LQR, the value function has the form

$$V(t, x) = x^\top P(t) x,$$

where $P(t)$ satisfies the backward differential Riccati equation

$$-\dot{P} = Q + A^\top P + PA - PBR^{-1}B^\top P, \qquad P(t_f) = S_f.$$

The optimal control law is

$$u^\star(t) = -R^{-1}B^\top P(t)x(t).$$

Proof. The proof is the continuous-time analogue of the discrete Bellman derivation. One substitutes the ansatz

$$V(t, x) = x^\top P(t) x$$

into the Hamilton-Jacobi-Bellman equation

$$-\partial_t V(t, x) = \min_u \left\{ x^\top Q x + u^\top R u + \nabla_x V(t, x)^\top (Ax + Bu) \right\},$$

with terminal condition

$$V(t_f, x) = x^\top S_f x.$$

Since

$$\nabla_x V(t, x) = 2P(t)x,$$

the minimization problem becomes

$$\min_u \left\{ x^\top Q x + u^\top R u + 2x^\top PAx + 2u^\top B^\top P x \right\}.$$

The unique minimizer satisfies

$$2Ru + 2B^\top P x = 0,$$

hence

$$u^\star = -R^{-1}B^\top P x.$$

Substituting this control into the HJB equation and identifying coefficients of the quadratic form in x yields the differential Riccati equation. ■

Definition 6.9 (Continuous-time algebraic Riccati equation). In the infinite-horizon continuous-time case, stationarity of the value matrix leads to the CARE

$$A^\top P + PA - PBR^{-1}B^\top P + Q = 0.$$

The optimal constant gain is

$$K = R^{-1}B^\top P, \qquad u^\star(t) = -Kx(t).$$

Example (Scalar continuous-time LQR). Consider

$$\dot{x} = ax + bu,$$

with infinite-horizon cost

$$J = \int_0^\infty \left(qx(t)^2 + ru(t)^2\right)\, dt, \qquad q \geq 0, \quad r > 0.$$

The CARE is

$$2aP - \frac{b^2}{r}P^2 + q = 0,$$

or equivalently

$$b^2P^2 - 2arP - qr = 0.$$

The two roots are

$$P = \frac{r}{b^2}\left(a \pm \sqrt{a^2 + \frac{b^2 q}{r}}\right).$$

The stabilizing solution is the root with the plus sign:

$$P = \frac{r}{b^2}\left(a + \sqrt{a^2 + \frac{b^2 q}{r}}\right),$$

because then the closed-loop coefficient becomes

$$a - bK = a - \frac{b^2}{r}P = -\sqrt{a^2 + \frac{b^2 q}{r}} < 0.$$

Theorem 6.8 (Classical robustness margins of continuous-time LQR). For the continuous-time full-state-feedback LQR law

$$u = -Kx,$$

the loop enjoys guaranteed robustness margins: gain margin at least 6 dB and phase margin at least 60°.

Proof sketch. The key fact is that the return-difference matrix of the LQR loop satisfies a strict positive-real identity implied by the CARE. From this one shows that the Nyquist plot of the loop transfer function stays a controlled distance away from the critical point -1. Geometrically, the positive-real property places the Nyquist image in a sector that excludes destabilizing perturbations smaller than a factor of 2 in gain and $\pm 60°$ in phase. The full proof is classical but substantially more involved than the Riccati derivations above. ■

6.7 LQR and Pontryagin's Minimum Principle

Remark. LQR is one of the cleanest demonstrations that dynamic programming and Pontryagin's principle are complementary rather than competing theories. Dynamic programming produces the Riccati equation from the value function. PMP produces the same Riccati equation from the adjoint dynamics.

Theorem 6.9 (PMP derivation of the continuous-time Riccati equation)**.** Consider the finite-horizon continuous-time LQR problem

$$\begin{aligned} \min_{u(\cdot)} \quad & x(t_f)^\top S_f x(t_f) + \int_{t_0}^{t_f} \left(x^\top Q x + u^\top R u\right)\, dt \\ \text{subject to} \quad & \dot{x} = Ax + Bu. \end{aligned}$$

Pontryagin's minimum principle yields the same control law and Riccati equation as dynamic programming:

$$u^\star(t) = -R^{-1}B^\top P(t)x(t),$$

where $P(t)$ satisfies

$$-\dot{P} = Q + A^\top P + PA - PBR^{-1}B^\top P, \qquad P(t_f) = S_f.$$

Proof. The Hamiltonian is

$$H(x, u, p) = x^\top Q x + u^\top R u + p^\top (Ax + Bu).$$

PMP gives

$$\begin{aligned} \dot{x} &= Ax + Bu, \\ -\dot{p} &= 2Qx + A^\top p, \end{aligned}$$

and the stationarity condition

$$0 = \partial_u H = 2Ru + B^\top p.$$

Hence

$$u^\star = -\frac{1}{2}R^{-1}B^\top p.$$

Now make the ansatz

$$p(t) = 2P(t)x(t),$$

where $P(t)$ is a matrix to be determined. Then

$$u^\star(t) = -R^{-1}B^\top P(t)x(t).$$

Differentiate the costate ansatz:

$$\dot{p} = 2\dot{P}\,x + 2P\dot{x}.$$

Using the state equation under the optimal control,

$$\dot{x} = \left(A - BR^{-1}B^\top P\right)x,$$

we obtain

$$\dot{p} = 2\dot{P}\,x + 2P\left(A - BR^{-1}B^\top P\right)x.$$

On the other hand, the costate equation gives

$$\dot{p} = -2Qx - 2A^\top P x.$$

Since this must hold for every trajectory state x, we conclude that

$$\dot{P} + PA - PBR^{-1}B^\top P = -Q - A^\top P,$$

or equivalently

$$-\dot{P} = Q + A^\top P + PA - PBR^{-1}B^\top P.$$

The transversality condition for the terminal cost is

$$p(t_f) = 2S_f x(t_f),$$

which implies

$$P(t_f) = S_f.$$

■

Remark (Two roads to the same destination). In LQR,

- HJB starts from a quadratic value-function ansatz and produces the Riccati equation,
- PMP starts from a linear costate ansatz and produces the same Riccati equation.

This coincidence is not accidental. It reflects the deep relation

$$p(t) = 2\nabla_x V(t, x(t))$$

when the value function is smooth.

Lecture 7: Estimation – The Kalman Filter

Intuition. The Kalman filter is the estimation counterpart of LQR. LQR chooses a control law that minimizes a quadratic cost for a linear system; the Kalman filter chooses an estimator that minimizes mean-square estimation error for a linear stochastic system. The same Riccati structure reappears, but with the roles of dynamics, actuation, and sensing transposed. This lecture follows the planned path: algorithm, gain derivation, covariance recursion, steady-state behavior, duality, innovations, EKF, and a full scalar worked example.

7.1 Problem Setup

Definition 7.1 (Linear stochastic state-space model). The discrete-time Kalman-filter setting is the linear stochastic system

$$\begin{aligned} x_{k+1} &= Ax_k + Bu_k + w_k, \\ y_k &= Cx_k + v_k, \end{aligned}$$

where

$$x_k \in \mathbb{R}^n, \qquad u_k \in \mathbb{R}^m, \qquad y_k \in \mathbb{R}^p.$$

The random variables satisfy

$$\mathbb{E}[w_k] = 0, \qquad \mathbb{E}[v_k] = 0,$$

with covariances

$$\mathbb{E}[w_k w_k^\top] = Q_w \succeq 0, \qquad \mathbb{E}[v_k v_k^\top] = R_v \succ 0.$$

Notation (Noise assumptions). The standard Kalman assumptions are:

- the initial state error, process noise, and measurement noise are mutually independent,
- the sequences $\{w_k\}$ and $\{v_k\}$ are white,
- often, though not always, they are assumed Gaussian.

Gaussianity upgrades the Kalman filter from a best linear estimator to the exact minimum-variance conditional-mean estimator.

Definition 7.2 (State estimation problem). Given the measurement history

$$\mathcal{Y}_k = \sigma(y_0, \dots, y_k),$$

the state-estimation problem is to construct estimates of x_k from available data. Two standard estimates are:

- the *prediction*
$$\hat{x}_{k|k-1} = \mathbb{E}[x_k \mid \mathcal{Y}_{k-1}],$$
- the *filtered estimate*
$$\hat{x}_{k|k} = \mathbb{E}[x_k \mid \mathcal{Y}_k].$$

Definition 7.3 (Prediction and filtering error covariances). The corresponding error covariances are

$$P_{k|k-1} = \mathbb{E}\left[(x_k - \hat{x}_{k|k-1})(x_k - \hat{x}_{k|k-1})^\top\right],$$
$$P_{k|k} = \mathbb{E}\left[(x_k - \hat{x}_{k|k})(x_k - \hat{x}_{k|k})^\top\right].$$

Theorem 7.1 (Optimality criterion). Let $\mathcal{G}$ be any sigma-algebra and let x be square integrable. Then the conditional expectation

$$\mathbb{E}[x \mid \mathcal{G}]$$

is the unique minimizer of

$$\mathbb{E}\big[\|x - \xi\|^2\big]$$

over all square-integrable $\mathcal{G}$-measurable random vectors ξ. Consequently, under the linear-Gaussian state-space model, the Kalman filter is the minimum-mean-square-error estimator.

Proof. Fix any $\mathcal{G}$-measurable ξ and write

$$x - \xi = \big(x - \mathbb{E}[x \mid \mathcal{G}]\big) + \big(\mathbb{E}[x \mid \mathcal{G}] - \xi\big).$$

Squaring and taking expectations gives

$$\begin{aligned}\mathbb{E}\big[\|x - \xi\|^2\big] &= \mathbb{E}\big[\|x - \mathbb{E}[x \mid \mathcal{G}]\|^2\big] + \mathbb{E}\big[\|\mathbb{E}[x \mid \mathcal{G}] - \xi\|^2\big] \\ &\quad + 2\mathbb{E}\Big[\big(x - \mathbb{E}[x \mid \mathcal{G}]\big)^\top \big(\mathbb{E}[x \mid \mathcal{G}] - \xi\big)\Big].\end{aligned}$$

The cross term vanishes by the defining orthogonality property of conditional expectation: the error

$$x - \mathbb{E}[x \mid \mathcal{G}]$$

is orthogonal to every square-integrable $\mathcal{G}$-measurable random vector. Hence

$$\mathbb{E}\big[\|x - \xi\|^2\big] = \mathbb{E}\big[\|x - \mathbb{E}[x \mid \mathcal{G}]\|^2\big] + \mathbb{E}\big[\|\mathbb{E}[x \mid \mathcal{G}] - \xi\|^2\big].$$

The second term is nonnegative and vanishes only when

$$\xi = \mathbb{E}[x \mid \mathcal{G}]$$

almost surely. This proves optimality and uniqueness. ■

7.2 The Kalman Filter Algorithm

Definition 7.4 (Prediction step). Given $\hat{x}_{k-1|k-1}$ and $P_{k-1|k-1}$, the Kalman predictor is

$$\hat{x}_{k|k-1} = A\hat{x}_{k-1|k-1} + Bu_{k-1},$$

$$P_{k|k-1} = AP_{k-1|k-1}A^\top + Q_w.$$

Definition 7.5 (Correction step). After receiving measurement y_k, the Kalman correction is

$$L_k = P_{k|k-1}C^\top \left(CP_{k|k-1}C^\top + R_v\right)^{-1},$$

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + L_k \left(y_k - C\hat{x}_{k|k-1}\right),$$

$$P_{k|k} = (I - L_kC)P_{k|k-1}.$$

The matrix L_k is the *Kalman gain*.

Remark (Predict, then correct). The filter alternates between two distinct pieces of information:

- the model, which predicts where the state should move,
- the measurements, which reveal where the state appears to be.

The Kalman gain determines how much trust is assigned to each.

Algorithm 15: Discrete-Time Kalman Filter

Input: A, B, C, process covariance Q_w, measurement covariance R_v, initial estimate $\hat{x}_{0|-1}$, initial covariance $P_{0|-1}$

for $k = 0, 1, 2, \ldots$ **do**

Receive input u_{k-1} if applicable and measurement y_k

Compute the Kalman gain

$$L_k \leftarrow P_{k|k-1}C^\top \left(CP_{k|k-1}C^\top + R_v\right)^{-1}$$

Update the state estimate

$$\hat{x}_{k|k} \leftarrow \hat{x}_{k|k-1} + L_k(y_k - C\hat{x}_{k|k-1})$$

Update the covariance

$$P_{k|k} \leftarrow (I - L_kC)P_{k|k-1}$$

Predict one step ahead

$$\hat{x}_{k+1|k} \leftarrow A\hat{x}_{k|k} + Bu_k$$

and

$$P_{k+1|k} \leftarrow AP_{k|k}A^\top + Q_w$$

7.3 Derivation of the Kalman Gain

Theorem 7.2 (Kalman gain from covariance minimization). Fix a predicted estimate $\hat{x}_{k|k-1}$ with prediction covariance

$$P^- := P_{k|k-1}.$$

Among all affine updates of the form

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + L_k(y_k - C\hat{x}_{k|k-1}),$$

the choice that minimizes

$$\mathrm{tr}(P_{k|k})$$

is

$$L_k = P^- C^\top (CP^- C^\top + R_v)^{-1}.$$

Proof. Let

$$e^- = x_k - \hat{x}_{k|k-1}.$$

Since

$$y_k = Cx_k + v_k,$$

the updated estimation error is

$$\begin{aligned} e^+ &= x_k - \hat{x}_{k|k} \\ &= x_k - \hat{x}_{k|k-1} - L_k(y_k - C\hat{x}_{k|k-1}) \\ &= (I - L_k C)e^- - L_k v_k. \end{aligned}$$

Using independence of e^- and v_k, we obtain

$$P_{k|k} = (I - L_k C)P^-(I - L_k C)^\top + L_k R_v L_k^\top.$$

Expanding the trace gives

$$\operatorname{tr}(P_{k|k}) = \operatorname{tr}(P^-) - 2\operatorname{tr}(L_k C P^-) + \operatorname{tr}\left(L_k(CP^- C^\top + R_v)L_k^\top\right).$$

Set

$$S_k = CP^- C^\top + R_v.$$

Because $R_v \succ 0$, we have $S_k \succ 0$. Using standard matrix differentiation,

$$\nabla_{L_k} \operatorname{tr}(P_{k|k}) = -2P^- C^\top + 2L_k S_k.$$

Setting the gradient equal to zero yields

$$L_k S_k = P^- C^\top,$$

hence

$$L_k = P^- C^\top S_k^{-1} = P^- C^\top (CP^- C^\top + R_v)^{-1}.$$

Since the objective is strictly convex in L_k because $S_k \succ 0$, this minimizer is unique. ■

Remark (Joseph stabilized covariance form)**.** The covariance update can also be written as

$$P_{k|k} = (I - L_k C)P_{k|k-1}(I - L_k C)^\top + L_k R_v L_k^\top.$$

This is called the *Joseph form*. Algebraically it is equivalent to

$$P_{k|k} = (I - L_k C)P_{k|k-1}$$

when L_k is the optimal Kalman gain, but numerically the Joseph form preserves symmetry and positive semidefiniteness more robustly in finite precision.

7.4 Error Covariance and Riccati Recursion

Theorem 7.3 (Riccati recursion for the error covariance)**.** The Kalman-filter covariance recursion is

$$P_{k|k-1} = AP_{k-1|k-1}A^\top + Q_w,$$

$$P_{k|k} = P_{k|k-1} - P_{k|k-1}C^\top \left(CP_{k|k-1}C^\top + R_v\right)^{-1} CP_{k|k-1}.$$

Combining the two steps gives the one-step prior recursion

$$P_{k+1|k} = AP_{k|k-1}A^\top + Q_w - AP_{k|k-1}C^\top \left(CP_{k|k-1}C^\top + R_v\right)^{-1} CP_{k|k-1}A^\top.$$

Proof. The prediction covariance formula follows from

$$x_k - \hat{x}_{k|k-1} = A(x_{k-1} - \hat{x}_{k-1|k-1}) + w_{k-1},$$

combined with independence of the previous estimation error and the process noise.

The filtering covariance formula was derived in the proof of the Kalman gain theorem:

$$P_{k|k} = (I - L_k C)P_{k|k-1}(I - L_k C)^\top + L_k R_v L_k^\top.$$

Substituting the optimal gain

$$L_k = P_{k|k-1}C^\top \left(CP_{k|k-1}C^\top + R_v\right)^{-1}$$

and simplifying yields

$$P_{k|k} = P_{k|k-1} - P_{k|k-1}C^\top \left(CP_{k|k-1}C^\top + R_v\right)^{-1} CP_{k|k-1}.$$

Applying the prediction step once more gives the combined recursion. ■

Remark (Riccati structure)**.** The covariance recursion is a Riccati equation in disguise. Structurally it mirrors the LQR Riccati equation, but with sensing matrices and noise covariances replacing actuation matrices and control penalties.

Proposition 7.1 (Limiting behavior of the Kalman gain with respect to noise levels)**.** Let

$$L_k = P_{k|k-1}C^\top \left(CP_{k|k-1}C^\top + R_v\right)^{-1}.$$

Then:

- as the measurement noise covariance R_v decreases, the gain increases and the filter trusts the measurements more;
- as $R_v \to \infty$, one has
$$L_k \to 0,$$
so the filter ignores the measurements;
- increasing the process noise covariance Q_w increases the predicted covariance, which typically increases the gain and makes the filter trust the measurements more;
- decreasing Q_w has the opposite effect and makes the filter trust the model more.

Proof. The formula for L_k shows immediately that larger R_v enlarges the matrix being inverted in the denominator and therefore suppresses the gain. In the limit

$$R_v \to \infty,$$

the inverse tends to zero, so

$$L_k \to 0.$$

On the other hand, the predicted covariance

$$P_{k|k-1} = AP_{k-1|k-1}A^\top + Q_w$$

increases with Q_w. Since $P_{k|k-1}$ appears multiplicatively in the numerator and also enlarges the denominator through

$$CP_{k|k-1}C^\top,$$

the gain increases in the Loewner-order sense along observed directions. Intuitively, larger process noise means the model is less trustworthy, so the filter leans more heavily on measurements. ■

7.5 Steady-State Kalman Filter

Theorem 7.4 (Steady-state convergence conditions). Assume

$$R_v \succ 0, \qquad Q_w \succeq 0,$$

and suppose that

- (A, C) is detectable,
- $(A, Q_w^{1/2})$ is stabilizable.

Then the prior covariance recursion converges to a steady-state value $\bar{P} \succeq 0$ satisfying the estimator DARE

$$\bar{P} = A\bar{P}A^\top + Q_w - A\bar{P}C^\top \left(C\bar{P}C^\top + R_v\right)^{-1} C\bar{P}A^\top.$$

Proof sketch. The Kalman covariance recursion is monotone on the cone of positive semidefinite matrices and remains bounded under the detectability and stabilizability assumptions. Bounded monotone sequences converge, and continuity of the Riccati map shows that the limit must solve the fixed-point equation. The same hypotheses also guarantee that the associated steady-state observer is stable in the estimation-error dynamics. ■

Definition 7.6 (Steady-state Kalman filter). If

$$P_{k|k-1} \to \bar{P},$$

then the corresponding steady-state gain is

$$\bar{L} = \bar{P}C^\top(C\bar{P}C^\top + R_v)^{-1}.$$

The steady-state filter is

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + \bar{L}(y_k - C\hat{x}_{k|k-1}),$$
$$\hat{x}_{k+1|k} = A\hat{x}_{k|k} + Bu_k.$$

Remark (DARE in disguise). The steady-state covariance equation is exactly a DARE, but with the LQR data transposed:

$$A \leftrightarrow A^\top, \qquad B \leftrightarrow C^\top, \qquad Q \leftrightarrow Q_w, \qquad R \leftrightarrow R_v.$$

Remark (Observability and what fails without it). Observability, or more precisely detectability, ensures that unstable modes leave a signature in the measurements. If (A, C) is not detectable, there can exist unstable state components that never affect the outputs. No estimator can recover those components from data, so the estimation error along those directions cannot be driven to zero, and the covariance recursion may fail to converge to a stabilizing steady state.

7.6 Duality Between LQR and Kalman Filter

Theorem 7.5 (LQR-Kalman duality). The Riccati equation of the Kalman filter is the transpose-dual of the Riccati equation of LQR. Under the mapping

$$A_{\mathrm{LQR}} \leftrightarrow A_{\mathrm{KF}}^\top, \qquad B_{\mathrm{LQR}} \leftrightarrow C_{\mathrm{KF}}^\top, \qquad Q_{\mathrm{LQR}} \leftrightarrow Q_w, \qquad R_{\mathrm{LQR}} \leftrightarrow R_v,$$

the LQR Riccati recursion becomes the Kalman prior-covariance recursion.

Proof. The LQR Riccati recursion is

$$P_k = Q + A^\top P_{k+1}A - A^\top P_{k+1}B(R + B^\top P_{k+1}B)^{-1}B^\top P_{k+1}A.$$

Replace

$$A \mapsto A^\top, \qquad B \mapsto C^\top, \qquad Q \mapsto Q_w, \qquad R \mapsto R_v.$$

Then the recursion becomes

$$\Pi_k = Q_w + A\Pi_{k+1}A^\top - A\Pi_{k+1}C^\top(R_v + C\Pi_{k+1}C^\top)^{-1}C\Pi_{k+1}A^\top,$$

which is exactly the Kalman prior-covariance recursion after a shift of the time index:

$$P_{k+1|k} = AP_{k|k-1}A^\top + Q_w - AP_{k|k-1}C^\top(CP_{k|k-1}C^\top + R_v)^{-1}CP_{k|k-1}A^\top.$$

So the two Riccati theories are the same algebra viewed through transposed roles of control and sensing. ■

Remark (Meaning of the duality). The duality means that:

- stabilizability for control becomes detectability for estimation,
- control gain design becomes observer gain design,
- the same numerical Riccati solvers apply to both problems after transposition and relabeling.

In that precise sense, solving one problem immediately solves the other.

7.7 Innovation Sequence

Definition 7.7 (Innovation). The *innovation* at time k is

$$\nu_k = y_k - C\hat{x}_{k|k-1}.$$

It is the part of the measurement that was not predictable from past data.

Proposition 7.2 (Innovations are white for a correctly tuned filter). Assume the Kalman filter is correctly matched to the system and noise statistics. Then the innovation sequence $\{\nu_k\}$ is zero mean and white:

$$\mathbb{E}[\nu_k] = 0, \qquad \mathbb{E}[\nu_k\nu_j^\top] = 0 \quad \text{for } j < k.$$

Proof. By definition,

$$\nu_k = y_k - \mathbb{E}[y_k \mid \mathcal{Y}_{k-1}],$$

because

$$C\hat{x}_{k|k-1} = C\mathbb{E}[x_k \mid \mathcal{Y}_{k-1}] = \mathbb{E}[Cx_k \mid \mathcal{Y}_{k-1}]$$

and $\mathbb{E}[v_k \mid \mathcal{Y}_{k-1}] = 0$. Thus ν_k is a martingale difference:

$$\mathbb{E}[\nu_k \mid \mathcal{Y}_{k-1}] = 0.$$

Taking expectations yields

$$\mathbb{E}[\nu_k] = 0.$$

Now let $j < k$. Since ν_j is measurable with respect to $\mathcal{Y}_{k-1}$,

$$\begin{aligned}\mathbb{E}[\nu_k\nu_j^\top] &= \mathbb{E}\big[\mathbb{E}[\nu_k\nu_j^\top \mid \mathcal{Y}_{k-1}]\big] \\ &= \mathbb{E}\big[\mathbb{E}[\nu_k \mid \mathcal{Y}_{k-1}]\nu_j^\top\big] \\ &= 0.\end{aligned}$$

So the innovation sequence is white. ■

Remark (Why innovations matter). If the innovations cease to look white in practice, then either the model, the assumed noise covariances, or the numerical implementation is wrong. Innovation whiteness is therefore a standard diagnostic for filter health monitoring and covariance tuning.

7.8 Extended Kalman Filter

Definition 7.8 (Extended Kalman filter). For the nonlinear system

$$\begin{aligned} x_{k+1} &= f(x_k, u_k) + w_k, \\ y_k &= h(x_k) + v_k, \end{aligned}$$

the extended Kalman filter (EKF) linearizes the dynamics and measurement maps around the current estimate:

$$A_k = \left.\frac{\partial f}{\partial x}\right|_{(\hat{x}_{k|k}, u_k)}, \qquad C_k = \left.\frac{\partial h}{\partial x}\right|_{\hat{x}_{k|k-1}}.$$

It then runs the standard Kalman recursion using these time-varying Jacobians.

Remark (Limitations of the EKF). The EKF is only a local linearization method.

- It is no longer globally optimal, even under Gaussian noise.
- Large nonlinearities can make the covariance inaccurate.
- Poor linearization points can cause divergence.

So the EKF should be understood as a practical approximation, not as an exact nonlinear filtering theory.

Remark (Information filter). The information filter works with the inverse covariance

$$Y_k = P_{k|k}^{-1}$$

and the information state

$$\eta_k = Y_k \hat{x}_{k|k}.$$

In this form, measurement updates become additive in information space:

$$Y_{k|k} = Y_{k|k-1} + C^\top R_v^{-1} C, \qquad \eta_{k|k} = \eta_{k|k-1} + C^\top R_v^{-1} y_k.$$

This representation is advantageous when measurements are sparse, when many sensors contribute additively, or when distributed sensor fusion is required.

7.9 Scalar Kalman Filter Example

Example (Two Kalman-filter iterations by hand). Consider the scalar model

$$x_{k+1} = x_k + w_k, \qquad y_k = x_k + v_k,$$

with

$$w_k \sim \mathcal{N}(0, 1), \qquad v_k \sim \mathcal{N}(0, 4).$$

Start with

$$\hat{x}_{0|-1} = 0, \qquad P_{0|-1} = 10,$$

and measurements

$$y_0 = 3, \qquad y_1 = 4.$$

At time $k = 0$, the Kalman gain is

$$L_0 = \frac{10}{10+4} = \frac{5}{7}.$$

The filtered estimate is

$$\hat{x}_{0|0} = 0 + \frac{5}{7}(3-0) = \frac{15}{7}.$$

The filtered covariance is

$$P_{0|0} = \left(1 - \frac{5}{7}\right) 10 = \frac{20}{7}.$$

The one-step prediction to time $k = 1$ is

$$\hat{x}_{1|0} = \hat{x}_{0|0} = \frac{15}{7},$$

$$P_{1|0} = P_{0|0} + 1 = \frac{20}{7} + 1 = \frac{27}{7}.$$

At time $k = 1$, the Kalman gain is

$$L_1 = \frac{27/7}{27/7+4} = \frac{27}{55}.$$

The innovation is

$$\nu_1 = 4 - \frac{15}{7} = \frac{13}{7}.$$

So the updated estimate is

$$\hat{x}_{1|1} = \frac{15}{7} + \frac{27}{55}\frac{13}{7} = \frac{168}{55}.$$

The updated covariance is

$$P_{1|1} = \left(1 - \frac{27}{55}\right)\frac{27}{7} = \frac{108}{55}.$$

Thus after two iterations,

$$L_0 = \frac{5}{7}, \qquad \hat{x}_{0|0} = \frac{15}{7}, \qquad P_{0|0} = \frac{20}{7},$$

and

$$L_1 = \frac{27}{55}, \qquad \hat{x}_{1|1} = \frac{168}{55}, \qquad P_{1|1} = \frac{108}{55}.$$

Lecture 8: LQG Control, Trajectory Optimization, and iLQR

Intuition. Section 6.2 solved the fully observed linear-quadratic control problem, and Section 7.2 solved the linear-Gaussian state-estimation problem. The first half of this lecture shows how these pieces combine into the linear quadratic Gaussian controller. The second half leaves the linear world: once the dynamics or costs become nonlinear, exact Riccati theory is lost, and one instead repeatedly linearizes the dynamics, quadratizes the cost, solves a local LQR problem, and updates the nominal trajectory. That is the basic idea behind iLQR and DDP.

8.1 Linear Quadratic Gaussian Control

Definition 8.1 (Discrete-time LQG problem)**.** The discrete-time linear quadratic Gaussian (LQG) problem is

$$\begin{aligned} x_{k+1} &= Ax_k + Bu_k + w_k, \\ y_k &= Cx_k + v_k, \end{aligned}$$

with zero-mean white noises

$$\mathbb{E}[w_k w_k^\top] = Q_w \succeq 0, \qquad \mathbb{E}[v_k v_k^\top] = R_v \succ 0,$$

and quadratic cost

$$J = \mathbb{E}\left[x_N^\top Q_f x_N + \sum_{k=0}^{N-1} \left(x_k^\top Q x_k + u_k^\top R u_k\right)\right],$$

where

$$Q \succeq 0, \qquad R \succ 0, \qquad Q_f \succeq 0.$$

The information structure is *output feedback*: at time k, the control u_k must be measurable with respect to

$$\mathcal{I}_k = \sigma(y_0, \ldots, y_k, u_0, \ldots, u_{k-1}).$$

Theorem 8.1 (Separation principle)**.** For the linear-Gaussian-quadratic problem above, the optimal controller has the certainty-equivalent structure

$$u_k^\star = -K_k \hat{x}_{k|k},$$

where:

- $\hat{x}_{k|k}$ is the Kalman-filter estimate of the state given $\mathcal{I}_k$,
- K_k is the finite-horizon LQR gain obtained from the Riccati recursion in Theorem 6.1.

In particular, optimal estimation and optimal control can be designed independently and then interconnected.

Proof outline. Write the state as

$$x_k = \hat{x}_{k|k} + e_k,$$

where

$$\hat{x}_{k|k} = \mathbb{E}[x_k \mid \mathcal{I}_k]$$

and e_k is the estimation error. Since $\hat{x}_{k|k}$ is the conditional mean, it is orthogonal to the error:

$$\mathbb{E}[e_k \mid \mathcal{I}_k] = 0.$$

Substituting $x_k = \hat{x}_{k|k} + e_k$ into the stage cost and taking conditional expectations gives

$$\begin{aligned}\mathbb{E}[x_k^\top Q x_k + u_k^\top R u_k \mid \mathcal{I}_k] = \hat{x}_{k|k}^\top Q \hat{x}_{k|k} + u_k^\top R u_k \\ + \mathbb{E}[e_k^\top Q e_k \mid \mathcal{I}_k].\end{aligned}$$

The last term does not depend on the control. So, conditioned on the information state, the controllable part of the cost is exactly the same quadratic form as in the fully observed LQR problem, but with the true state replaced by the estimate.

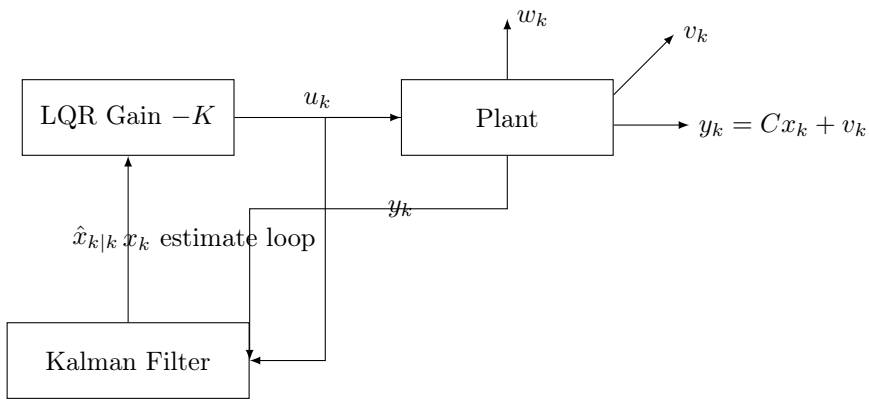
Meanwhile, the Kalman filter is the minimum-variance estimator and its covariance recursion is independent of the control law in the standard additive-noise LQG setup. Therefore the estimator design can be solved independently through Section 7.2, and the control design can be solved independently through Section 6.2.

The resulting optimal controller is certainty equivalent:

$$u_k^\star = -K_k \hat{x}_{k|k}.$$

This separation is special to the linear-Gaussian-quadratic setting and does not hold in general for nonlinear systems or non-Gaussian noise. ■

Remark (Block diagram)**.** The LQG architecture is the interconnection of a Kalman filter and an LQR state-feedback law:



The plant is driven by the feedback input computed from the estimated state, not the true state.

Example (Scalar LQG design). Consider

$$x_{k+1} = 0.8x_k + u_k + w_k, \qquad y_k = x_k + v_k,$$

with

$$Q = 1, \qquad R = 1, \qquad Q_w = 0.5, \qquad R_v = 2.$$

For the control design, the scalar DARE is

$$P = 1 + 0.64P - \frac{0.64P^2}{1+P},$$

which simplifies to

$$P^2 - 0.64P - 1 = 0.$$

The stabilizing root is

$$P = \frac{0.64 + \sqrt{4.4096}}{2} \approx 1.370.$$

Hence the steady-state LQR gain is

$$K = \frac{0.8P}{1+P} \approx 0.462.$$

For the estimation design, using the prior-covariance DARE from Theorem 7.4 gives

$$\bar{P} = 0.64\bar{P} + 0.5 - \frac{0.64\bar{P}^2}{\bar{P}+2},$$

or equivalently

$$\bar{P}^2 + 0.22\bar{P} - 1 = 0.$$

The positive root is

$$\bar{P} = \frac{-0.22 + \sqrt{4.0484}}{2} \approx 0.896.$$

The steady-state Kalman gain is therefore

$$L = \frac{\bar{P}}{\bar{P}+2} \approx 0.309.$$

The complete steady-state LQG controller is

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + 0.309\,(y_k - \hat{x}_{k|k-1}),$$

$$\hat{x}_{k+1|k} = 0.8\hat{x}_{k|k} + u_k,$$

$$u_k = -0.462\,\hat{x}_{k|k}.$$

8.2 Robustness of LQG

Theorem 8.2 (Doyle's counterexample). LQG does not inherit the guaranteed gain and phase margins of full-state-feedback LQR. In fact, there exist plants for which the LQG controller has arbitrarily poor robustness margins.

Explanation. The LQR robustness theorem Theorem 6.8 is a property of the full-state-feedback loop and the return-difference identity induced by the Riccati equation. Once the true state is replaced by an observer state, the controller becomes a dynamic output-feedback compensator. The added estimator dynamics introduce new poles and zeros into the loop and destroy the return-difference structure responsible for the LQR margins. Doyle's counterexample shows that even though the LQR part and the Kalman-filter part are each individually optimal, their interconnection can have extremely weak robustness margins. ■

Remark (Why the observer can destroy robustness). The Kalman filter is tuned for estimation performance under the assumed stochastic model, not for worst-case loop shaping. If the plant model is slightly wrong, if hidden nonminimum-phase behavior appears, or if the estimator bandwidth interacts badly with the controller bandwidth, then the output-feedback loop can amplify uncertainty much more strongly than the full-state-feedback loop would.

Definition 8.2 (Loop transfer recovery). Loop transfer recovery (LTR) is a design methodology that modifies the Kalman-filter design, usually by artificially inflating the process-noise covariance, so that the open-loop transfer of the resulting LQG compensator approaches the loop transfer of the underlying LQR design over a desired frequency range.

Remark (Tradeoff in LTR). LTR can recover much of the lost robustness, but this comes at a price: the estimator becomes high gain, measurement noise is amplified more strongly, and the ideal recovery may fail when plant nonminimum-phase zeros or modeling limitations intervene.

Remark (LQG as $\mathcal{H}_2$ optimal control). LQG is a special case of $\mathcal{H}_2$ optimal control. The quadratic cost and Gaussian-noise covariance interpretation lead to minimization of the mean output energy of a generalized closed-loop system driven by white noise. In this setting, the Riccati equations of control and estimation are precisely the two Riccati equations that appear in the standard $\mathcal{H}_2$ synthesis formulas.

Remark (Limitations and common extensions). Standard LQG has four structural limitations:

- linear dynamics: addressed by gain scheduling, EKF-based output feedback, iLQR/DDP, or NMPC,
- Gaussian noise: addressed by robust filtering, particle filtering, or risk-sensitive control,
- quadratic cost: addressed by $\mathcal{H}_\infty$ or more general optimal-control formulations,
- no constraints: addressed by MPC and constrained trajectory optimization.

So LQG is powerful but narrow.

8.3 Nonlinear Trajectory Optimization

Definition 8.3 (General nonlinear trajectory optimization problem). A discrete-time nonlinear trajectory-

optimization problem has the form

$$\begin{aligned} \min_{\{x_k,u_k\}} \quad & \phi(x_N) + \sum_{k=0}^{N-1} \ell_k(x_k,u_k) \\ \text{subject to} \quad & x_{k+1} = f_k(x_k,u_k), \qquad k = 0,\dots,N-1, \\ & x_0 \text{ given.} \end{aligned}$$

The dynamics f_k and costs ℓ_k may be nonlinear and time varying.

Remark (Why LQR and LQG are insufficient)**.** LQR and LQG solve linear systems with quadratic objectives. Real robotic systems are often nonlinear, may operate around highly nonstationary trajectories, and can have objectives that are only locally quadratic. In such settings, exact Riccati theory does not apply globally.

Intuition. The central idea behind local trajectory methods is:

(a) choose a nominal state-control trajectory,

(b) linearize the dynamics and quadratize the cost around that nominal trajectory,

(c) solve the resulting local LQR problem,

(d) update the nominal trajectory and repeat.

The local Riccati solve is cheap; the iteration handles the nonlinearity.

8.4 Iterative LQR

Definition 8.4 (What iLQR solves)**.** Iterative LQR (iLQR) solves nonlinear finite-horizon trajectory-optimization problems by repeatedly approximating them with time-varying linear-quadratic subproblems. Standard LQR computes the optimal controller for a *given* linear-quadratic model. iLQR handles nonlinear dynamics and, more generally, smooth nonlinear costs by solving a sequence of local LQR problems.

Definition 8.5 (Linearization and quadratization about a nominal trajectory)**.** Given a nominal trajectory

$$\{(\bar{x}_k,\bar{u}_k)\}_{k=0}^{N-1},$$

define perturbations

$$\delta x_k = x_k - \bar{x}_k, \qquad \delta u_k = u_k - \bar{u}_k.$$

The dynamics are linearized as

$$\delta x_{k+1} = A_k \delta x_k + B_k \delta u_k,$$

where

$$A_k = \left.\frac{\partial f_k}{\partial x}\right|_{(\bar{x}_k,\bar{u}_k)}, \qquad B_k = \left.\frac{\partial f_k}{\partial u}\right|_{(\bar{x}_k,\bar{u}_k)}.$$

A smooth stage cost is quadratized as

$$\begin{aligned} \ell_k(x_k,u_k) \approx\ & \ell_k(\bar{x}_k,\bar{u}_k) + \ell_{x,k}^\top \delta x_k + \ell_{u,k}^\top \delta u_k \\ & + \frac{1}{2}\delta x_k^\top \ell_{xx,k}\delta x_k + \delta u_k^\top \ell_{ux,k}\delta x_k + \frac{1}{2}\delta u_k^\top \ell_{uu,k}\delta u_k, \end{aligned}$$

with all derivatives evaluated at the nominal pair.

Remark. The cost must be quadratized because the backward Riccati pass requires a local quadratic model of the objective. A purely first-order model would identify only descent directions, not a curvature-based local feedback law.

Definition 8.6 (Local Q-function derivatives in iLQR)**.** Let the quadratic local value approximation at stage $k+1$ be

$$\delta V_{k+1}(\delta x) = V_{x,k+1}^\top \delta x + \frac{1}{2}\delta x^\top V_{xx,k+1}\delta x.$$

Then the one-step local action-value derivatives are

$$Q_{x,k} = \ell_{x,k} + A_k^\top V_{x,k+1}, \qquad Q_{u,k} = \ell_{u,k} + B_k^\top V_{x,k+1},$$

$$Q_{xx,k} = \ell_{xx,k} + A_k^\top V_{xx,k+1} A_k,$$

$$Q_{ux,k} = \ell_{ux,k} + B_k^\top V_{xx,k+1} A_k, \qquad Q_{uu,k} = \ell_{uu,k} + B_k^\top V_{xx,k+1} B_k.$$

Proposition 8.1 (Backward-pass gains)**.** If $Q_{uu,k} \succ 0$, then the minimizer of the local quadratic model at stage k is

$$\delta u_k^\star = k_k + K_k \delta x_k,$$

with

$$k_k = -Q_{uu,k}^{-1} Q_{u,k}, \qquad K_k = -Q_{uu,k}^{-1} Q_{ux,k}.$$

Proof. The local quadratic model in $(\delta x_k, \delta u_k)$ is

$$\delta Q_k = c_k + Q_{x,k}^\top \delta x_k + Q_{u,k}^\top \delta u_k + \frac{1}{2}\delta x_k^\top Q_{xx,k}\delta x_k + \delta u_k^\top Q_{ux,k}\delta x_k + \frac{1}{2}\delta u_k^\top Q_{uu,k}\delta u_k.$$

For fixed δx_k, differentiating with respect to δu_k gives

$$0 = Q_{u,k} + Q_{ux,k}\delta x_k + Q_{uu,k}\delta u_k.$$

Solving for δu_k yields

$$\delta u_k^\star = -Q_{uu,k}^{-1} Q_{u,k} - Q_{uu,k}^{-1} Q_{ux,k}\delta x_k.$$

■

Algorithm 16: iLQR

Input: Nominal controls $\{\bar{u}_k\}_{k=0}^{N-1}$, initial state x_0, tolerance ε

Simulate the dynamics forward to obtain the nominal trajectory $\{\bar{x}_k\}$

for $r = 0, 1, 2, \ldots$ **do**

- Linearize the dynamics and quadratize the stage and terminal costs along the current nominal trajectory
- Run the backward Riccati-style sweep to compute feedforward terms k_k and feedback gains K_k
- Perform a forward rollout with line search using

$$u_k^{\text{new}} = \bar{u}_k + \alpha k_k + K_k(x_k^{\text{new}} - \bar{x}_k)$$

  for a step size $\alpha \in (0, 1]$
- Accept the new trajectory if the cost decreases sufficiently
- Stop if the decrease and gain corrections are below tolerance

Proposition 8.2 (iLQR as approximate Newton on trajectory space)**.** iLQR can be interpreted as an approximate Newton method for the nonlinear trajectory-optimization problem. The approximation is that iLQR keeps second-order cost terms but neglects second-order derivatives of the dynamics in the local quadratic model.

Proof. Let

$$z = (x_1, \dots, x_N, u_0, \dots, u_{N-1})$$

denote the stacked trajectory variables, and write the dynamics defects as

$$c_k(z) = x_{k+1} - f_k(x_k, u_k) = 0.$$

A Newton or SQP step for the trajectory problem constructs a second-order model of the Lagrangian

$$\mathcal{L}(z, \lambda) = J(z) + \sum_{k=0}^{N-1} \lambda_{k+1}^\top c_k(z)$$

together with linearized constraints. The exact Hessian of the Lagrangian includes not only the Hessian of the cost but also terms of the form

$$\sum_i \lambda_{k+1,i} \nabla^2 f_{k,i}(x_k, u_k),$$

coming from the second derivatives of the dynamics.

In DDP, these second-order dynamics terms appear explicitly in the local Bellman expansion. In iLQR, one linearizes the dynamics and therefore discards those second-order contributions, while retaining the second-order cost terms and the curvature induced by the propagated value Hessian. The resulting local subproblem is exactly a time-varying LQR problem on the linearized system.

Thus iLQR is Newton-like because it uses local second-order curvature information, but it is only an *approximate* Newton method because it replaces the full second-order constrained model by a Gauss-Newton-type approximation that ignores the dynamics Hessians. ■

Remark (No global optimality guarantee). iLQR is a local method. The trajectory-optimization problem is generally nonconvex, so the algorithm may converge to a local minimum, a saddle-adjacent stationary point, or may fail if the local quadratic model is poor. Initialization matters.

8.5 iLQR: Practical Considerations

Definition 8.7 (Regularization). Regularization modifies the backward pass to keep

$$Q_{uu,k}$$

positive definite and the local model numerically well conditioned. A common strategy is Levenberg–Marquardt regularization:

$$Q_{uu,k} \leftarrow Q_{uu,k} + \rho I, \qquad \rho > 0.$$

Remark (Why regularization is needed). Without regularization, the local curvature matrix $Q_{uu,k}$ may become indefinite or nearly singular. Then the local subproblem can fail to define a descent direction, the gains can explode, and the forward rollout can become unstable.

Remark (Line search). The forward pass can increase the true nonlinear cost even when the local model predicts improvement. So iLQR typically uses a backtracking line search on a scalar step size α multiplying the feedforward term:

$$u_k^{\text{trial}} = \bar{u}_k + \alpha k_k + K_k(x_k^{\text{trial}} - \bar{x}_k).$$

If the trial rollout fails to decrease the true cost sufficiently, α is reduced and the rollout is repeated.

Remark (Computational complexity). For state dimension n, control dimension m, and horizon length

N, one backward-forward sweep of iLQR costs roughly

$$O\big(N(n^3 + n^2m + nm^2 + m^3)\big)\,.$$

The key point is linear scaling in the horizon N because the method uses dynamic-programming structure rather than solving one giant dense optimization problem. This is much cheaper than generic direct-transcription methods when the horizon is long and the control dimension is moderate.

8.6 Differential Dynamic Programming

Definition 8.8 (DDP versus iLQR)**.** Differential dynamic programming (DDP) is the full second-order version of local dynamic-programming trajectory optimization. It expands both the cost and the dynamics to second order. iLQR is the simplified variant that retains only first-order dynamics and therefore ignores the second-order dynamics tensors.

Proposition 8.3 (Full second-order Q-function derivatives in DDP)**.** Let

$$x_{k+1} = f_k(x_k, u_k)$$

and suppose the stage cost is $\ell_k(x_k, u_k)$, with local quadratic value model at stage $k+1$

$$V_{k+1}(\bar{x}_{k+1} + \delta x) \approx V_{k+1} + V_{x,k+1}^\top \delta x + \frac{1}{2}\delta x^\top V_{xx,k+1}\delta x.$$

Write

$$A_k = \partial_x f_k(\bar{x}_k, \bar{u}_k), \qquad B_k = \partial_u f_k(\bar{x}_k, \bar{u}_k),$$

and let

$$f_{xx,k}^{(i)}, \qquad f_{ux,k}^{(i)}, \qquad f_{uu,k}^{(i)}$$

denote the Hessians of the ith component of f_k. Then the DDP local action-value derivatives are

$$Q_{x,k} = \ell_{x,k} + A_k^\top V_{x,k+1}, \qquad Q_{u,k} = \ell_{u,k} + B_k^\top V_{x,k+1},$$

$$Q_{xx,k} = \ell_{xx,k} + A_k^\top V_{xx,k+1} A_k + \sum_{i=1}^{n} (V_{x,k+1})_i\, f_{xx,k}^{(i)},$$

$$Q_{ux,k} = \ell_{ux,k} + B_k^\top V_{xx,k+1} A_k + \sum_{i=1}^{n} (V_{x,k+1})_i\, f_{ux,k}^{(i)},$$

$$Q_{uu,k} = \ell_{uu,k} + B_k^\top V_{xx,k+1} B_k + \sum_{i=1}^{n} (V_{x,k+1})_i\, f_{uu,k}^{(i)}.$$

Proof sketch. The Bellman one-step function is

$$Q_k(x_k, u_k) = \ell_k(x_k, u_k) + V_{k+1}(f_k(x_k, u_k)).$$

The first derivatives follow from the chain rule exactly as in iLQR. The second derivatives follow by differentiating the composite term

$$V_{k+1}(f_k(x_k, u_k))$$

one more time. This produces two kinds of curvature terms:

- propagated curvature from the Hessian $V_{xx,k+1}$ acting through the Jacobians A_k and B_k,
- direct second-order curvature from the dynamics themselves, weighted by the gradient $V_{x,k+1}$.

The latter are exactly the tensor-contraction terms absent from iLQR. ■

Proposition 8.4 (DDP backward-pass update). If

$$Q_{uu,k} \succ 0,$$

then the minimizing local quadratic control law in DDP is

$$\delta u_k^\star = k_k + K_k \delta x_k,$$

with

$$k_k = -Q_{uu,k}^{-1} Q_{u,k}, \qquad K_k = -Q_{uu,k}^{-1} Q_{ux,k}.$$

The corresponding value-function derivatives update according to

$$V_{x,k} = Q_{x,k} - Q_{xu,k} Q_{uu,k}^{-1} Q_{u,k},$$

$$V_{xx,k} = Q_{xx,k} - Q_{xu,k} Q_{uu,k}^{-1} Q_{ux,k}.$$

Proof. Once the full DDP derivatives are available, the local model is still quadratic in $(\delta x_k, \delta u_k)$. Therefore minimizing with respect to δu_k is the same algebraic step as in Proposition 8.1: differentiate the quadratic model, solve the linear system in δu_k, and substitute the minimizer back into the local model. The only difference from iLQR lies upstream, in the presence of the second-order dynamics terms inside $Q_{xx,k}$, $Q_{ux,k}$, and $Q_{uu,k}$. ■

Remark. In DDP, the quadratic terms include tensor contractions such as

$$\sum_i V_{x,k+1,i} f_{xx,k}^{(i)}, \qquad \sum_i V_{x,k+1,i} f_{ux,k}^{(i)}, \qquad \sum_i V_{x,k+1,i} f_{uu,k}^{(i)},$$

which are absent in iLQR. These terms matter when the dynamics are strongly nonlinear and second-order dynamics curvature materially changes the local model.

Remark. iLQR can therefore be viewed as the Gauss–Newton approximation to DDP's full Newton step. The gain is simplicity and numerical robustness; the cost is that the local model can be less accurate on strongly nonlinear problems.

Algorithm 17: Differential Dynamic Programming

Input: Nominal controls $\{\bar{u}_k\}_{k=0}^{N-1}$, initial state x_0, dynamics derivatives up to second order, tolerance ε

Roll out the nominal trajectory $\{\bar{x}_k\}$ and compute the nominal cost
for $r = 0, 1, 2, \ldots$ *until convergence* **do**
  Form first- and second-order derivatives of the stage costs and dynamics along the current trajectory
  Run a backward pass using Proposition 8.3 and Proposition 8.4 to compute k_k, K_k, $V_{x,k}$, and $V_{xx,k}$
  Regularize $Q_{uu,k}$ when necessary to maintain a well-posed local minimization
  Perform a forward rollout with line search using

$$u_k^{\text{new}} = \bar{u}_k + \alpha k_k + K_k(x_k^{\text{new}} - \bar{x}_k)$$

  and accept the step when the true nonlinear cost decreases sufficiently

Output: Locally optimal state-control trajectory and local feedback gains

Remark (When DDP earns its extra cost). If the dynamics are only mildly nonlinear over the region explored by the current trajectory, then iLQR is often preferable because it avoids second-order dynamics tensors and is easier to regularize. DDP becomes attractive when the local curvature of the dynamics genuinely matters, for example in aggressive underactuated maneuvers, contact-rich transitions, or problems where the first-order dynamics model is too poor to predict useful descent

steps.

Example (Acrobot swing-up as a motivating example). The Acrobot is a two-link underactuated pendulum with actuation only at the elbow joint. A typical swing-up objective is to move from the hanging configuration to the upright unstable equilibrium while minimizing control effort and terminal error.

This problem is a natural test case for iLQR and DDP because:

- the dynamics are strongly nonlinear away from the upright equilibrium,
- the maneuver requires energy pumping rather than local stabilization,
- the system is underactuated, so naive local feedback around the start state is useless.

In practice, both iLQR and DDP can solve the swing-up from a crude initial guess by repeatedly improving a nominal trajectory. DDP can produce a more faithful local model when the second-order dynamics terms materially affect the energy-transfer mechanism, but iLQR is often preferred in implementation because it avoids third-order derivative bookkeeping while retaining the same Riccati-style computational structure.

8.7 Handling Constraints in Trajectory Optimization

Remark. Standard iLQR and DDP do not natively handle general inequality constraints such as actuator saturation, state bounds, or obstacle avoidance. Their native formulation is essentially unconstrained.

Definition 8.9 (Augmented-Lagrangian iLQR). Let the nonlinear trajectory optimization problem include constraints

$$c_k(x_k, u_k) = 0, \qquad g_k(x_k, u_k) \leq 0.$$

Augmented-Lagrangian iLQR (AL-iLQR) forms a penalized local objective

$$\mathcal{L}_\rho = J + \sum_k \lambda_k^\top c_k + \frac{\rho}{2} \sum_k \|c_k\|^2 + (\text{analogous penalty/barrier terms for inequalities}).$$

One then applies iLQR to the augmented objective while updating multipliers and penalty parameters across outer iterations.

Remark (Other approaches). Common alternatives include:

- barrier penalties for state or control limits,
- projected or clamped forward passes for box-constrained controls,
- hybrid schemes that solve local QPs inside the backward pass,
- switching to direct transcription or MPC when constraints dominate the problem geometry.

8.8 Worked iLQR Example

Example (One iLQR iteration for a scalar nonlinear system). Consider

$$x_{k+1} = x_k + u_k + 0.1x_k^2,$$

with horizon $N = 2$, initial state

$$x_0 = 1,$$

and cost

$$J = x_2^2 + \sum_{k=0}^{1}(x_k^2 + u_k^2).$$

Choose the nominal controls

$$\bar{u}_0 = \bar{u}_1 = 0.$$

The forward rollout gives

$$\bar{x}_0 = 1, \qquad \bar{x}_1 = 1 + 0 + 0.1(1)^2 = 1.1,$$

$$\bar{x}_2 = 1.1 + 0 + 0.1(1.1)^2 = 1.221.$$

The dynamics Jacobians are

$$A_k = 1 + 0.2\bar{x}_k, \qquad B_k = 1.$$

So

$$A_0 = 1.2, \qquad A_1 = 1.22, \qquad B_0 = B_1 = 1.$$

The stage cost

$$\ell(x, u) = x^2 + u^2$$

is already quadratic, so

$$\ell_x = 2\bar{x}, \qquad \ell_u = 2\bar{u}, \qquad \ell_{xx} = 2, \qquad \ell_{uu} = 2, \qquad \ell_{ux} = 0.$$

The terminal cost

$$\phi(x_2) = x_2^2$$

has derivatives

$$V_{x,2} = 2\bar{x}_2 = 2.442, \qquad V_{xx,2} = 2.$$

At stage $k = 1$, the local derivatives are

$$Q_{x,1} = 2.2 + 1.22(2.442) = 5.17924,$$

$$Q_{u,1} = 2.442, \qquad Q_{xx,1} = 2 + 1.22^2(2) = 4.9768,$$

$$Q_{ux,1} = 2.44, \qquad Q_{uu,1} = 4.$$

Hence

$$k_1 = -\frac{2.442}{4} = -0.6105, \qquad K_1 = -\frac{2.44}{4} = -0.61.$$

The propagated value derivatives are

$$V_{x,1} = Q_{x,1} - Q_{ux,1}Q_{uu,1}^{-1}Q_{u,1} = 3.68962,$$

$$V_{xx,1} = Q_{xx,1} - Q_{ux,1}Q_{uu,1}^{-1}Q_{ux,1} = 3.4884.$$

At stage $k = 0$, this gives

$$Q_{x,0} = 2 + 1.2(3.68962) = 6.427544,$$

$$Q_{u,0} = 3.68962, \qquad Q_{xx,0} = 2 + 1.2^2(3.4884) = 7.023296,$$

$$Q_{ux,0} = 1.2(3.4884) = 4.18608, \qquad Q_{uu,0} = 2 + 3.4884 = 5.4884.$$

So

$$k_0 = -\frac{3.68962}{5.4884} \approx -0.6723, \qquad K_0 = -\frac{4.18608}{5.4884} \approx -0.7627.$$

Now perform a full forward update with step size $\alpha = 1$. Since

$$\delta x_0 = 0,$$

we obtain

$$u_0^{\text{new}} = \bar{u}_0 + k_0 \approx -0.6723.$$

The new state is

$$x_1^{\text{new}} = 1 - 0.6723 + 0.1(1)^2 = 0.4277.$$

Thus

$$\delta x_1 = x_1^{\text{new}} - \bar{x}_1 \approx -0.6723.$$

At stage $k = 1$,

$$u_1^{\text{new}} = \bar{u}_1 + k_1 + K_1 \delta x_1 \approx -0.6105 + (-0.61)(-0.6723) \approx -0.2003.$$

The terminal state becomes

$$x_2^{\text{new}} = 0.4277 - 0.2003 + 0.1(0.4277)^2 \approx 0.2457.$$

The nominal cost was

$$J_{\text{nom}} = 1^2 + 1.1^2 + 1.221^2 \approx 3.7008.$$

The updated cost is approximately

$$J_{\text{new}} = 1^2 + (-0.6723)^2 + 0.4277^2 + (-0.2003)^2 + 0.2457^2 \approx 1.736.$$

So one iLQR iteration produces a substantial cost decrease.

8.9 Choosing Between Direct Collocation, iLQR, and DDP

Property	Direct collocation	iLQR	DDP
Decision variables	Full discretized state-control trajectory in an NLP	Nominal control trajectory plus local feedback corrections	Same rollout structure as iLQR, but with full second-order dynamics terms
Constraint handling	Natural for boundary, path, and state constraints	Indirect; usually via penalties, projection, or augmented Lagrangians	Same limitation as iLQR; local constrained variants are more delicate
Local model	Sparse NLP with generic nonlinear constraints	Quadratic cost with linearized dynamics	Quadratic cost with second-order-expanded dynamics
Derivative burden	First derivatives and often sparse second derivatives of the NLP	First derivatives of dynamics and second derivatives of cost	First and second derivatives of both cost and dynamics
Convergence behavior	Robust with mature SQP/interior-point globalization	Fast local improvement when initialized reasonably	Potentially better local Newton step, but more sensitive numerically
Best use case	Hard constraints, stiff dynamics, or general-purpose transcription	Fast local trajectory improvement for moderate nonlinearities	Strongly nonlinear problems where dynamics curvature matters materially

Remark. This table extends the broader trajectory-optimization comparison in Section 5.9. The main tradeoff is now clear: direct methods win on generality and constraint handling, while Riccati-style methods win on horizon scalability and fast local policy improvement.

Lecture 9: Model Predictive Control

Intuition. Model predictive control turns optimal control into an online feedback law. At every sampling instant, one solves a finite-horizon optimal-control problem using the current state as the initial condition, applies only the first control move, shifts the horizon forward, and repeats. This simple idea is what allows MPC to combine prediction, constraint handling, and feedback in one unified framework.

9.1 The MPC Concept

Definition 9.1 (Discrete-time MPC problem formulation)**.** A standard discrete-time MPC problem at time t is

$$\begin{aligned}
\min_{\{x_{k|t},u_{k|t}\}} \quad & V_f(x_{N|t}) + \sum_{k=0}^{N-1} \ell(x_{k|t}, u_{k|t}) \\
\text{subject to} \quad & x_{0|t} = x(t), \\
& x_{k+1|t} = f(x_{k|t}, u_{k|t}), \qquad k = 0, \dots, N-1, \\
& x_{k|t} \in \mathcal{X}, \qquad u_{k|t} \in \mathcal{U}, \\
& x_{N|t} \in \mathcal{X}_f \quad \text{(optional terminal constraint)}.
\end{aligned}$$

Here $x_{k|t}$ denotes the state predicted k steps ahead from time t.

Remark (Why MPC instead of LQR)**.** LQR gives an elegant closed-form law for linear-quadratic problems, but it does not natively handle hard state or input constraints. MPC does. This is one of its decisive practical advantages.

Definition 9.2 (Receding-horizon strategy)**.** After solving the finite-horizon problem at time t, MPC applies only the first input

$$u(t) = u^\star_{0|t},$$

then measures the new state at time $t+1$, resolves the optimization problem with the horizon shifted forward, and repeats. This is the *receding-horizon* or *moving-horizon* strategy.

Algorithm 18: Receding-Horizon MPC

Input: Prediction model, stage cost ℓ, terminal cost V_f, horizon N, constraints

for $t = 0, 1, 2, \dots$ **do**

- Measure or estimate the current state $x(t)$
- Solve the finite-horizon optimal-control problem with initial condition $x_{0|t} = x(t)$
- Obtain the optimal sequence $\{u^\star_{0|t}, \dots, u^\star_{N-1|t}\}$
- Apply only the first control move
$$u(t) \leftarrow u^\star_{0|t}$$
- Shift the horizon forward and repeat at the next sampling instant

Proposition 9.1 (MPC reduces to LQR in the unconstrained linear-quadratic case)**.** Consider the unconstrained linear system

$$x_{k+1} = Ax_k + Bu_k$$

with quadratic stage cost

$$x_k^\top Q x_k + u_k^\top R u_k,$$

and terminal cost

$$V_f(x) = x^\top P x,$$

where P is the stabilizing DARE solution from Theorem 6.3. Then finite-horizon MPC produces the infinite-horizon LQR control law

$$u^\star_{0|t} = -Kx(t)$$

for every horizon length $N \geq 1$.

Proof. Since P solves the DARE,

$$P = Q + A^\top PA - A^\top PB(R + B^\top PB)^{-1} B^\top PA.$$

So if the terminal Riccati matrix is initialized with $P_N = P$, the backward Riccati recursion leaves it unchanged:

$$P_{N-1} = P.$$

Repeating this argument shows

$$P_k = P$$

for all k. Hence every stage gain equals the infinite-horizon LQR gain

$$K = (R + B^\top PB)^{-1} B^\top PA,$$

and in particular the first MPC move is

$$u^\star_{0|t} = -Kx(t).$$

■

9.2 Linear MPC and QP Formulation

Definition 9.3 (Linear MPC). Linear MPC specializes the MPC problem to linear dynamics

$$x_{k+1} = Ax_k + Bu_k,$$

quadratic cost

$$x_N^\top Px_N + \sum_{k=0}^{N-1} \left(x_k^\top Qx_k + u_k^\top Ru_k\right),$$

and typically polyhedral constraints

$$x_k \in \mathcal{X}, \qquad u_k \in \mathcal{U}, \qquad x_N \in \mathcal{X}_f.$$

Theorem 9.1 (Condensing linear MPC into a quadratic program). For linear MPC, the predicted state sequence can be eliminated to produce a finite-dimensional quadratic program in the stacked input vector alone.

Proof. Define the stacked vectors

$$X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix}, \qquad U = \begin{bmatrix} u_0 \\ u_1 \\ \vdots \\ u_{N-1} \end{bmatrix}.$$

Repeated substitution of the dynamics yields

$$x_1 = Ax_0 + Bu_0,$$

$$x_2 = A^2x_0 + ABu_0 + Bu_1,$$

and in general

$$X = \mathcal{A}x_0 + \mathcal{B}U,$$

where

$$\mathcal{A} = \begin{bmatrix} A \\ A^2 \\ \vdots \\ A^N \end{bmatrix},$$

and

$$\mathcal{B} = \begin{bmatrix} B & 0 & \cdots & 0 \\ AB & B & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ A^{N-1}B & A^{N-2}B & \cdots & B \end{bmatrix}.$$

Now define the block-diagonal weight matrices

$$\bar{Q} = \mathrm{blkdiag}(Q, \ldots, Q, P), \qquad \bar{R} = \mathrm{blkdiag}(R, \ldots, R).$$

The cost becomes

$$J = X^\top \bar{Q} X + U^\top \bar{R} U.$$

Substituting

$$X = \mathcal{A} x_0 + \mathcal{B} U$$

gives

$$J = U^\top (\mathcal{B}^\top \bar{Q} \mathcal{B} + \bar{R}) U + 2 x_0^\top \mathcal{A}^\top \bar{Q} \mathcal{B} U + x_0^\top \mathcal{A}^\top \bar{Q} \mathcal{A} x_0.$$

So, up to a constant independent of U, this is a quadratic function of U.

State and input constraints likewise become linear inequalities in U. For instance, if

$$F_x x_k \le b_x, \qquad F_u u_k \le b_u,$$

then stacking them yields a condensed form

$$GU \le g + E x_0.$$

Hence linear MPC can be written as the QP

$$\begin{aligned} \min_U \quad & \frac{1}{2} U^\top H U + h(x_0)^\top U \\ \text{subject to} \quad & GU \le g + E x_0, \end{aligned}$$

with

$$H = 2(\mathcal{B}^\top \bar{Q} \mathcal{B} + \bar{R}).$$

■

Example (Condensed QP for a two-step scalar MPC problem). Consider the scalar integrator

$$x_{k+1} = x_k + u_k$$

with horizon $N = 2$ and cost

$$J = x_2^2 + \sum_{k=0}^{1} (x_k^2 + u_k^2).$$

Let

$$U = \begin{bmatrix} u_0 \\ u_1 \end{bmatrix}.$$

Then

$$x_1 = x_0 + u_0, \qquad x_2 = x_0 + u_0 + u_1.$$

Substituting into the cost gives

$$\begin{aligned} J &= x_0^2 + u_0^2 + (x_0 + u_0)^2 + u_1^2 + (x_0 + u_0 + u_1)^2 \\ &= 3x_0^2 + 3u_0^2 + 2u_0 u_1 + 2u_1^2 + 4x_0 u_0 + 2x_0 u_1. \end{aligned}$$

So the condensed QP is

$$\min_U \quad \frac{1}{2} U^\top \begin{bmatrix} 6 & 2 \\ 2 & 4 \end{bmatrix} U + \begin{bmatrix} 4x_0 \\ 2x_0 \end{bmatrix}^\top U,$$

up to the constant term $3x_0^2$.

Remark (Dense versus sparse formulations). Condensing produces a dense QP in the control variables only. This can be advantageous for short horizons and small input dimensions. The alternative is a sparse formulation retaining both states and controls as decision variables, which produces a larger but highly structured QP. Modern MPC solvers choose between these formulations based on horizon length, state dimension, and hardware constraints.

9.3 Stability of MPC

Intuition. Finite-horizon optimization alone does not automatically imply closed-loop stability. An optimizer with too short a horizon may postpone corrective action or exploit short-term benefits that are harmful in the long run. Stability must therefore be enforced through terminal ingredients or sufficiently long horizons under additional assumptions.

Definition 9.4 (Terminal cost and terminal set). A *terminal cost* is a nonnegative function

$$V_f : \mathbb{R}^n \to \mathbb{R}_{\geq 0}.$$

A *terminal set* is a set

$$\mathcal{X}_f \subseteq \mathcal{X}$$

with the property that there exists a local terminal controller

$$\kappa_f : \mathcal{X}_f \to \mathcal{U}$$

such that

$$f(x, \kappa_f(x)) \in \mathcal{X}_f$$

for all $x \in \mathcal{X}_f$.

Theorem 9.2 (Stability of MPC via a terminal Lyapunov condition). Suppose the stage cost $\ell(x, u)$ is positive definite and there exists a terminal controller κ_f on $\mathcal{X}_f$ such that for all $x \in \mathcal{X}_f$,

$$\kappa_f(x) \in \mathcal{U}, \qquad f(x, \kappa_f(x)) \in \mathcal{X}_f,$$

and

$$V_f(f(x, \kappa_f(x))) - V_f(x) \leq -\ell(x, \kappa_f(x)).$$

Then the optimal MPC cost

$$J_N^\star(x)$$

is a Lyapunov function for the closed-loop system, and the MPC law asymptotically stabilizes the origin on the feasible set.

Proof sketch. Let

$$\{x_{k|t}^\star, u_{k|t}^\star\}_{k=0}^{N-1}$$

be the optimal solution at time t. After applying

$$u(t) = u_{0|t}^\star,$$

the realized next state is

$$x(t+1) = x_{1|t}^\star.$$

Construct a candidate control sequence for the MPC problem at time $t+1$ by shifting the old optimal sequence:

$$\tilde{u}_{k|t+1} = u_{k+1|t}^\star, \qquad k = 0, \ldots, N-2,$$

and appending the terminal controller

$$\tilde{u}_{N-1|t+1} = \kappa_f(x^\star_{N|t}).$$

The terminal-set invariance makes this shifted sequence feasible.

Evaluating the cost of this feasible candidate at time $t+1$ and comparing it with the optimal cost at time t yields

$$J^\star_N(x(t+1)) - J^\star_N(x(t)) \leq -\ell(x(t), u(t)).$$

So the optimal value decreases by at least the positive-definite stage cost. This makes $J^\star_N$ a Lyapunov function and implies asymptotic stability. ■

Remark (Terminal ingredients as an unconstrained tail). In linear MPC, a standard choice is:

- terminal cost
$$V_f(x) = x^\top P x,$$
where P solves the LQR DARE,
- terminal controller
$$\kappa_f(x) = -Kx,$$
where K is the LQR gain,
- terminal set $\mathcal{X}_f$ chosen as a positively invariant set for the constrained closed loop.

Then the tail of the MPC horizon is locally closed by the LQR law.

9.4 Feasibility and Recursive Feasibility

Definition 9.5 (Recursive feasibility). MPC is *recursively feasible* if feasibility of the optimization problem at time t implies feasibility at time $t+1$ under the applied MPC control law.

Proposition 9.2 (Terminal constraint ensures recursive feasibility). Under the same assumptions as the stability theorem, if the MPC problem is feasible at time t, then it is feasible at time $t+1$.

Proof. Use the same shifted-sequence construction as in the stability proof. If

$$\{u^\star_{0|t}, \ldots, u^\star_{N-1|t}\}$$

is feasible at time t, then after applying $u^\star_{0|t}$ the next state is $x^\star_{1|t}$. The shifted sequence

$$\tilde{u}_{0|t+1} = u^\star_{1|t}, \ldots, \tilde{u}_{N-2|t+1} = u^\star_{N-1|t},$$

augmented with

$$\tilde{u}_{N-1|t+1} = \kappa_f(x^\star_{N|t}),$$

satisfies all state, input, and terminal constraints because the original sequence was feasible and the terminal controller keeps the terminal state inside the terminal set. So the optimization problem at time $t+1$ admits at least one feasible point. ■

Remark (Soft constraints). Hard constraints may make the MPC problem infeasible under disturbances, modeling errors, or aggressive references. A common practical relaxation introduces slack variables

$$s_k \geq 0$$

and replaces hard inequalities by softened constraints such as

$$F_x x_k \leq b_x + s_k,$$

while penalizing $\|s_k\|$ heavily in the cost. This preserves solvability at the expense of allowing controlled constraint violation when necessary.

9.5 Nonlinear MPC

Definition 9.6 (Nonlinear MPC). Nonlinear MPC (NMPC) uses the same receding-horizon strategy as linear MPC but solves at each step a nonlinear optimal-control problem:

$$\begin{aligned} \min_{\{x_{k|t}, u_{k|t}\}} \quad & V_f(x_{N|t}) + \sum_{k=0}^{N-1} \ell(x_{k|t}, u_{k|t}) \\ \text{subject to} \quad & x_{0|t} = x(t), \\ & x_{k+1|t} = f(x_{k|t}, u_{k|t}), \\ & c(x_{k|t}, u_{k|t}) = 0, \\ & g(x_{k|t}, u_{k|t}) \leq 0. \end{aligned}$$

Remark (Computational challenge). NMPC is powerful because it handles nonlinear dynamics and constraints directly, but it is computationally demanding: a nonlinear program must be solved at every sampling instant, often within milliseconds.

Remark (Connection to iLQR). iLQR can be viewed as a specialized local solver for unconstrained or lightly constrained nonlinear finite-horizon optimal-control problems. In that sense, iLQR is a building block one may use inside NMPC, especially when Riccati-structured local approximations are preferable to generic sparse NLP solves.

Algorithm 19: Real-Time Iteration Scheme for NMPC

Input: Current state estimate $x(t)$, previous trajectory and multipliers, prediction model

1 Shift the previous solution forward in time to warm start the current problem
2 Linearize the dynamics and constraints, and quadratize the cost around the shifted trajectory
3 Solve one SQP-type QP subproblem or one local Riccati-structured approximation
4 Apply the first control move from the updated trajectory
5 At the next sampling instant, repeat rather than fully converging the NLP each time

Remark (Real-time iteration). The real-time iteration philosophy accepts that there may not be enough time to solve the nonlinear program to full convergence at each step. Instead, one performs one well-structured local optimization update per sample and relies on warm starts plus frequent feedback to keep the closed loop near the moving optimum.

9.6 MPC in Practice

Remark (QP solvers and timing constraints). Linear MPC reduces to a quadratic program, so real-time performance depends heavily on the QP solver. Active-set methods, operator-splitting methods, interior-point methods, and Riccati-based sparse solvers each occupy different regions of the speed-versus-accuracy tradeoff. The choice is dictated by sampling rate, horizon length, and available compute.

Remark (Warm starting). Successive MPC problems differ only slightly when the sampling period is short. Warm starting from the shifted previous solution can substantially reduce the number of iterations required by the underlying QP or NLP solver.

Remark (Explicit MPC). For constrained linear systems with modest dimension, one can solve the

parametric QP offline as a function of the current state. The result is an explicit control law

$$u(x) = F_i x + g_i$$

valid on polyhedral regions

$$x \in \mathcal{R}_i.$$

So explicit MPC realizes the optimizer as a piecewise-affine lookup law, trading online optimization for offline complexity and memory.

Remark. MPC is where the main threads of these notes converge:

- Section 3.3 supplied constrained optimization and KKT structure,
- Section 5.6 supplied transcription and numerical optimal-control machinery,
- Section 6.4 supplied the linear-quadratic terminal ingredients,
- Section 8.4 supplied local nonlinear solvers such as iLQR.

In that sense, MPC is not a separate island of theory. It is the operational synthesis of the entire subject.

A Attitude Control and Quaternion Optimization

Remark. Many robotic optimal-control problems live on nonlinear configuration spaces rather than Euclidean spaces. Attitude is the simplest and most important example: the physically meaningful state is a rotation, not an arbitrary vector in $\mathbb{R}^{3\times 3}$ or $\mathbb{R}^4$. This appendix summarizes the geometric constructions most often needed when trajectory optimization, estimation, and LQR are carried out with attitude states.

A.1 Rotation Matrices and Unit Quaternions

Definition A.1 (Special orthogonal group)**.** The set of three-dimensional rotation matrices is

$$SO(3) = \left\{ R \in \mathbb{R}^{3\times 3} : R^\top R = I,\ \det R = 1 \right\}.$$

Definition A.2 (Hat map)**.** For $\omega = (\omega_1, \omega_2, \omega_3)^\top \in \mathbb{R}^3$, define

$$\widehat{\omega} = \begin{bmatrix} 0 & -\omega_3 & \omega_2 \\ \omega_3 & 0 & -\omega_1 \\ -\omega_2 & \omega_1 & 0 \end{bmatrix}.$$

Then

$$\widehat{\omega} v = \omega \times v$$

for every $v \in \mathbb{R}^3$.

Definition A.3 (Unit quaternion)**.** A unit quaternion is a vector

$$q = \begin{bmatrix} q_0 \\ q_v \end{bmatrix} \in \mathbb{R}^4, \qquad q_0 \in \mathbb{R}, \quad q_v \in \mathbb{R}^3,$$

satisfying

$$q^\top q = q_0^2 + \|q_v\|^2 = 1.$$

The set of unit quaternions is the three-sphere

$$S^3 = \{ q \in \mathbb{R}^4 : q^\top q = 1 \}.$$

Remark (Double cover)**.** Every rotation in $SO(3)$ is represented by two antipodal unit quaternions:

$$q \qquad \text{and} \qquad -q.$$

They encode the same physical attitude. This is not a singularity, but it does matter numerically because optimization and filtering routines should maintain a consistent sign convention along a trajectory.

Definition A.4 (Quaternion-to-rotation map)**.** The rotation matrix corresponding to a unit quaternion

$$q = \begin{bmatrix} q_0 \\ q_v \end{bmatrix}$$

is

$$R(q) = (q_0^2 - q_v^\top q_v) I + 2 q_v q_v^\top + 2 q_0 \widehat{q_v}.$$

Definition A.5 (Quaternion kinematics)**.** If $\omega \in \mathbb{R}^3$ is the body angular velocity, then the quaternion

kinematics are

$$\dot{q} = \frac{1}{2} G(q)\omega,$$

where

$$G(q) = \begin{bmatrix} -q_v^\top \\ q_0 I + \widehat{q_v} \end{bmatrix} \in \mathbb{R}^{4\times 3}.$$

Proposition A.1 (Tangent-space property of the quaternion kinematics matrix). For every unit quaternion $q \in S^3$,

$$q^\top G(q) = 0$$

and

$$G(q)^\top G(q) = I_3.$$

Therefore the columns of $G(q)$ form an orthonormal basis for the tangent space

$$T_q S^3 = \{\delta q \in \mathbb{R}^4 : q^\top \delta q = 0\}.$$

Proof. Direct multiplication gives

$$q^\top G(q) = \begin{bmatrix} q_0 & q_v^\top \end{bmatrix} \begin{bmatrix} -q_v^\top \\ q_0 I + \widehat{q_v} \end{bmatrix} = -q_0 q_v^\top + q_0 q_v^\top + q_v^\top \widehat{q_v} = 0,$$

because

$$q_v^\top \widehat{q_v} = 0.$$

A second direct multiplication yields

$$G(q)^\top G(q) = q_v q_v^\top + (q_0 I - \widehat{q_v})(q_0 I + \widehat{q_v}) = (q_0^2 + \|q_v\|^2) I = I_3,$$

using

$$\widehat{q_v}^{\,2} = q_v q_v^\top - \|q_v\|^2 I$$

and the unit-norm condition on q. ■

A.2 Optimization Updates in the Quaternion Tangent Space

Remark. An unconstrained Newton or Gauss–Newton method wants to add a correction vector to the current iterate. For quaternions this is invalid: an arbitrary

$$q + \delta q$$

will not remain on S^3. The correction must instead be computed in a three-dimensional local coordinate chart and then mapped back to the manifold.

Definition A.6 (Small-angle quaternion update). Let $\phi \in \mathbb{R}^3$ be a small attitude correction. A corresponding first-order unit-quaternion update is

$$\delta q(\phi) \approx \begin{bmatrix} 1 \\ \frac{1}{2}\phi \end{bmatrix},$$

and the updated quaternion is obtained multiplicatively,

$$q^+ = \delta q(\phi) \otimes q,$$

followed by normalization if needed.

Remark (Why multiplicative updates are natural). Additive updates live in the ambient space $\mathbb{R}^4$. Multiplicative updates live on the group structure of rotations. The local coordinate $\phi \in \mathbb{R}^3$ is

unconstrained, while the updated quaternion remains on the unit sphere after renormalization.

Definition A.7 (Full-state projection matrix for error coordinates). Suppose the state contains position, quaternion attitude, linear velocity, and angular velocity:

$$x = \begin{bmatrix} r \\ q \\ v \\ \omega \end{bmatrix} \in \mathbb{R}^{13}.$$

The tangent-space error coordinate

$$\eta = \begin{bmatrix} \delta r \\ \delta\phi \\ \delta v \\ \delta\omega \end{bmatrix} \in \mathbb{R}^{12}$$

is lifted to an ambient perturbation by

$$\delta x = E(x)\eta,$$

with

$$E(x) = \operatorname{diag}\big(I_3, G(q), I_3, I_3\big) \in \mathbb{R}^{13\times 12}.$$

Proposition A.2 (Projected linearization in quaternion error coordinates). Let

$$x_{k+1} = f_k(x_k, u_k)$$

and linearize in the ambient coordinates:

$$\delta x_{k+1} = A_k \delta x_k + B_k \delta u_k.$$

If η_k and η_{k+1} are the tangent-space errors defined by

$$\delta x_k = E(\bar{x}_k)\eta_k, \qquad \eta_{k+1} = E(\bar{x}_{k+1})^\top \delta x_{k+1},$$

then the projected error dynamics are

$$\eta_{k+1} = \underbrace{E(\bar{x}_{k+1})^\top A_k E(\bar{x}_k)}_{A_k^\eta}\eta_k + \underbrace{E(\bar{x}_{k+1})^\top B_k}_{B_k^\eta}\delta u_k.$$

Proof. Substitute

$$\delta x_k = E(\bar{x}_k)\eta_k$$

into the ambient linearization:

$$\delta x_{k+1} = A_k E(\bar{x}_k)\eta_k + B_k \delta u_k.$$

Project the result back onto the tangent space at the next nominal state by multiplying with

$$E(\bar{x}_{k+1})^\top.$$

This gives the stated reduced linear dynamics directly. ■

A.3 Wahba's Problem and Gauss–Newton on S^3

Definition A.8 (Wahba's problem). Given known inertial-frame vectors

$$v_1^N, \ldots, v_m^N \in \mathbb{R}^3$$

and their corresponding body-frame measurements

$$v_1^B, \dots, v_m^B \in \mathbb{R}^3,$$

find the attitude q minimizing

$$J(q) = \frac{1}{2} \sum_{i=1}^{m} w_i \left\| R(q) v_i^B - v_i^N \right\|^2,$$

where $w_i > 0$ are weights.

Remark. Wahba's problem is the canonical example of optimization on a rotation manifold. It is also a clean model for attitude estimation from vector observations, such as star trackers, accelerometers, or magnetometers.

Algorithm 20: Gauss–Newton for Wahba's Problem in Quaternion Coordinates

Input: Initial unit quaternion q_0, vector correspondences (v_i^B, v_i^N), weights w_i

for $r = 0, 1, 2, \dots$ *until convergence* **do**

Compute residuals

$$r_i(q_r) = R(q_r) v_i^B - v_i^N$$

and stack them into $r(q_r)$

Linearize the residual with respect to a tangent-space correction $\phi \in \mathbb{R}^3$:

$$r(q_r \oplus \phi) \approx r(q_r) + J_\phi(q_r)\phi$$

Solve the normal equations

$$J_\phi(q_r)^\top J_\phi(q_r)\, \phi = -J_\phi(q_r)^\top r(q_r)$$

Form the multiplicative update $q_{r+1} = \delta q(\phi) \otimes q_r$

Renormalize q_{r+1} to unit length

Output: Estimated attitude quaternion

Remark (Role of the attitude Jacobian)**.** The Jacobian

$$J_\phi(q)$$

is the chain-rule Jacobian of the residual with respect to the three-dimensional correction variable ϕ, not with respect to the ambient four-dimensional quaternion coordinate. This is exactly the same philosophy as Proposition A.2: compute in the tangent space, then lift back to the manifold.

A.4 Quaternion Error-State LQR and Attitude Control

Definition A.9 (Quaternion error state)**.** Given a nominal quaternion $\bar{q}$ and actual quaternion q, define the error quaternion by

$$\delta q = \bar{q}^* \otimes q,$$

where $\bar{q}^*$ is the quaternion conjugate. For small errors,

$$\delta q \approx \begin{bmatrix} 1 \\ \frac{1}{2}\delta\phi \end{bmatrix},$$

so the three-vector $\delta\phi$ serves as the local attitude error coordinate.

Remark (Why naive LQR on the ambient quaternion is wrong)**.** A state containing a quaternion lives on a manifold of codimension one. A naive linearization in the ambient coordinate $\delta q \in \mathbb{R}^4$ creates a fictitious radial direction that changes $\|q\|$, but that direction is not physically actuated by the true dynamics. An LQR design built on the ambient linearization therefore solves the wrong local problem.

Algorithm 21: Quaternion Error-State LQR Design

Input: Nominal attitude trajectory $\{\bar{x}_k, \bar{u}_k\}$ or equilibrium, ambient linearization matrices (A_k, B_k), quadratic weights on tangent errors

1 Construct the projection matrices $E(\bar{x}_k)$ from Definition A.7

2 Project the ambient linearization onto tangent coordinates using Proposition A.2

3 Choose a quadratic cost in the reduced error state

$$\eta_k = \begin{bmatrix} \delta r_k \\ \delta \phi_k \\ \delta v_k \\ \delta \omega_k \end{bmatrix}$$

4 Solve the resulting reduced LQR problem using algorithm 14 or the algebraic Riccati equation from Theorem 6.3

5 Implement the feedback in quaternion error coordinates and renormalize the propagated quaternion state

Output: Locally optimal feedback law respecting the attitude manifold

Remark (Quadrotor attitude stabilization)**.** For a rigid body or quadrotor, the practical pattern is standard:

- propagate the full nonlinear quaternion dynamics,
- compute the attitude error through the quaternion product,
- feed the reduced error vector into an LQR or MPC law designed in tangent coordinates,
- renormalize the quaternion after numerical integration.

This is the manifold-aware version of linear quadratic feedback. The control law is linear in the local error coordinates, not in the raw quaternion itself.